BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT THÀNH PHỐ HỒ CHÍ MINH

KHOA ĐIỆN – ĐIỆN TỬ

BỘ MÔN KỸ THUẬT ĐIỆN TỬ - THÔNG TIN



BÁO CÁO CUỐI KỲ

MÔN HỌC: HỆ THỐNG NHÚNG - EMSY435664

MÃ LỚP HỌC: EMSY435664_01

HỌC KỲ I – NĂM HỌC 2026 - 2027

THIẾT KẾ VÀ MÔ PHỎNG HỆ THỐNG GIÁM SÁT MÔI TRƯỜNG VÀ ĐIỀU KHIỂN THIẾT BỊ NHÀ THÔNG MINH THỜI GIAN THỰC SỬ DỤNG ESP32-S3, FREERTOS VÀ WOKWI

Giảng viên hướng dẫn: **PGS. TS. Nguyễn Mạnh Hùng**

Sinh viên thực hiện: **Lê Quốc Khánh**

MSSV: 24161276

Nguyễn Hữu Đức

MSSV: 24161221

Lê Đình Hoàng

MSSV: 24161241

Thành phố Hồ Chí Minh, tháng 10 năm 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT THÀNH PHỐ HỒ CHÍ MINH

KHOA ĐIỆN – ĐIỆN TỬ

BỘ MÔN KỸ THUẬT ĐIỆN TỬ - THÔNG TIN



BÁO CÁO CUỐI KỲ

MÔN HỌC: HỆ THỐNG NHÚNG - EMSY435664

MÃ LỚP HỌC: EMSY435664_01

HỌC KỲ I – NĂM HỌC 2026 - 2027

THIẾT KẾ VÀ MÔ PHỎNG HỆ THỐNG GIÁM SÁT MÔI TRƯỜNG VÀ ĐIỀU KHIỂN THIẾT BỊ NHÀ THÔNG MINH THỜI GIAN THỰC SỬ DỤNG ESP32-S3, FREERTOS VÀ WOKWI

Giảng viên hướng dẫn: **PGS. TS. Nguyễn Mạnh Hùng**

Sinh viên thực hiện: **Lê Quốc Khánh**

MSSV: 24161276

Nguyễn Hữu Đức

MSSV: 24161221

Lê Đình Hoàng

MSSV: 24161241

Thành phố Hồ Chí Minh, tháng 10 năm 2026

DANH SÁCH NHÓM VIẾT BÁO CÁO CUỐI KỲ MÔN HỆ THỐNG NHÚNG

HỌC KỲ I – NĂM HỌC 2026 - 2027

1. Mã lớp môn học: EMSY435664_01

2. Giảng viên hướng dẫn: PGS. TS. Nguyễn Mạnh Hùng

3. Tên đề tài: Thiết kế và mô phỏng hệ thống giám sát môi trường và điều khiển thiết bị nhà thông minh thời gian thực sử dụng ESP32-S3, freertos và wokwi

4. Danh sách nhóm và phân công nhiệm vụ

STT	Họ và tên	MSSV	Nội dung thực hiện	Kí tên
01	**Lê Quốc Khánh**	24161276	*- Xây dựng kiến trúc tổng thể và thuật toán FSM.* *- Phân công xử lý dữ liệu giữa các FreeRTOS Task; tích hợp Queue, Semaphore và Mutex.* *- Kiểm tra hoạt động tổng thể, tổng hợp báo cáo.*	
02	**Nguyễn Hữu Đức**	24161221	*- Xây dựng SensorTask để đọc cảm biến và xử lý dữ liệu REAL.* *- Phát triển HTTP Web Server, Web Dashboard và các API.* *- Kiểm tra trao đổi dữ liệu giữa trình duyệt và ESP32-S3.*	
03	**Lê Đình Hoàng**	24161241	*- Thiết kế sơ đồ kết nối linh kiện trên Wokwi và xử lý ngắt PIR.* *- Xây dựng chức năng điều khiển relay, quạt PWM và buzzer.* *- Tích hợp OLED, kiểm tra thiết bị đầu ra và bổ sung minh chứng mô phỏng.*	

Nhận xét của giảng viên

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Ngày ..... tháng ..... năm 2026

Giáo viên chấm điểm

LỜI CAM ĐOAN

Nhóm chúng em xin cam đoan rằng đề tài nghiên cứu là trung thực. Những kết quả, số liệu và những thông tin phục vụ cho quá trình xử lý và hoàn thành bài nghiên cứu đều được thu thập từ nhiều nguồn khác nhau và được ghi rõ nguồn gốc. Tất cả thông tin về số liệu, bảng, hình ảnh và kết quả nghiên cứu đều được trình bày một cách minh bạch và chân thực, không chứa bất kỳ sự biến đổi hay điều chỉnh nào có thể làm ảnh hưởng đến tính chính xác của kết quả. Những số liệu chưa được đo trực tiếp trên mô phỏng sẽ không được trình bày vào báo cáo.

Trong đó những dữ liệu và thông tin được sử dụng trong nghiên cứu đã được thu thập từ các nguồn đáng tin cậy và được liệt kê trong phần tài liệu tham khảo và những tài liệu tham khảo và nghiên cứu trước đây hoặc những ý kiến cá nhân sử dụng đều đã được đặc tả một cách rõ ràng để đảm bảo tính minh bạch và khách quan. Nhóm sẽ chịu trách nhiệm về tính trung thực của nội dung và các kết luận trong phạm vi mô phỏng của đề tài.

LỜI CẢM ƠN

Nhóm xin gửi lời cảm ơn đến giảng viên là PGS. TS. Nguyễn Mạnh Hùng giảng dạy bộ môn Hệ thống nhúng lớp 261EMSY435664_01. Trong suốt quá trình học tập, Thầy đã rất tâm huyết giảng dạy và hướng dẫn cho em nhiều kiến thức bổ ích trong môn học như hướng dẫn về các phương pháp phân tích yêu cầu, tổ chức phần mềm nhúng và đánh giá hệ thống theo tư duy thời gian thực và các kỹ năng làm một bài nghiên cứu để chúng em có đủ kiến thức thực hiện bài báo cáo này. Quá trình thực hiện đề tài giúp nhóm củng cố kiến thức về ESP32, FreeRTOS, giao tiếp cảm biến, ngắt phần cứng, đồng bộ giữa các tác vụ và xây dựng giao diện giám sát trên nền tảng Web.

Tuy nhiên vì kiến thức của mỗi người trong nhóm còn nhiều hạn chế và sự tìm hiểu chưa sâu sắc nên không tránh khỏi những thiếu sót. Mong Thầy sẽ châm chước và cho chúng em những lời góp ý để bài nghiên cứu của chúng tôi sẽ hoàn thiện hơn. Một lần nữa, chúng em xin gửi lời cảm ơn sâu sắc đến Thầy và chúc Thầy luôn mạnh khỏe, hạnh phúc và thành công trong sự nghiệp.

Thành phố Hồ Chí Minh, tháng 10 năm 2026

TÓM TẮT

Đề tài trình bày quá trình thiết kế và mô phỏng hệ thống giám sát môi trường, điều khiển thiết bị nhà thông minh sử dụng ESP32-S3, FreeRTOS và Wokwi. Hệ thống thu thập nhiệt độ, độ ẩm từ cảm biến DHT22, mức sáng từ LDR và trạng thái chuyển động từ PIR. Dữ liệu được xử lý theo các luật điều khiển để lựa chọn một trong năm chế độ: COMFORT, COOLING, LIGHTING, ENERGY_SAVING và WARNING. Các chức năng được tổ chức thành những tác vụ FreeRTOS riêng, sử dụng Queue để trao đổi dữ liệu, Semaphore để xử lý sự kiện PIR và Mutex để bảo vệ dữ liệu dùng chung giữa các tác vụ.

ESP32-S3 điều khiển relay cho đèn, tạo tín hiệu PWM để mô phỏng hoạt động của quạt và kích hoạt buzzer khi cảnh báo. Trạng thái hệ thống được hiển thị trên OLED SSD1306 và Web Dashboard. Web Dashboard trao đổi dữ liệu với ESP32-S3 qua Wi-Fi và HTTP để giám sát cảm biến REAL và cấu hình ngưỡng COOLING, LIGHTING; ngưỡng WARNING cố định 45 °C. Đề tài cũng xây dựng các bước kiểm thử trên Wokwi; các kết quả định lượng sẽ được bổ sung sau khi đo trực tiếp trong quá trình mô phỏng.

Từ khóa: *ESP32-S3, FreeRTOS, Wokwi, cảm biến môi trường, nhà thông minh, HTTP, PWM.*

MỤC LỤC

CHƯƠNG 1 – GIỚI THIỆU TỔNG QUAN VỀ ĐỀ TÀI 14
1.1. Tình hình nghiên cứu 14
1.2. Đặt vấn đề 15
1.3. Lý do chọn đề tài 15
1.4. Mục tiêu đề tài 16
1.5. Nội dung nghiên cứu 16
1.6. Đối tượng và phạm vi nghiên cứu 17
1.7. Phương pháp nghiên cứu 17
CHƯƠNG 2 – CƠ SỞ LÝ THUYẾT 18
2.1. Hệ thống nhà thông minh 18
2.2. Tổng quan kiến trúc hệ thống 18
2.3. Vi điều khiển ESP32-S3 18
2.3.1. Các ngoại vi phần cứng và phương thức giao tiếp sử dụng trong hệ thống 20
2.3.2. Kiến trúc giao tiếp giữa phần mềm và phần cứng trên ESP32-S3 20
2.3.3. GPIO 22
2.3.4. ADC 24
2.3.5. LEDC và PWM 29
2.3.6. I2C 31
2.4. Cảm biến và thiết bị ngoại vi trong hệ thống 32
2.4.1. Các cảm biến DHT22, LDR và PIR 32
2.4.2. Các thiết bị đầu ra: Relay, buzzer, LED mô phỏng quạt và OLED 37
2.5. Hệ điều hành thời gian thực FreeRTOS 38
2.5.1. Cơ chế lập lịch, độ ưu tiên và trạng thái của Task 38
2.5.2. Queue 39
2.5.3. Binary Semaphore và Mutex 40
2.6. Giao tiếp mạng và Web Dashboard 41
2.6.1. Kết nối Wi-Fi và các giao thức TCP, IP, HTTP 41
2.6.2. Định dạng dữ liệu và cơ chế trao đổi với Web Dashboard 42
2.7. Nguyên lý lựa chọn chế độ theo luật ưu tiên 43
2.8. Nền tảng mô phỏng Wokwi và giới hạn mô phỏng 45
CHƯƠNG 3 – PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG NHÚNG THỜI GIAN THỰC 46
3.1. Phân tích yêu cầu hệ thống 46
3.1.1. Yêu cầu chức năng 46
3.1.2. Yêu cầu phi chức năng 47
3.2. Thiết kế kiến trúc tổng thể và phần cứng mô phỏng 47
3.2.1. Kiến trúc hệ thống và luồng xử lý dữ liệu 47
3.2.2. Sơ đồ kết nối và ánh xạ chân GPIO 48

3.2.3. Cấu trúc tổ chức mã nguồn 52
3.3. Thiết kế hệ thống đa tác vụ FreeRTOS 53
3.3.1. Khởi tạo và phân công các Task 53
3.3.2. Thu thập và xử lý dữ liệu trong SensorTask 54
3.3.3. Thiết kế Queue, Binary Semaphore, Mutex và xử lý ngắt PIR 56
3.4. Thiết kế thuật toán quyết định và điều khiển 58
3.4.1. Xây dựng luật xác định chế độ hoạt động 59
3.4.2. Thiết kế DecisionTask và ControlTask 60
3.5. Thiết kế giao diện giám sát và hiển thị 63
3.5.1. Thiết kế HTTP Web Server và Web Dashboard 63
3.5.2. Thiết kế DisplayTask và hiển thị OLED 65
3.6. Thiết kế cơ chế đo và đánh giá thời gian thực 67
CHƯƠNG 4 – THỰC NGHIỆM, KIỂM CHỨNG VÀ ĐÁNH GIÁ 70
4.1. Kết quả thiết kế phần cứng mô phỏng 70
4.2. Kết quả thiết kế phần mềm 71
4.3. Mô phỏng một số kịch bản 73
4.3.1. Kiểm thử cảm biến DHT22 và PIR 73
4.3.2. Kiểm thử mô hình LDR và chuyển đổi ADC 74
4.3.3. Kiểm thử các chế độ điều khiển 76
4.3.4. Kiểm chứng khả năng đáp ứng thời gian thực 80
4.4. Nhận xét và đánh giá kết quả 83
CHƯƠNG 5 – KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 84
5.1. Kết luận 84
5.2. Hướng phát triển 84
TÀI LIỆU THAM KHẢO 86

DANH MỤC HÌNH

Hình 2.1. Sơ đồ tổng thể hệ thống giám sát và điều khiển nhà thông minh 18
Hình 2.2. Sơ đồ chân của bo mạch ESP32-S3 DevKitC [7]. 19
Hình 2.3. Sơ đồ kiến trúc phân lớp phần mềm và phần cứng trên ESP32-S3 21
Hình 2.4. Cấu trúc phần cứng bên trong chân GPIO của ESP32-S3 ([5], Hình 6.3-2, tr. 474). 22
Hình 2.5. Vị trí các bit điều khiển GPIO6 và đọc tín hiệu GPIO16 trong thanh ghi GPIO 32 bit 23
Hình 2.6. Kiến trúc SAR ADC của ESP32-S3 (trích Hình 39.3-2 trong [5], trang 1467). 24
Hình 2.7. Cấu trúc các trường bit chính của thanh ghi điều khiển và cấu hình suy hao ADC1 trên ESP32-S3 (biên soạn theo [5]). 26
Hình 2.8. Quá trình chuyển đổi độ rọi thành điện áp và mã ADC trong đề tài 27
Hình 2.9. Mạch phân áp LDR nối với GPIO4 của ESP32-S3 27
Hình 2.10. Sơ đồ bộ tạo tín hiệu PWM của LEDC trên ESP32-S3 (trích Hình 35.3-1 trong [5], trang 1311). 29
Hình 2.11. Cấu trúc các trường bit chính của thanh ghi cấu hình timer, điều khiển kênh và thiết lập duty của LEDC trên ESP32-S3 31
Hình 2.12. Trình tự master ghi dữ liệu tới slave (nguồn: Espressif, ESP-IDF Programming Guide, “I2C Master Write”). 32
Hình 2.13. Trình tự master đọc dữ liệu từ slave (nguồn: Espressif, ESP-IDF Programming Guide, “I2C Master Read”). 32
Hình 2.14. Cảm biến nhiệt độ và độ ẩm DHT22 33
Hình 2.15. Mô-đun cảm biến ánh sáng LDR 33
Hình 2.16. Cảm biến chuyển động PIR 34
Hình 2.17. Dạng tín hiệu và khung dữ liệu 40 bit của DHT22 35
Hình 2.18. Ví dụ giải mã dữ liệu nhiệt độ và độ ẩm từ khung 40 bit của DHT22 35
Hình 2.19. Nguyên lý phát hiện chuyển động của cảm biến PIR HC-SR501 (nguồn: MakerGuides) 36
Hình 2.20. Sơ đồ mức logic PIR trên GPIO16 theo thông số mặc định của Wokwi 36
Hình 2.21. Các thiết bị đầu ra và hiển thị 37
Hình 2.22. Các trạng thái và chuyển đổi trạng thái cơ bản của Task trong FreeRTOS 38
Hình 2.23. Cơ chế trao đổi dữ liệu qua hai Queue một phần tử trong hệ thống 39
Hình 2.24. Minh họa Binary Semaphore (a) và Mutex (b) 40
Hình 2.25. Nguyên lý yêu cầu và phản hồi HTTP giữa trình duyệt và ESP32 41
Hình 2.26. Các yêu cầu HTTP phục vụ giám sát và cấu hình điều khiển trên Web Dashboard 42
Hình 2.27. Nguyên lý lựa chọn chế độ theo luật ưu tiên với hai ngưỡng cấu hình 44
Hình 2.28. Các thành phần chính của mô hình Wokwi 45
Hình 3.1. Sơ đồ kết nối toàn mạch ESP32-S3 trên Wokwi 49

Hình 3.7. Giao diện giám sát môi trường và cấu hình ngưỡng điều khiển trên Web Dashboard. 65
Hình 3.8. Kết quả hiển thị dữ liệu cảm biến và trạng thái hệ thống trên OLED SSD1306.. 67
Hình 4.1. Giao diện Wokwi và phần cứng mô phỏng của hệ thống..... 71
Hình 4.2. Web Dashboard giám sát cảm biến, cấu hình ngưỡng và hiển thị trạng thái điều khiển. 72
Hình 4.3. Bộ điều khiển DHT22 và kích hoạt cảm biến PIR trên Wokwi..... 74
Hình 4.4. Năm mức đo LDR trên Web Dashboard: (a) 5,0; (b) 50,1; (c) 501,5; (d) 5.006,0 và (e) 100.000,0 lux. 75
Hình 4.5. Kết quả WARNING: 46,3 °C, PIR = 0, buzzer bật và quạt 100%. 78
Hình 4.6. Kết quả COOLING: 31,6 °C, PIR = 1 và LED mô phỏng quạt 70%. 78
Hình 4.7. Trạng thái ENERGY_SAVING khi PIR = 0: các thiết bị đầu ra tắt. 79
Hình 4.8. Trạng thái COMFORT với PIR = 1, nhiệt độ 26,4 °C và ánh sáng 75,6%..... 79
Hình 4.9. Kết quả quan sát LIGHTING: 26,4 °C; 21,9 lux; PIR = 1; relay bật và quạt 0%.... 80
Hình 4.10. Năm mốc thống kê deadline từ Dashboard của source báo cáo. 82

DANH MỤC BẢNG BIỂU

Bảng 2.1. Tổng hợp thiết bị, kết nối và phương thức giao tiếp của hệ thống 20
Bảng 2.2. Các thanh ghi GPIO phục vụ giao tiếp tín hiệu số 23
Bảng 2.3. Các thanh ghi điều khiển ADC1 trên ESP32-S3 25
Bảng 2.4. Các thanh ghi LEDC liên quan đến việc tạo tín hiệu PWM trên ESP32-S3..... 30
Bảng 2.5. Phân biệt các cơ chế FreeRTOS được sử dụng trong đề tài [6], [16]. 41
Bảng 2.6. Các đường dẫn HTTP và dữ liệu trao đổi với Web Dashboard. 43
Bảng 3.1. Yêu cầu chức năng và thành phần mã nguồn minh chứng 47
Bảng 3.2. Các nhánh dữ liệu chính trong kiến trúc hệ thống..... 48
Bảng 3.3. Ánh xạ chân giữa ESP32-S3 và linh kiện mô phỏng..... 50
Bảng 3.2. Phân công tác vụ và thông số lập lịch của hệ thống..... 54
Bảng 3.5. Các trường dữ liệu trong cấu trúc SensorData 55
Bảng 3.6. Các đối tượng trao đổi dữ liệu và đồng bộ trong hệ thống. 57
Bảng 3.7. Các luật xác định chế độ hoạt động theo thứ tự ưu tiên. 59
Bảng 3.8. Trạng thái điều khiển thiết bị theo chế độ hoạt động. 61
Bảng 3.9. Các API HTTP phục vụ giám sát và cấu hình hệ thống. 63
Bảng 3.10. Các chỉ số đánh giá thời gian thực và phương pháp đo trong hệ thống..... 68
Bảng 4.1. Cấu hình dùng làm cơ sở kiểm thử..... 70
Bảng 4.2. Các HTTP API phục vụ giám sát và cấu hình 73
Bảng 4.3. Đối chiếu dữ liệu đầu vào từ DHT22 và PIR..... 74
Bảng 4.4. Đối chiếu các mức độ rọi đầu vào và kết quả chuyển đổi LDR..... 74
Bảng 4.5. Đối chiếu điều kiện thử nghiệm và các chế độ FSM 77
Bảng 4.6. Trạng thái đầu ra tương ứng các chế độ 77
Bảng 4.7. Các mốc thống kê deadline trên source báo cáo 81

DANH MỤC CÁC CHỮ VIẾT TẮT

Từ viết tắt	Tên tiếng Anh	Giải thích tiếng Việt
ADC	Analog-to-Digital Converter	Bộ chuyển đổi tín hiệu tương tự sang số
API	Application Programming Interface	Giao diện lập trình ứng dụng
AO	Analog Output	Ngõ ra tín hiệu tương tự của cảm biến LDR
CPU	Central Processing Unit	Bộ xử lý trung tâm
FSM	Finite State Machine	Máy trạng thái hữu hạn
FIFO	First In, First Out	Nguyên tắc vào trước, ra trước
GPIO	General Purpose Input/Output	Chân vào và ra số đa dụng
HAL	Hardware Abstraction Layer	Lớp trừu tượng phần cứng
HTTP	Hypertext Transfer Protocol	Giao thức truyền tải siêu văn bản
I2C	Inter-Integrated Circuit	Giao tiếp nối tiếp đồng bộ hai dây
IP	Internet Protocol	Giao thức Internet
ISR	Interrupt Service Routine	Trình phục vụ ngắt
JSON	JavaScript Object Notation	Định dạng dữ liệu trao đổi JSON
LDR	Light Dependent Resistor	Điện trở quang nhạy sáng
LED	Light-Emitting Diode	Điốt phát quang
LEDC	LED Control	Ngoại vi điều khiển tín hiệu PWM của ESP32-S3
LL	Low-Level	Lớp điều khiển mức thấp
NVS	Non-Volatile Storage	Bộ nhớ lưu trữ không mất dữ liệu
OLED	Organic Light-Emitting Diode	Màn hình điốt phát quang hữu cơ
PIR	Passive Infrared	Cảm biến hồng ngoại thụ động phát hiện chuyển động
PWM	Pulse Width Modulation	Điều chế độ rộng xung
RAM	Random Access Memory	Bộ nhớ truy cập ngẫu nhiên
RMSE	Root Mean Square Error	Căn bậc hai của sai số bình phương trung bình
SAR	Successive Approximation Register	Thanh ghi xấp xỉ liên tiếp trong ADC
SCL	Serial Clock Line	Đường xung đồng hồ của I2C
SDA	Serial Data Line	Đường dữ liệu của I2C
TCP	Transmission Control Protocol	Giao thức điều khiển truyền tải
Wi-Fi	Wi-Fi (tên công nghệ)	Công nghệ mạng không dây trong hệ thống

CHƯƠNG 1
GIỚI THIỆU TỔNG QUAN VỀ ĐỀ TÀI

1.1. Tình hình nghiên cứu

Nhà thông minh là hướng ứng dụng Internet vạn vật, trong đó cảm biến, bộ điều khiển và thiết bị điện tử phối hợp để theo dõi môi trường và hỗ trợ tự động hóa. Những đại lượng thường được quan tâm gồm nhiệt độ, độ ẩm, chất lượng không khí, mức chiếu sáng và trạng thái thiết bị.

Rhee và cộng sự tổng quan nghiên cứu về giám sát sức khỏe và môi trường trong nhà thông minh giai đoạn 2010 - 2021. Từ 4.476 bài được tìm thấy, nhóm tác giả phân tích 159 bài đáp ứng tiêu chí; 107 bài thuộc nhóm nghiên cứu ứng dụng, trong đó 26 bài về giám sát môi trường, gồm 16 bài theo dõi năng lượng và 10 bài về chất lượng không khí trong nhà [1]. Số liệu này cho thấy giám sát môi trường là một hướng nghiên cứu cụ thể trong nhà thông minh, làm căn cứ lựa chọn bài toán thu thập và xử lý dữ liệu cảm biến cho đề tài. Đây là kết quả tổng quan tài liệu, không phải số liệu đo của nhóm.

Bakare và Abubaker xây dựng hệ thống giám sát môi trường trong nhà dùng ESP32-WROOM và Wi-Fi. Bài báo báo cáo sai số khoảng 2% so với giá trị tham chiếu, độ trễ truyền 215–310 ms, tỷ lệ chuyển gói thành công 99,1%, cùng $R^2 > 0,98$ và RMSE < 0,5 [2]. Những chỉ số này được dẫn để minh họa các tiêu chí có thể đánh giá ở một hệ thống giám sát nối mạng: sai số, độ trễ và độ tin cậy truyền dữ liệu. Chúng thuộc cấu hình và quy trình thử nghiệm của bài báo, không thể xem là kết quả của mô hình ESP32-S3 trong đề tài.

Gao nghiên cứu hệ thống giám sát và điều khiển dùng STM32F103C8T6 kết hợp FreeRTOS. Bài báo mô tả thử nghiệm liên tục 24 giờ với chu kỳ lấy mẫu 1 giây; kết quả báo cáo sai số nhiệt độ tối đa 0,3 °C, sai số độ ẩm 2% RH, phản hồi tự động khoảng 0,8 ± 0,1 s và phản hồi từ xa khoảng 1,0 ± 0,1 s [3]. Các số liệu này cho thấy thời gian phản hồi, độ chính xác và tính ổn định đa tác vụ là những tiêu chí đáng xem xét. Do khác vi điều

khiển, cảm biến và môi trường thử nghiệm, chúng chỉ dùng làm căn cứ tham khảo, không phải mục tiêu đã đạt của đồ án.

Từ các nghiên cứu trên, đề tài tập trung vào việc thu thập nhiều loại dữ liệu, tổ chức xử lý bằng FreeRTOS và quan sát luồng dữ liệu trên giao diện. Mô hình sử dụng ESP32-S3, DHT22, LDR và PIR; dữ liệu được xử lý theo các điều kiện xác định trước để điều khiển relay, quạt PWM và buzzer. Kết quả định lượng của nhóm chỉ được trình bày sau khi chạy mô phỏng và đo theo phương pháp cụ thể.

1.2. Đặt vấn đề

Một bộ điều khiển nhà thông minh có thể đồng thời đọc cảm biến, xác định trạng thái môi trường, điều khiển thiết bị, cập nhật OLED và phục vụ giao diện web. Nếu thực hiện nối tiếp trong một vòng lặp, thao tác chờ hoặc xử lý kéo dài có thể làm chậm công việc khác. Vì vậy, chương trình cần phân chia chức năng và quy định cách truyền dữ liệu, đồng bộ sự kiện cũng như thứ tự xử lý.

Đề tài sử dụng FreeRTOS để tổ chức các task theo chức năng. Queue, semaphore và mutex được dùng theo vai trò trong mã nguồn để truyền dữ liệu hoặc đồng bộ phần dùng chung. Logic FSM lựa chọn COMFORT, COOLING, LIGHTING, ENERGY_SAVING hoặc WARNING dựa trên các điều kiện đã thiết lập. Việc kiểm thử cần đối chiếu đầu vào, trạng thái FSM và đầu ra thực tế; không mặc định rằng sử dụng FreeRTOS tự bảo đảm một thời gian phản hồi cụ thể.

1.3. Lý do chọn đề tài

Đề tài kết hợp nhiều nội dung của môn Hệ thống nhúng trong một mô hình có đầu vào, xử lý và đầu ra rõ ràng. DHT22 cung cấp dữ liệu nhiệt độ, độ ẩm; LDR cung cấp tín hiệu analog để đọc qua ADC; PIR phát hiện chuyển động. Relay, PWM, buzzer và OLED thể hiện các chức năng điều khiển và hiển thị.

ESP32-S3 đáp ứng yêu cầu kết nối cảm biến, điều khiển thiết bị và giao tiếp mạng; FreeRTOS hỗ trợ khảo sát lập lịch, đồng bộ và chia sẻ dữ liệu giữa các task. Wokwi cho

phép xây dựng, quan sát và kiểm thử mô hình mà không cần hoàn thiện phần cứng. Tuy nhiên, mô phỏng không thay thế phép đo cảm biến trên phần cứng, đánh giá an toàn điện hay kiểm tra độ tin cậy dài hạn. Các công trình [2] và [3] được dùng để nhận diện những tiêu chí thường được đánh giá, chứ không dùng làm bằng chứng rằng hệ thống của nhóm đạt các giá trị tương tự.

1.4. Mục tiêu đề tài

Mục tiêu tổng quát là thiết kế và mô phỏng hệ thống nhúng đa tác vụ sử dụng ESP32-S3 và FreeRTOS để giám sát một số thông số môi trường, điều khiển thiết bị cơ bản và hiển thị trạng thái hệ thống.

Mục tiêu cụ thể gồm: thu thập nhiệt độ, độ ẩm, ánh sáng và chuyển động; tổ chức chương trình thành các task; sử dụng Queue, Binary Semaphore và Mutex theo mã nguồn; xây dựng FSM với năm chế độ hoạt động; điều khiển relay, quạt PWM và buzzer; hiển thị dữ liệu trên OLED và Web Dashboard, cấu hình các ngưỡng COOLING và LIGHTING qua HTTP; mô phỏng, kiểm thử các trường hợp đầu vào; ghi nhận độ trễ xử lý PIR và thời gian phản hồi sau khi nhóm thực hiện phép đo.

1.5. Nội dung nghiên cứu

Đề tài tìm hiểu kiến trúc ESP32-S3, các giao tiếp của cảm biến và ngoại vi, cách tổ chức chương trình FreeRTOS và cơ chế truyền dữ liệu giữa các task. Trên cơ sở đó, nhóm thiết kế FSM, tích hợp cảm biến với thiết bị đầu ra, OLED và Web Dashboard, rồi xây dựng mô hình trên Wokwi.

Quá trình kiểm thử thay đổi đầu vào cảm biến trong Wokwi và cấu hình các ngưỡng điều khiển từ Web Dashboard theo từng trường hợp, sau đó quan sát Serial Monitor, OLED, giao diện và trạng thái thiết bị. Nhóm ghi lại đầu vào, chế độ FSM và đầu ra tương ứng. Giá trị điện áp suy ra từ ADC được ghi rõ là ước tính; số liệu thời gian được trình bày như kết quả mô phỏng khi có phép đo và minh chứng; phần chưa đo để "<đo>".

1.6. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu là hệ thống nhúng giám sát và điều khiển nhà thông minh dùng ESP32-S3, FreeRTOS, cảm biến DHT22, LDR, PIR, màn hình OLED, relay, quạt PWM và buzzer. Nội dung tập trung vào thu thập dữ liệu, xử lý và truyền dữ liệu giữa các task, lựa chọn chế độ bằng FSM, điều khiển thiết bị và hiển thị trạng thái.

Phạm vi thực hiện giới hạn ở thiết kế chương trình và mô phỏng trên Wokwi, bao gồm kiểm tra các trường hợp đầu vào đại diện và giao diện web theo chức năng đã lập trình. Đề tài chưa triển khai, đo kiểm trên phần cứng thực tế; vì vậy không kết luận về sai số cảm biến thực, mức tiêu thụ điện thực, độ an toàn tải điện hoặc độ ổn định dài hạn.

1.7. Phương pháp nghiên cứu

Nhóm nghiên cứu tài liệu về ESP32-S3, FreeRTOS, cảm biến, giao tiếp ngoại vi và Wokwi; các thông số phần cứng và API được đối chiếu với tài liệu chính thức tương ứng. Phương pháp phân tích và thiết kế được dùng để xác định đầu vào, chức năng xử lý, đầu ra, cấu trúc task và FSM.

Phương pháp lập trình và tích hợp được thực hiện trên Arduino Framework kết hợp FreeRTOS. Mô phỏng và kiểm thử được tiến hành trên Wokwi bằng cách thiết lập từng tình huống, quan sát trạng thái và lưu minh chứng. Khi đo thời gian hoặc điện áp, nhóm ghi rõ điểm đầu, điểm cuối, công cụ và điều kiện đo; không thay số đo chưa thực hiện bằng ước lượng.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT

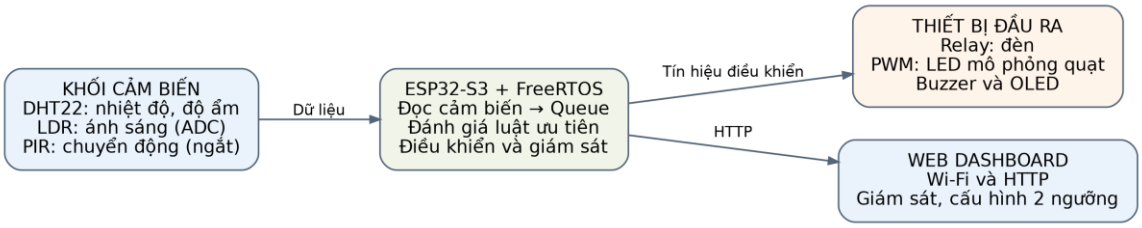
2.1. Hệ thống nhà thông minh

Nhà thông minh là mô hình ứng dụng công nghệ cảm biến, truyền thông và điều khiển để kết nối các thiết bị trong không gian sống, thu thập thông tin về môi trường hoặc hoạt động của người dùng, từ đó hỗ trợ giám sát, tự động hóa và điều khiển thiết bị. Theo Rhee và cộng sự, các ứng dụng tiêu biểu của nhà thông minh gồm giám sát môi trường, quản lý năng lượng, hỗ trợ an toàn và chăm sóc sức khỏe [1].

Trong phạm vi đề tài, mô hình nhà thông minh được tổ chức theo chuỗi đầu vào – xử lý – đầu ra. DHT22, LDR và PIR cung cấp dữ liệu môi trường hoặc trạng thái chuyển động; ESP32-S3 xử lý dữ liệu và lựa chọn chế độ điều khiển; relay, LED mô phỏng quạt, buzzer, OLED và Web Dashboard thể hiện kết quả. Cách phân chia này là cơ sở để lựa chọn các ngoại vi và cơ chế phần mềm được trình bày ở những mục tiếp theo.

2.2. Tổng quan kiến trúc hệ thống

Hệ thống gồm khối cảm biến, bộ điều khiển ESP32-S3 và các thiết bị đầu ra. DHT22, LDR và PIR cung cấp dữ liệu; ESP32-S3 sử dụng FreeRTOS và các luật ưu tiên để xử lý, điều khiển relay, LED PWM, buzzer và hiển thị OLED. Web Dashboard hỗ trợ giám sát và điều chỉnh hai ngưỡng điều khiển.

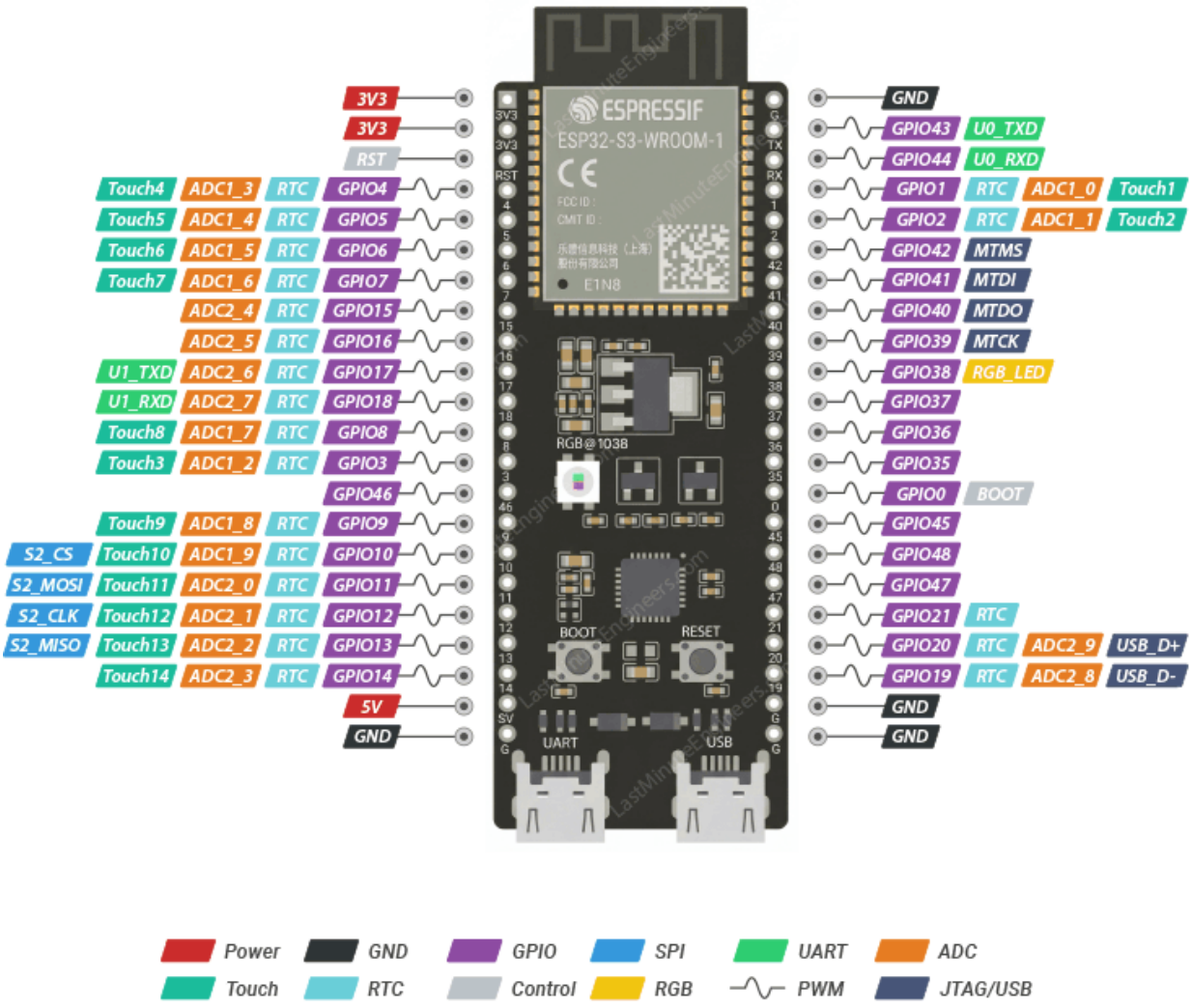


Hình 2.1. Sơ đồ tổng thể hệ thống giám sát và điều khiển nhà thông minh

2.3. Vi điều khiển ESP32-S3

ESP32-S3 là vi điều khiển tích hợp bộ xử lý Xtensa LX7 hai nhân, hỗ trợ kết nối Wi-Fi và các ngoại vi phục vụ ứng dụng hệ thống nhúng [4]. Trong đề tài, ESP32-S3 tiếp nhận dữ liệu cảm biến, xử lý điều kiện điều khiển, cập nhật thiết bị đầu ra và trao đổi dữ liệu

với Web Dashboard. Các khối ngoại vi được sử dụng gồm GPIO, ADC1, LEDC, I2C và phần giao tiếp mạng [5], [6]. Cấu trúc hai nhân tạo điều kiện tổ chức nhiều chức năng xử lý, nhưng khả năng đáp ứng thời gian thực còn phụ thuộc vào cách lập lịch, mức ưu tiên và sự đồng bộ giữa các tác vụ FreeRTOS. Do đó, phần giới thiệu phần cứng dưới đây tập trung vào những ngoại vi được dùng trực tiếp trong mô hình, thay vì liệt kê toàn bộ khả năng của vi điều khiển.



Hình 2.2. Sơ đồ chân của bo mạch ESP32-S3 DevKitC [7].

2.3.1. Các ngoại vi phần cứng và phương thức giao tiếp sử dụng trong hệ thống

Bảng 2.1 tổng hợp chân kết nối, phương thức giao tiếp và vai trò của các thiết bị sử dụng trong mô hình. Thuật ngữ "phương thức giao tiếp và điều khiển" bao gồm giao thức dữ liệu, tín hiệu tương tự, GPIO số và PWM; những loại tín hiệu này không phải đều là giao thức truyền thông [6].

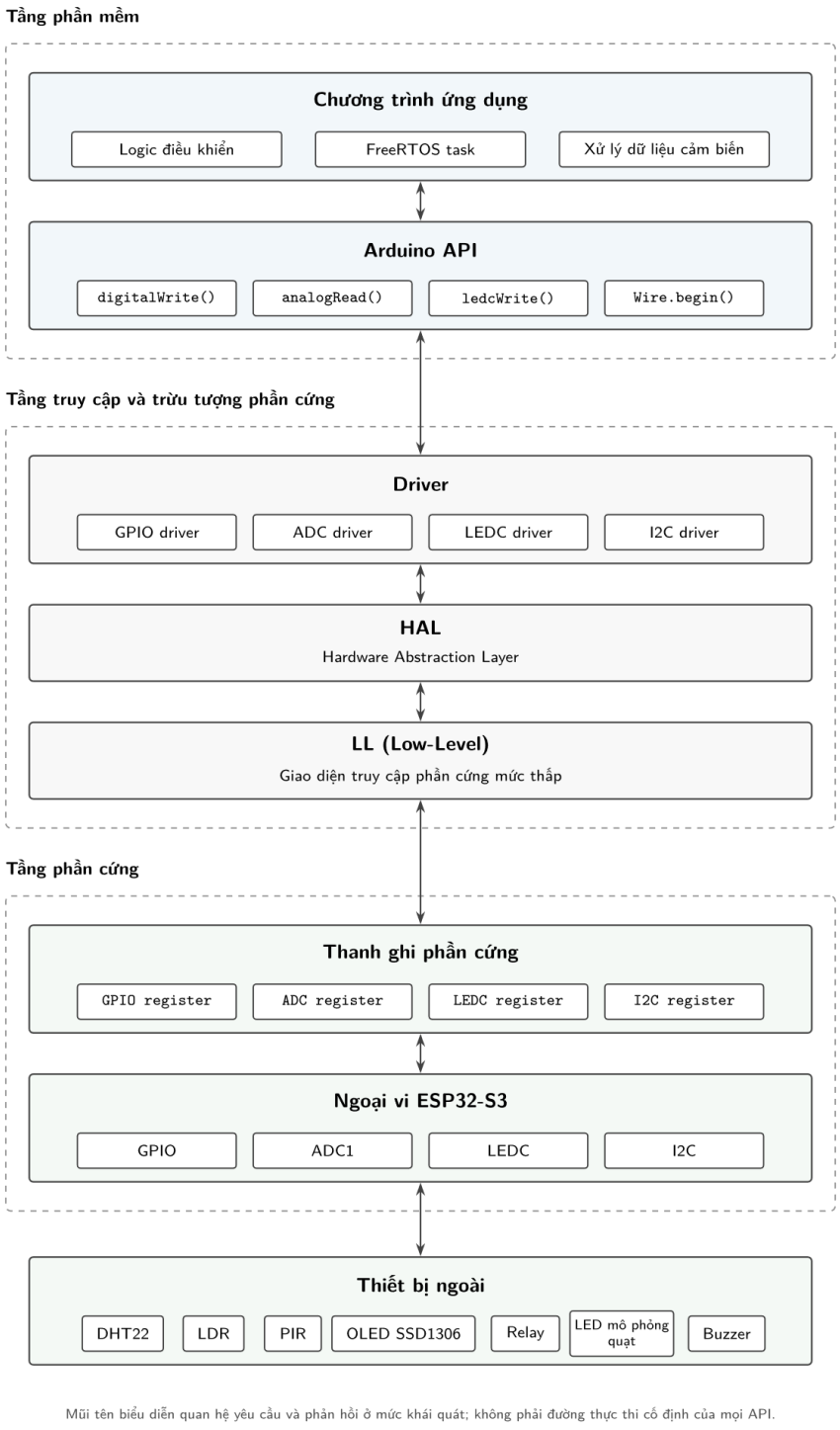
Thiết bị	**Chân kết nối**	**Phương thức giao tiếp và điều khiển**	**Vai trò**
Cảm biến DHT22	GPIO15	Giao thức dữ liệu một dây riêng của DHT22	Đo nhiệt độ, độ ẩm
Cảm biến LDR	GPIO4, ADC1_CH3	Tín hiệu điện áp tương tự	Đo mức ánh sáng
Cảm biến PIR	GPIO16	GPIO đầu vào số, hỗ trợ ngắt	Phát hiện chuyển động
Relay	GPIO6	GPIO đầu ra số	Bật, tắt đèn
LED mô phỏng cho thiết bị quạt	GPIO17	PWM do ngoại vi LEDC tạo ra, 5 kHz	Mô phỏng điều khiển quạt
Buzzer	GPIO7	Xung số tạo bằng tone()	Phát âm thanh cảnh báo
OLED SSD1306	SDA: GPIO8; SCL: GPIO9	Giao thức I2C	Hiển thị dữ liệu, trạng thái
Wi-fi tích hợp	Đã tích hợp trên ESP32-S3	Wi-Fi, TCP, IP và HTTP	Kết nối Web Dashboard

Bảng 2.1. Tổng hợp thiết bị, kết nối và phương thức giao tiếp của hệ thống

2.3.2. Kiến trúc giao tiếp giữa phần mềm và phần cứng trên ESP32-S3

Chương trình nhúng điều khiển ngoại vi chủ yếu thông qua các hàm Arduino API. Ở mức khái quát, quá trình truy cập phần cứng liên quan đến các thành phần driver, lớp trừu tượng phần cứng (HAL), mã điều khiển mức thấp và thanh ghi ngoại vi. Driver cung cấp các thao tác cấu hình và truyền nhận; các thành phần thấp hơn đảm nhiệm truy cập tài nguyên phần cứng. Hình 2.3 là sơ đồ minh họa những lớp chức năng này, không hàm

ý mọi lời gọi Arduino API đều đi qua một chuỗi Driver → HAL → LL giống hệt nhau [5], [6].

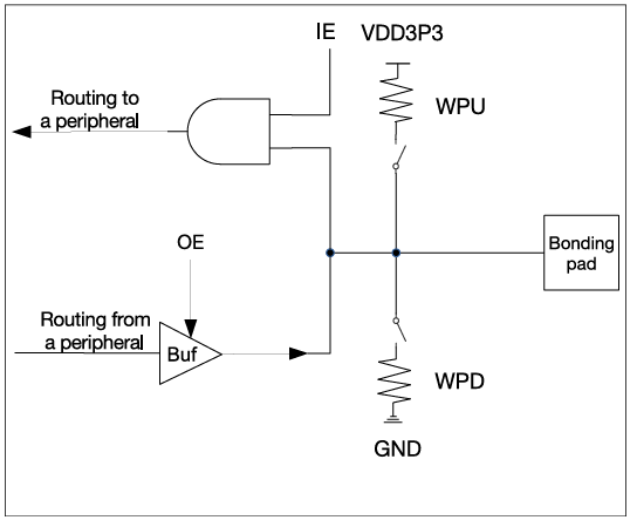


Hình 2.3. Sơ đồ kiến trúc phân lớp phần mềm và phần cứng trên ESP32-S3

Sơ đồ phân lớp giúp phân biệt lệnh chương trình, cơ chế truy cập ngoại vi và thiết bị vật lý. Đây không phải sơ đồ trao đổi dữ liệu giữa các task FreeRTOS; cấu trúc tác vụ và cơ chế đồng bộ thuộc phần kiến trúc phần mềm ứng dụng.

2.3.3. GPIO

GPIO (General Purpose Input/Output) là ngoại vi cho phép ESP32-S3 đọc và xuất tín hiệu số thông qua các chân được cấu hình bằng phần mềm. ESP32-S3 hỗ trợ 45 chân GPIO, có khả năng định tuyến tín hiệu qua GPIO Matrix và IO MUX. Các chân GPIO có thể cấu hình đầu vào, đầu ra, ngắt và điện trở kéo nội bộ. Cấu trúc phần cứng cơ bản được thể hiện trong Hình 2.4 [5].



Hình 2.4. Cấu trúc phần cứng bên trong chân GPIO của ESP32-S3 ([5], Hình 6.3-2, tr. 474).

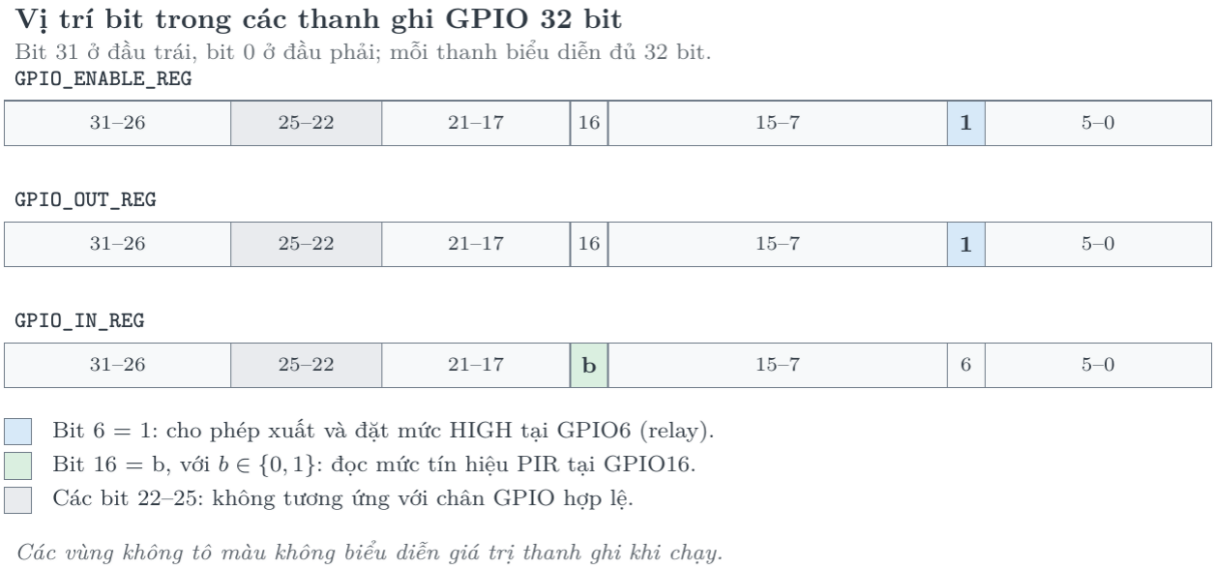
Trong hệ thống, GPIO16 tiếp nhận tín hiệu từ cảm biến PIR thông qua digitalRead() và attachInterrupt(). GPIO15 giao tiếp với DHT22 theo giao thức một dây riêng, truyền khung dữ liệu 40 bit gồm độ ẩm, nhiệt độ và byte kiểm tra. Ở chiều đầu ra, GPIO6 điều khiển relay bằng digitalWrite(), trong khi GPIO7 tạo xung 1000 Hz để kích hoạt buzzer thông qua tone() [6], [8], [9]. Ở mức phần cứng, GPIO được quản lý thông qua các thanh ghi điều khiển. Bảng 2.2 trình bày ba thanh ghi tiêu biểu có độ rộng 32 bit, phục vụ cấu hình đầu ra, xuất và đọc mức logic [5].

Thanh ghi	Địa chỉ	Chức năng
GPIO_ENABLE_REG	0x60004020	Cho phép chân GPIO hoạt động ở chế độ đầu ra.
GPIO_OUT_REG	0x60004004	Lưu mức logic HIGH hoặc LOW xuất ra chân GPIO.
GPIO_IN_REG	0x6000403C	Đọc mức logic hiện tại tại chân GPIO.

Bảng 2.2. Các thanh ghi GPIO phục vụ giao tiếp tín hiệu số

Trong đề tài, việc điều khiển GPIO được thực hiện thông qua Arduino API, không thao tác trực tiếp trên các thanh ghi phần cứng [5], [6].

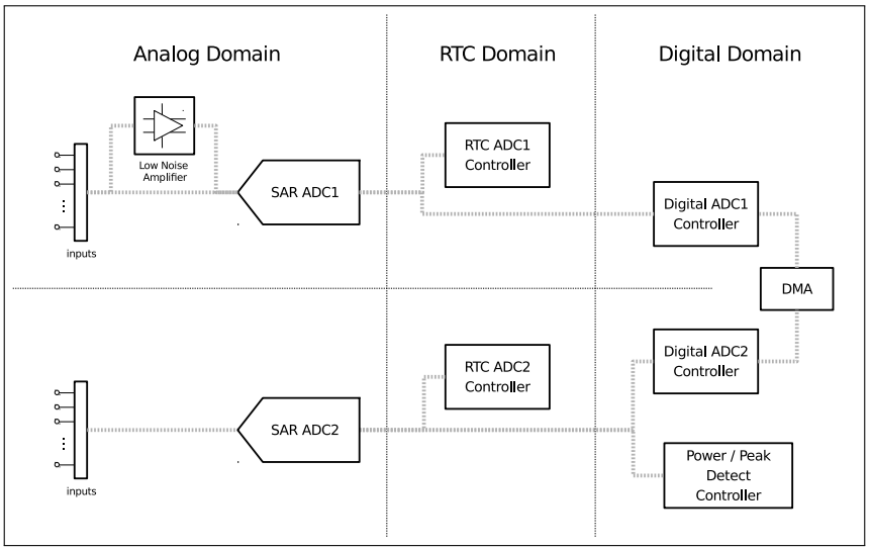
Mặt nạ nhị phân xác định vị trí bit thứ n được tính theo biểu thức $\boldsymbol{M^N = 2^N}$. Với GPIO6, mặt nạ tương ứng là 0x00000040 (hệ thập lục phân). Khi GPIO6 được cấu hình ở chế độ đầu ra và xuất mức HIGH, bit 6 của GPIO_ENABLE_REG và GPIO_OUT_REG được đặt bằng 1. Trong khi đó, bit 16 của GPIO_IN_REG phản ánh mức logic trên GPIO16 nối với PIR. Mặt nạ chỉ xác định vị trí thao tác, không biểu diễn toàn bộ giá trị thanh ghi; Hình 2.5 minh họa các vị trí bit liên quan [5].



Hình 2.5. Vị trí các bit điều khiển GPIO6 và đọc tín hiệu GPIO16 trong thanh ghi GPIO 32 bit

2.3.4. ADC

ADC (Analog-to-Digital Converter) chuyển tín hiệu điện áp tương tự thành dữ liệu số. ESP32-S3 tích hợp SAR ADC với độ phân giải 12 bit (mã từ 0 đến 4095). Trong đề tài, tín hiệu AO của LDR được đưa đến GPIO4 (ADC1_CH3) và đọc bằng analogRead(LDR_PIN). Hình 2.6 minh họa kiến trúc SAR ADC của ESP32-S3 [5], [6], [10].



Hình 2.6. Kiến trúc SAR ADC của ESP32-S3 (trích Hình 39.3-2 trong [5], trang 1467).

Ở mức phần cứng, các thanh ghi thuộc khối SENS hỗ trợ điều khiển phép chuyển đổi và cấu hình dải điện áp đầu vào. Bảng 2.3 giới thiệu hai thanh ghi ADC1 tiêu biểu; trong chương trình, việc đọc ADC được thực hiện thông qua Arduino API [5], [6].

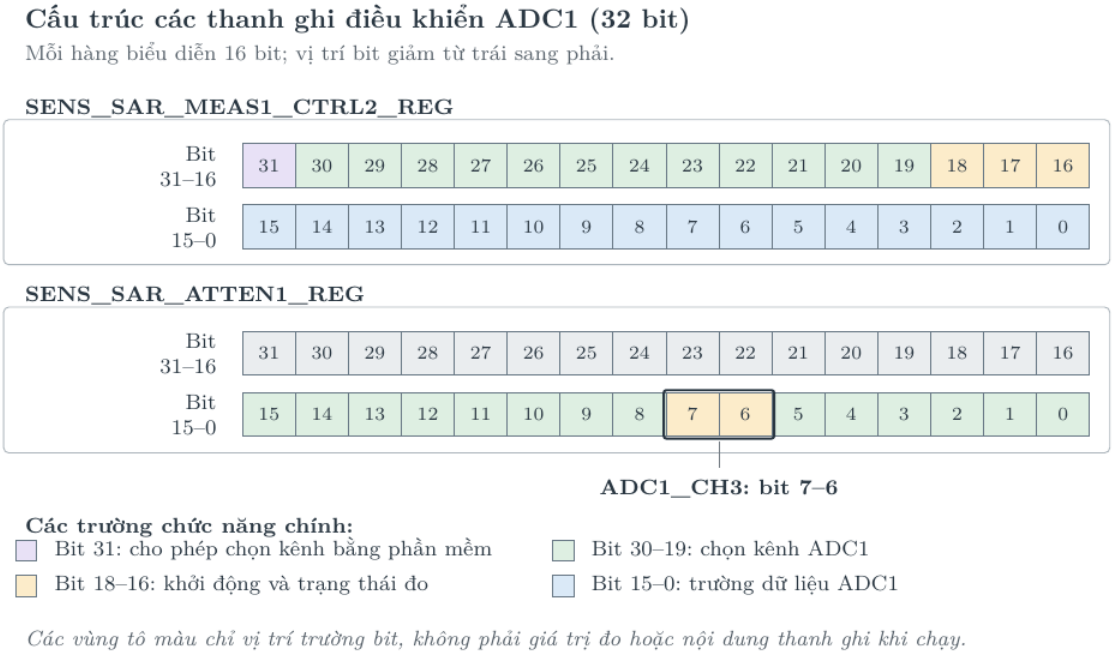
Thanh ghi	Địa chỉ	Chức năng
SENS_SAR_MEAS1_CTRL2_REG	0x6000880C	Điều khiển quá trình chuyển đổi ADC1, lựa chọn kênh đo và theo dõi kết quả chuyển đổi
SENS_SAR_ATTEN1_REG	0x60008814	Cấu hình mức suy hao điện áp đầu vào của các kênh ADC1

Bảng 2.3. Các thanh ghi điều khiển ADC1 trên ESP32-S3

Thanh ghi SENS_SAR_MEAS1_CTRL2_REG phục vụ điều khiển phép đo; SENS_SAR_ATTEN1_REG thiết lập mức suy hao để lựa chọn dải điện áp. Đối với ADC1_CH3, các bit 7 và 6 biểu diễn cấu hình suy hao. Phần mềm không trực tiếp ghi các bit này trong chương trình của nhóm [5], [6].

Hai thanh ghi đều có độ rộng 32 bit. Trong đó, SENS_SAR_MEAS1_CTRL2_REG chứa các trường bit phục vụ điều khiển phép đo và đọc kết quả, còn SENS_SAR_ATTEN1_REG sử dụng các trường 2 bit để cấu hình mức suy hao theo từng

kênh. Riêng ADC1_CH3 sử dụng bit 7 và bit 6 để thiết lập mức suy hao đầu vào. Cấu trúc các trường bit chính được minh họa trong Hình 2.7 [5]. kênh.



Hình 2.7. Cấu trúc các trường bit chính của thanh ghi điều khiển và cấu hình suy hao ADC1 trên ESP32-S3 (biên soạn theo [5]).

Từ độ rọi đến mã ADC. Trong mô hình Wokwi, ánh sáng tác động lên quang trở LDR làm thay đổi điện trở theo độ rọi E (lux). Quan hệ này được biểu diễn bằng phương trình (2.1) [11]:

$$R_{\mathrm{LDR}} = R_{L10}\left(\frac{10}{E}\right)^{\gamma}, \qquad E > 0 \tag{2.1}$$

Trong đó, R_{L10} = 50 kΩ là điện trở tại 10 lux và γ = 0,7 là hệ số đặc tuyến của LDR mặc định trên Wokwi. Giá trị điện trở chỉ được suy ra trực tiếp từ công thức khi E > 0 [11].

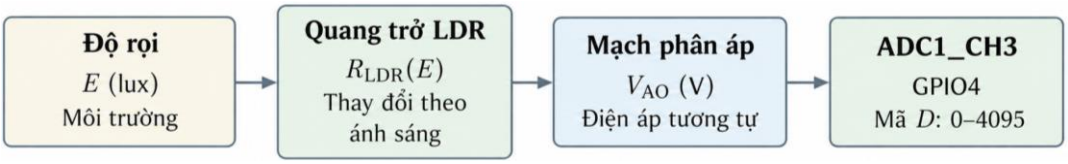
LDR mắc nối tiếp với điện trở cố định R_F = 10 kΩ. Điện áp tương tự tại điểm giữa hai điện trở (chân AO) được xác định theo mạch phân áp trong (2.2) [11]:

$$V_{\mathrm{AO}} = V_{\mathrm{CC}}\frac{R_{\mathrm{LDR}}}{R_F + R_{\mathrm{LDR}}} \tag{2.2}$$

Trong đó, V_{CC} = 3,3 V là điện áp cấp trong sơ đồ Wokwi. Khi độ rọi tăng, R_{LDR} giảm và V_{AO} cũng giảm. Chân AO nối với GPIO4 (ADC1_CH3); ADC 12 bit chuyển điện áp đầu vào thành mã số D từ 0 đến 4095 theo mô hình lý tưởng (2.3):

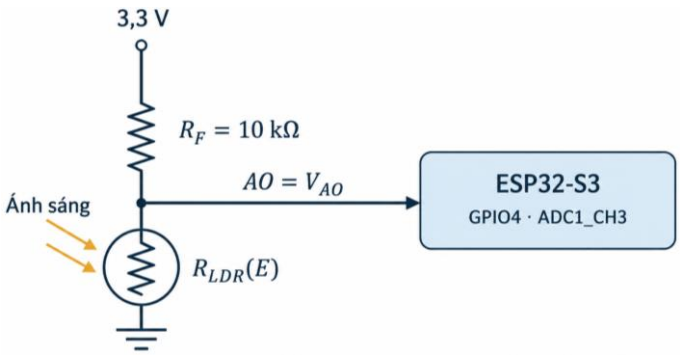
$$D \approx \text{clip}\left(\text{round}\left(\frac{4095\,V_{AO}}{V_{FS}}\right), 0, 4095\right) \quad (2.3)$$

Trong biểu thức (2.3), VFS là điện áp toàn thang và mã ADC được giới hạn từ 0 đến 4095. Chương trình sử dụng giá trị 3,3 V để ước tính điện áp từ mã ADC theo mô hình lý tưởng. Đây không phải điện áp đo trực tiếp trên phần cứng, do dải đo thực tếcủa ESP32-S3 còn phụ thuộc cấu hình suy hao và đặc tính ADC [5], [6], [10].



Hình 2.8. Quá trình chuyển đổi độ rọi thành điện áp và mã ADC trong đề tài

Hình 2.8 khái quát chuỗi biến đổi; Hình 2.9 thể hiện cụ thể mạch phân áp và đường tín hiệu AO nối đến ADC1_CH3 tại GPIO4. Hai hình bổ trợ cho nhau, không biểu diễn hai phép đo riêng biệt.



Hình 2.9. Mạch phân áp LDR nối với GPIO4 của ESP32-S3

Từ mã ADC đến độ rọi ước tính. Chương trình lấy D bằng analogRead(GPIO4), rồi suy ra điện áp AO theo phép quy đổi mô hình ở (2.3). Từ điện áp này, điện trở quang trở được tính ngược theo (2.4):

$$R_{\mathrm{LDR}} = R_F \frac{V_{\mathrm{AO}}}{V_{\mathrm{CC}} - V_{\mathrm{AO}}}, \qquad 0 < V_{\mathrm{AO}} < V_{\mathrm{CC}} \tag{2.4}$$

Thay R_{LDR} vào biểu thức đặc tuyến (2.1), độ rọi tương ứng được ước tính theo công thức (2.5):

$$E = 10 \left(\frac{R_{L10}}{R_{\mathrm{LDR}}}\right)^{1/\gamma} \text{ lux} \tag{2.5}$$

Hàm ldrFromAdc() sử dụng mã ADC để ước tính điện áp, điện trở, độ rọi và mức sáng tương đối. Khi ADC bằng 0, chương trình giới hạn độ rọi ở 100.000 lux; khi ADC bằng 4095, độ rọi bằng 0 và điện trở được giới hạn ở 1.000.000 kΩ. Các giá trị biên nhằm tránh phép chia cho 0, không phải kết quả đo vật lý chính xác.

Ví dụ tính toán. Với độ rọi E = 100 lux, R_{L10} = 50 kΩ, γ = 0,7, R_F = 10 kΩ và V_{CC} = V_{FS} = 3,3 V (giả định mô hình), lần lượt thế vào (2.1), (2.2) và (2.3) như sau:

$$R_{\mathrm{LDR}} = 50 \left(\frac{10}{100}\right)^{0,7} \approx 9,976 \text{ k}\Omega$$

$$V_{\mathrm{AO}} = 3,3 \times \frac{9,976}{10 + 9,976} \approx 1,648 \text{ V}$$

$$D \approx \mathrm{round}\left(\frac{4095 \times 1,648}{3,3}\right) \approx 2045$$

Như vậy, ở 100 lux, mã ADC lý tưởng xấp xỉ 2045. Đây là kết quả tính theo mô hình, không phải số đo điện áp trên phần cứng thực tế.

Chỉ số mức sáng tương đối dùng cho thuật toán điều khiển được chuẩn hóa từ mã ADC theo biểu thức sau, với 0% là tối và 100% là sáng.

$$P_{\text{sáng}} = \left(1 - \frac{D}{4095}\right) \times 100\%$$

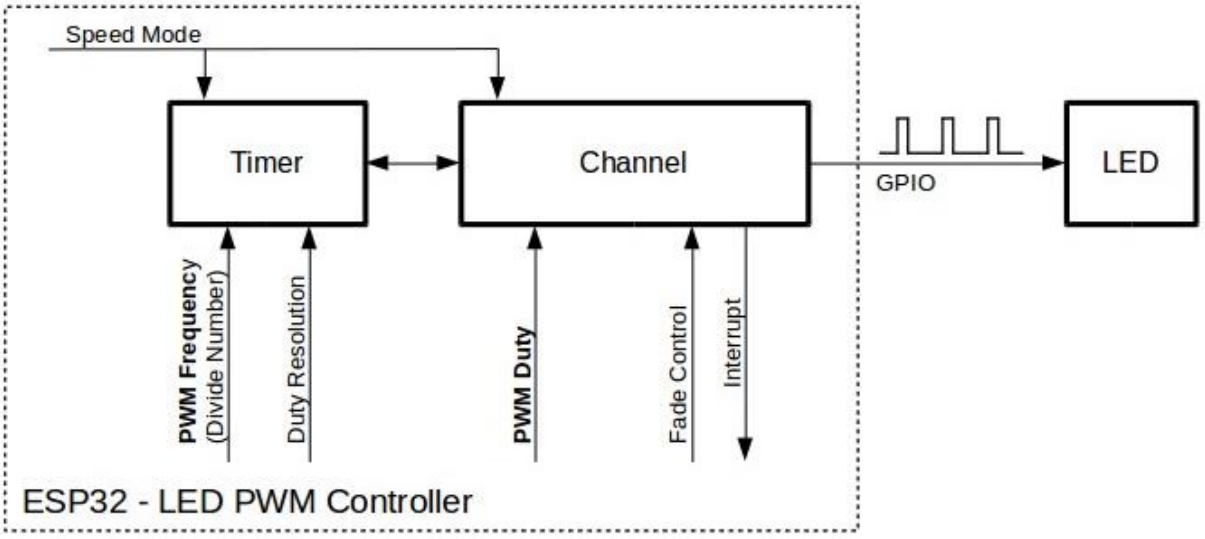
Trong đó, D nằm trong [0, 4095]; quy ước 0% là tối và 100% là sáng. DecisionTask so sánh chỉ số này với ngưỡng LIGHTING được cấu hình từ Web Dashboard; đây là tỷ lệ quy ước, không phải phép đo trực tiếp độ rọi. Các bước xử lý chương trình được trình bày ở Chương 3 [6].

2.3.5. LEDC và PWM

PWM (Pulse Width Modulation) là phương pháp điều chế độ rộng xung, thay đổi thời gian tín hiệu ở mức HIGH trong một chu kỳ. Ngoại vi LEDC trên ESP32-S3 tạo PWM bằng bộ định thời (Timer) và kênh (Channel); duty cycle được xác định theo (2.6) [5].

$$\delta = \frac{t_{\mathrm{HIGH}}}{T} \times 100\%, \qquad T = \frac{1}{f_{\mathrm{PWM}}} \quad (2.6)$$

Trong đó, t_{HIGH} là thời gian ở mức HIGH, T là chu kỳ tín hiệu và f_{PWM} là tần số PWM. Thay đổi duty cycle điều khiển mức tác động trung bình của tín hiệu, không trực tiếp đo được tốc độ quay của quạt.



Hình 2.10. Sơ đồ bộ tạo tín hiệu PWM của LEDC trên ESP32-S3 (trích Hình 35.3-1 trong [5], trang 1311).

Hình 2.10 minh họa LEDC: Timer thiết lập tần số và độ phân giải, còn Channel điều chỉnh giá trị duty cycle rồi xuất tín hiệu qua GPIO. Trong mô hình, chương trình sử dụng kênh 0, tần số 5 kHz, độ phân giải 8 bit và GPIO17 để điều khiển độ sáng LED mô phỏng mức hoạt động của quạt [5], [6].

Với độ phân giải N = 8 bit, giá trị điều khiển d được chọn từ 0 đến 255. Chu kỳ tín hiệu ở 5 kHz là T = 200 μs. Tỷ lệ HIGH lý tưởng tương ứng với (2.7):

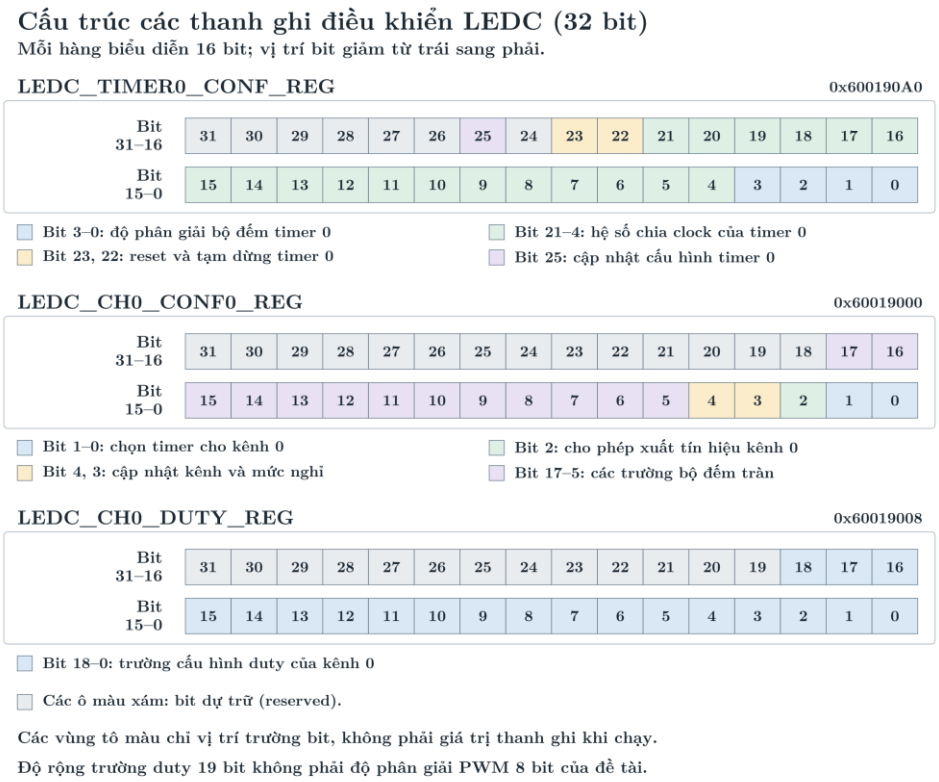
$$\delta \approx \frac{d}{2^N} \times 100\%, \qquad 0 \leq d \leq 2^N - 1 \quad (2.7)$$

Các mức d = 0, 180 và 255 tương ứng khoảng 0%, 70,31% và 99,61% thời gian HIGH theo công thức (2.7). Web Dashboard hiện hiển thị mức quạt theo d/255 × 100% và làm tròn thành số nguyên; đây là tỷ lệ điều khiển mô phỏng, không phải tốc độ động cơ đo được [5], [6]. Các thanh ghi LEDC dùng để cấu hình timer, kênh và duty cycle; Bảng 2.4 tổng hợp ba thanh ghi tiêu biểu liên quan đến kênh 0 [5].

Thanh ghi	**Địa chỉ**	**Độ rộng**	**Chức năng**
LEDC_TIMER0_CONF_REG	0x600190A0	32 bit	Thiết lập hệ số chia clock và độ phân giải của timer 0.
LEDC_CH0_CONF0_REG	0x60019000	32 bit	Lựa chọn timer và cho phép phát tín hiệu PWM ở kênh 0.
LEDC_CH0_DUTY_REG	0x60019008	32 bit	Lưu giá trị điều khiển duty cycle của kênh 0.

Bảng 2.4. Các thanh ghi LEDC liên quan đến việc tạo tín hiệu PWM trên ESP32-S3.

Ba thanh ghi LEDC trong Bảng 2.4 đều có độ rộng 32 bit. Trong đó, LEDC_TIMER0_CONF_REG chứa các trường cấu hình độ phân giải, hệ số chia clock và điều khiển timer 0; LEDC_CH0_CONF0_REG cho phép lựa chọn timer, bật đầu ra và cập nhật cấu hình kênh 0; LEDC_CH0_DUTY_REG chứa trường cấu hình duty tại các bit 18–0. Cấu trúc các trường bit chính được minh họa trong Hình 2.11 [5].



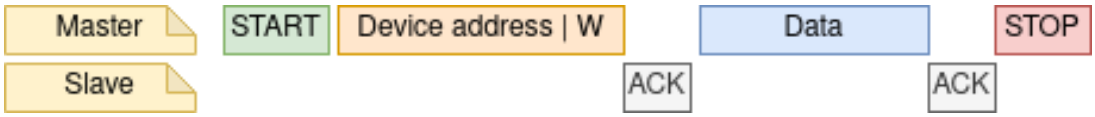
Hình 2.11. Cấu trúc các trường bit chính của thanh ghi cấu hình timer, điều khiển kênh và thiết lập duty của LEDC trên ESP32-S3

2.3.6. I2C

I2C (Inter-Integrated Circuit) là giao thức truyền thông nối tiếp đồng bộ, bán song công, sử dụng hai đường tín hiệu hai chiều: SDA (Serial Data) để truyền dữ liệu và SCL (Serial Clock) để tạo xung đồng hồ. Hai đường này hoạt động theo kiểu open-drain và cần điện trở kéo lên. Một bus có thể kết nối nhiều thiết bị, mỗi thiết bị được nhận diện bằng địa chỉ I2C, thường dài 7 bit [5].

Trong đề tài, ESP32-S3 đóng vai trò master, còn màn hình OLED SSD1306 là slave. Theo sơ đồ kết nối, SDA sử dụng GPIO8 và SCL sử dụng GPIO9. ESP32-S3 phát xung SCL và gửi dữ liệu hiển thị đến OLED; do đó, giao dịch ghi là thao tác chính cần xét trong hệ thống [5], [6].

Khi ghi dữ liệu, master phát điều kiện START, gửi địa chỉ thiết bị cùng bit W (ghi), sau đó truyền các byte dữ liệu. Slave xác nhận bằng tín hiệu ACK sau byte địa chỉ và từng byte dữ liệu nhận thành công; giao dịch kết thúc bằng điều kiện STOP. Trình tự được thể hiện trong Hình 2.12.



Hình 2.12. Trình tự master ghi dữ liệu tới slave (nguồn: Espressif, ESP-IDF Programming Guide, "I2C Master Write").

Trong giao dịch đọc, master gửi địa chỉ cùng bit R (đọc), sau đó slave phát dữ liệu trên SDA. Master xác nhận các byte còn cần đọc bằng ACK và kết thúc lần đọc bằng NACK trước điều kiện STOP như Hình 2.13. Đây là nguyên lý chung của I2C, không hàm ý OLED trong đề tài phải thực hiện giao dịch đọc.



Hình 2.13. Trình tự master đọc dữ liệu từ slave (nguồn: Espressif, ESP-IDF Programming Guide, "I2C Master Read").

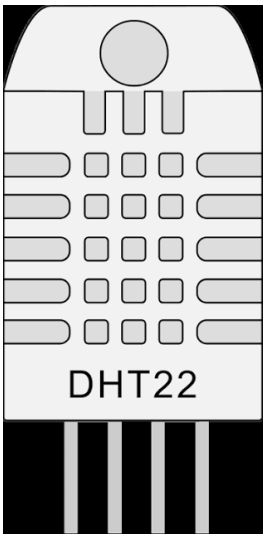
Theo Espressif, I2C của ESP32-S3 hỗ trợ Standard-mode đến 100 kHz và Fast-mode đến 400 kHz. Tốc độ bus thực tế phụ thuộc cấu hình phần mềm cùng đặc tính điện trở kéo lên và điện dung đường truyền; các giá trị 100 kHz và 400 kHz là mức hỗ trợ của giao thức, không phải kết quả đo trong Wokwi. Trong chương trình, việc truy cập OLED được thực hiện thông qua Arduino Wire và thư viện điều khiển màn hình, thay vì trực tiếp cấu hình các thanh ghi I2C [5], [6].

2.4. Cảm biến và thiết bị ngoại vi trong hệ thống

2.4.1. Các cảm biến DHT22, LDR và PIR

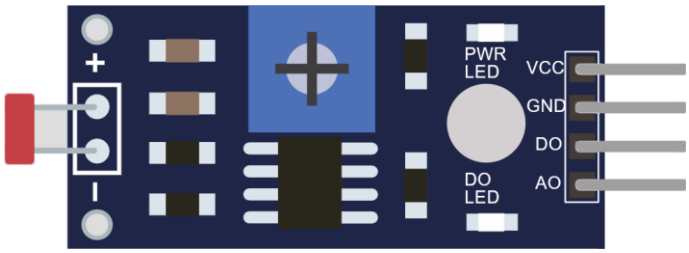
DHT22 đo nhiệt độ và độ ẩm qua GPIO15, truyền dữ liệu bằng khung 40 bit. SensorTask sử dụng thư viện DHTesp để đọc và kiểm tra giá trị bằng isfinite(). Khi dữ

liệu mới không hợp lệ, chương trình giữ giá trị hợp lệ gần nhất; nếu chưa có đủ dữ liệu hợp lệ, SensorTask chưa cập nhật sensorQueue [6], [12].



Hình 2.14. Cảm biến nhiệt độ và độ ẩm DHT22

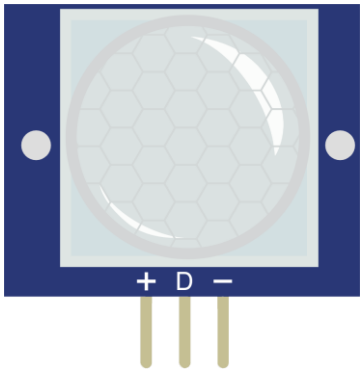
LDR (Light Dependent Resistor) là quang trở có điện trở thay đổi theo độ rọi. Mô-đun LDR trên Wokwi xuất điện áp tương tự tại chân AO đưa đến GPIO4 (ADC1_CH3) của ESP32-S3. Mã ADC được chương trình quy đổi thành mức sáng tương đối 0–100% (0% là tối, 100% là sáng); từ mô hình phân áp và đặc tuyến quang trở Wokwi, chương trình cũng ước tính điện áp, điện trở và độ rọi (lux). Các giá trị ước tính này không thay thế phép đo đã hiệu chuẩn. Quan hệ giữa độ rọi, điện trở, điện áp và mã ADC đã được phân tích ở mục 2.3.4 [6], [11].



Hình 2.15. Mô-đun cảm biến ánh sáng LDR

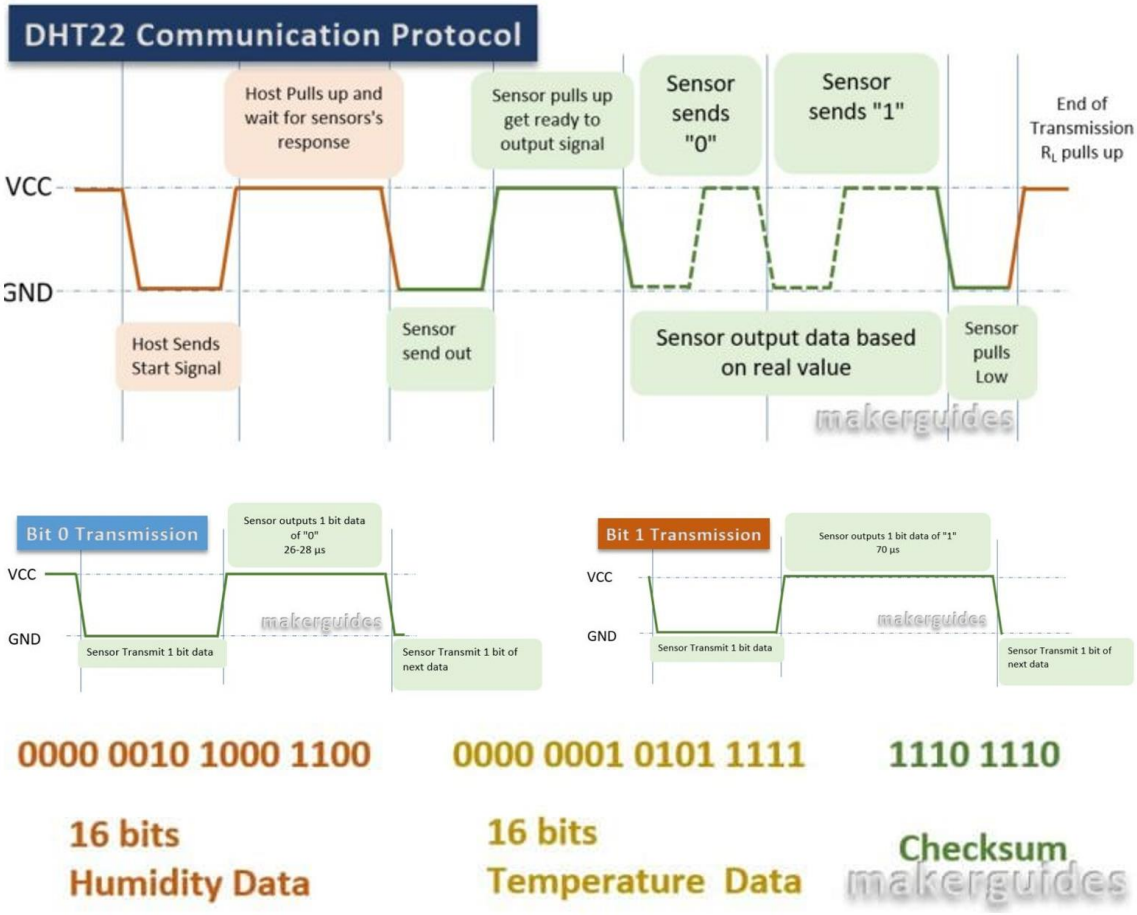
PIR phát hiện chuyển động thông qua sự thay đổi bức xạ hồng ngoại, với chân OUT nối GPIO16. Chương trình sử dụng ngắt CHANGE để phát hiện thay đổi mức logic. ISR

phát Binary Semaphore cho PIRTask; tác vụ này đọc trạng thái mới, cập nhật sensorQueue nếu đã có mẫu và thông báo cho DecisionTask. Tín hiệu PIR không khẳng định sự hiện diện liên tục của con người[6].

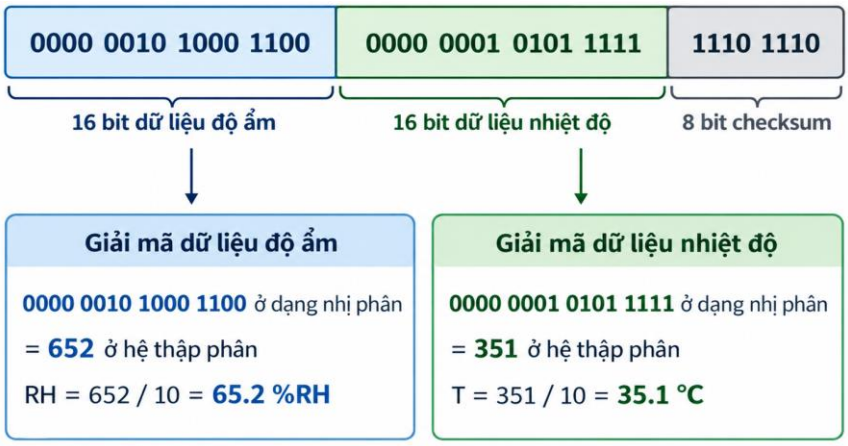


Hình 2.16. Cảm biến chuyển động PIR

Đối với DHT22, mỗi bit dữ liệu được truyền trên đường DATA bằng khoảng LOW xấp xỉ 50 µs, sau đó là mức HIGH khoảng 26–28 µs đối với bit 0 hoặc khoảng 70 µs đối với bit 1. Khung 40 bit gồm 16 bit độ ẩm, 16 bit nhiệt độ và 8 bit kiểm tra checksum; chương trình sử dụng DHTesp để giải mã thành °C và %RH. Hình 2.17 minh họa trình tự truyền và cấu trúc dữ liệu từ hình gốc của MakerGuides.

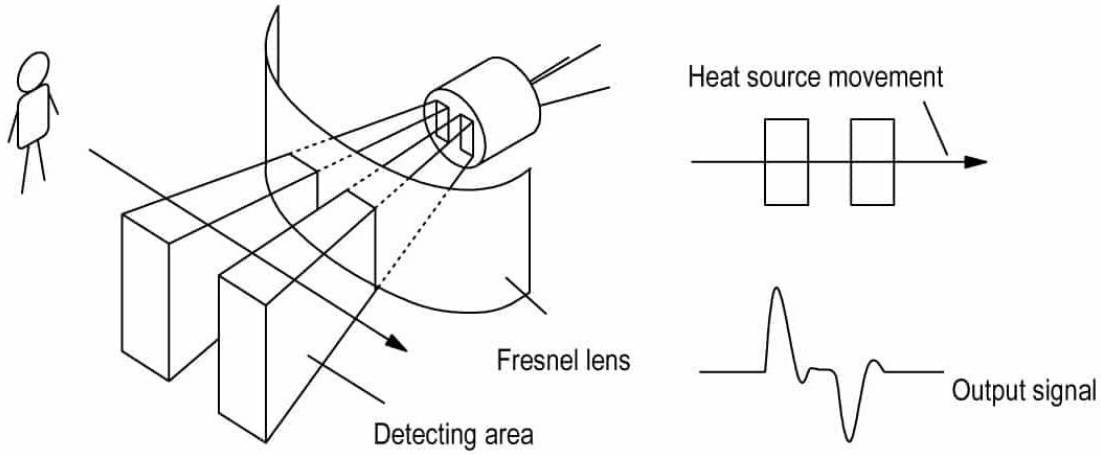


Hình 2.17. Dạng tín hiệu và khung dữ liệu 40 bit của DHT22



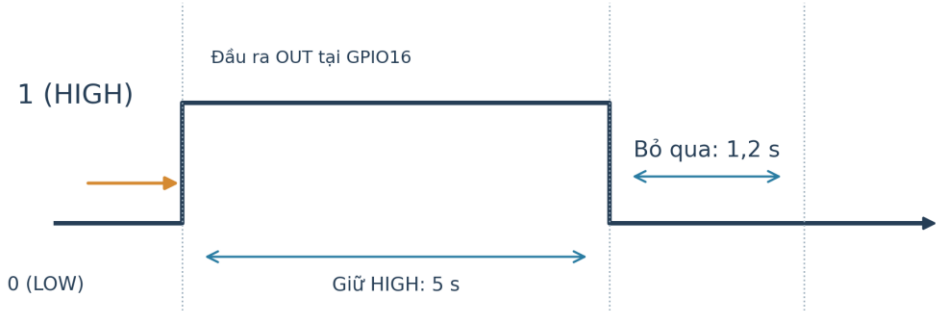
Hình 2.18. Ví dụ giải mã dữ liệu nhiệt độ và độ ẩm từ khung 40 bit của DHT22

Với PIR, thấu kính Fresnel hướng bức xạ hồng ngoại tới hai phần tử cảm nhận. Khi nguồn nhiệt di chuyển qua vùng quan sát, mức hồng ngoại thay đổi không đồng đều, tạo tín hiệu biến thiên được mạch xử lý chuyển thành mức logic trên chân OUT. Hình 2.19 trình bày nguyên lý này theo tài liệu MakerGuides; đây là nguyên lý cảm nhận, không phải dạng xung số tại GPIO16.



Hình 2.19. Nguyên lý phát hiện chuyển động của cảm biến PIR HC-SR501 (nguồn: MakerGuides).

Trong mô hình Wokwi đang sử dụng, chân OUT của PIR đưa mức HIGH về GPIO16 khi kích hoạt sự kiện chuyển động, rồi trở về LOW. Theo cấu hình mặc định, thời gian giữ HIGH là 5 s và thời gian bỏ qua sự kiện tiếp theo là 1,2 s; các thông số này có thể được thay đổi bằng thuộc tính của phần tử mô phỏng. Hình 2.20 chỉ biểu diễn mức logic tại GPIO16, không phải phép đo dạng sóng điện thực tế [13].



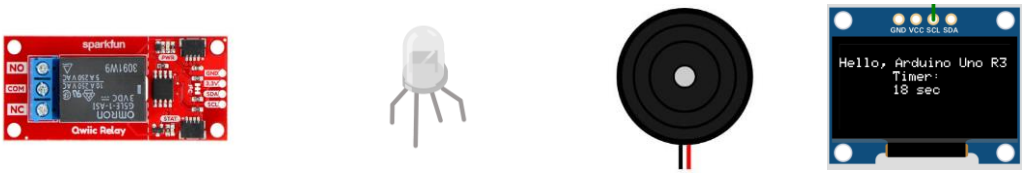
Hình 2.20. Sơ đồ mức logic PIR trên GPIO16 theo thông số mặc định của Wokwi

2.4.2. Các thiết bị đầu ra: Relay, buzzer, LED mô phỏng quạt và OLED

Relay là phần tử đóng hoặc ngắt tải đèn theo tín hiệu điều khiển số. Trong mô hình, GPIO6 xuất mức HIGH để kích hoạt relay, còn mức LOW để ngắt; đây là quy ước active-high của cấu hình đang mô phỏng. Buzzer kết nối GPIO7 tạo âm báo 1 kHz khi hệ thống ở chế độ WARNING, qua đó thể hiện chức năng cảnh báo bằng âm thanh [6].

Thiết bị quạt được biểu diễn bằng một LED nhận tín hiệu PWM tại GPIO17. Ngoại vi LEDC được cấu hình tần số 5 kHz và độ phân giải 8 bit; thay đổi giá trị duty từ 0 đến 255 làm thay đổi độ sáng của LED, dùng để minh họa mức điều khiển quạt. Mức PWM không được xem là tốc độ quay thực tế vì hệ thống không mô phỏng hoặc đo RPM của động cơ. Nguyên lý LEDC và PWM đã được trình bày ở mục 2.3.5 [5], [6].

OLED SSD1306 có độ phân giải 128 × 64 pixel, giao tiếp I2C qua GPIO8 (SDA) và GPIO9 (SCL), địa chỉ 0x3C. DisplayTask cập nhật mỗi 500 ms, hiển thị nhiệt độ, độ ẩm, mức sáng, PIR, chế độ điều khiển, phần trăm PWM của quạt và trạng thái đèn. Các thông số LDR chi tiết được theo dõi trên Web Dashboard. Nguyên lý giao tiếp I2C được trình bày ở mục 2.3.6 [6], [14], [15].



(a) Relay *(b) LED mô phỏng quạt* *(c) Buzzer* *(d) OLED SSD1306*

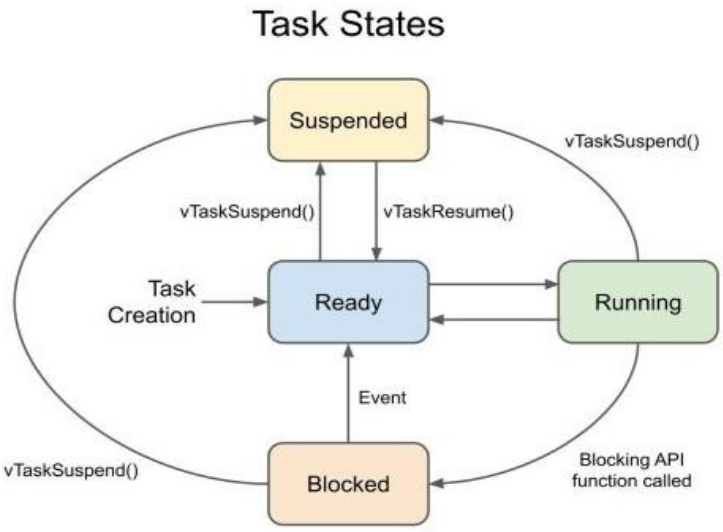
Hình 2.21. Các thiết bị đầu ra và hiển thị

2.5. Hệ điều hành thời gian thực FreeRTOS

FreeRTOS là nhân hệ điều hành thời gian thực, cho phép tổ chức ứng dụng nhúng thành nhiều tác vụ (Task) và quản lý thời gian sử dụng bộ xử lý thông qua bộ lập lịch. Trên ESP32-S3, FreeRTOS được tích hợp trong nền tảng phần mềm của Espressif để phục vụ xử lý đa tác vụ. Trong đề tài, các chức năng đọc cảm biến, quyết định chế độ, điều khiển thiết bị, xử lý PIR, hiển thị OLED và phục vụ Web được phân chia thành những Task riêng [6], [16], [17].

2.5.1. Cơ chế lập lịch, độ ưu tiên và trạng thái của Task

Bộ lập lịch ưu tiên các Task ở trạng thái sẵn sàng (Ready) có độ ưu tiên cao, đồng thời cho phép ngắt quyền thực thi của Task hiện tại khi Task ưu tiên hơn có thể chạy. Các Task cùng mức ưu tiên có thể chia sẻ thời gian CPU bằng cơ chế time slicing. ESP32-S3 có hai nhân nên việc chọn Task còn phụ thuộc khả năng chạy trên từng nhân và cấu hình của hệ thống; không thể suy ra thứ tự chạy tuyệt đối chỉ từ mức ưu tiên [16], [17].



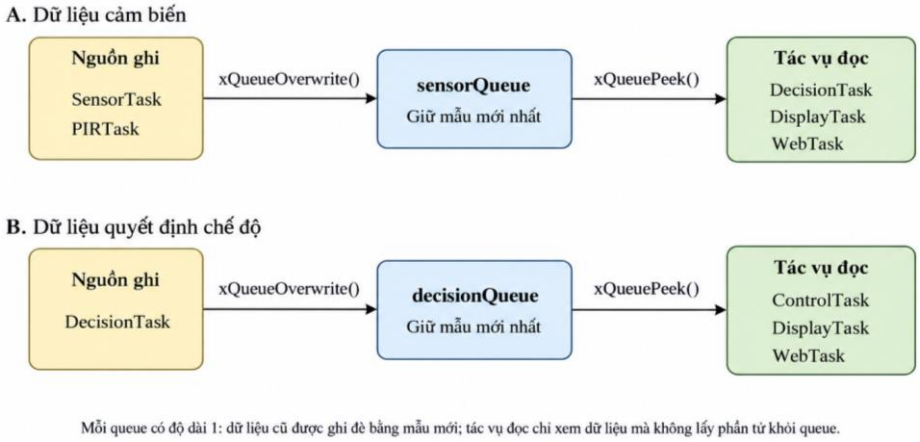
Hình 2.22. Các trạng thái và chuyển đổi trạng thái cơ bản của Task trong FreeRTOS

Trong đó, Running là trạng thái Task đang thực thi; Ready là trạng thái sẵn sàng chờ CPU; Blocked là trạng thái chờ thời gian hoặc sự kiện; Suspended là trạng thái tạm dừng theo yêu cầu. Task ở trạng thái Blocked có thể trở về Ready khi điều kiện chờ được đáp ứng [16].

Mã nguồn tạo sáu Task bằng xTaskCreate(): PIRTask ưu tiên 4, ControlTask ưu tiên 3; SensorTask và DecisionTask ưu tiên 2; DisplayTask và WebTask ưu tiên 1. Các tác vụ định kỳ dùng vTaskDelay() để nhường CPU trong khoảng chờ; riêng PIRTask có thể bị chặn để đợi sự kiện ngắt. Độ ưu tiên giúp tổ chức xử lý nhưng không tự bảo đảm mọi thời hạn đáp ứng [6], [17].

2.5.2. Queue

Queue là vùng đệm của FreeRTOS để trao đổi dữ liệu giữa các Task thông qua thao tác gửi, nhận và xem. Task có thể chờ dữ liệu khi cần; với Queue nhiều phần tử, thứ tự mặc định là FIFO [16].



Hình 2.23. Cơ chế trao đổi dữ liệu qua hai Queue một phần tử trong hệ thống

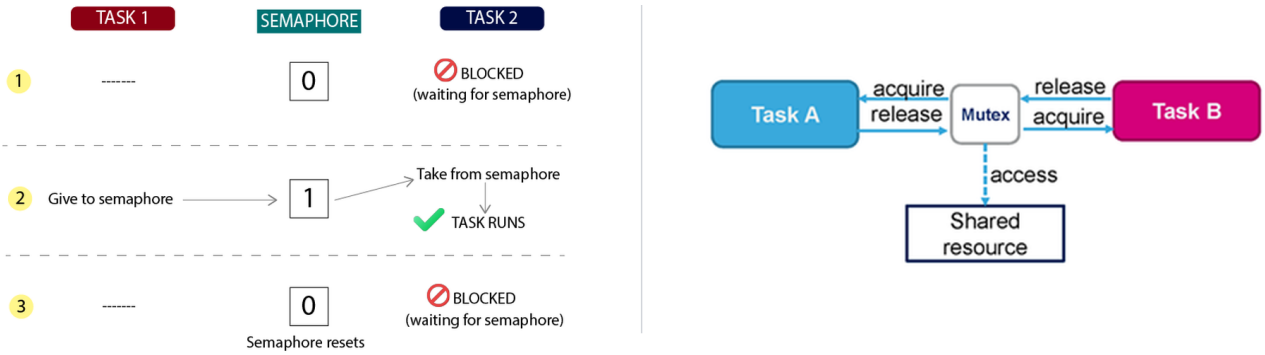
Hệ thống sử dụng sensorQueue và decisionQueue, mỗi Queue có sức chứa một phần tử. Hàm xQueueOverwrite() cập nhật dữ liệu mới nhất, còn xQueuePeek() đọc dữ liệu mà không xóa phần tử. decisionQueue lưu DecisionData gồm chế độ điều khiển, bản sao dữ liệu cảm biến và cấu hình ngưỡng. Ngoài Queue, Task Notification được dùng để đánh thức DecisionTask và ControlTask khi có dữ liệu cần xử lý[6].

2.5.3. Binary Semaphore và Mutex

Binary Semaphore dùng để đồng bộ sự kiện giữa ngắt và Task. Trong hệ thống, ISR của PIR gọi xSemaphoreGiveFromISR() để báo hiệu cho PIRTask đang đợi bằng xSemaphoreTake(). Công việc xử lý chính diễn ra trong Task, giúp ISR ngắn gọn [6], [16].

ControllerSettings gồm hai ngưỡng coolingTempC, darkLightPct và biến revision theo dõi phiên bản cấu hình. Khi WebTask cập nhật ngưỡng, chương trình sử dụng configMutex để bảo vệ dữ liệu, tăng revision và thông báo cho DecisionTask đánh giá lại chế độ. Ngưỡng WARNING cố định ở 45 °C, không thuộc ControllerSettings và không thể thay đổi từ Web Dashboard [6], [16].

Hình 2.24 minh họa cơ chế Binary Semaphore và Mutex; trong hệ thống, tín hiệu cấp Semaphore xuất phát từ ISR của PIR.



Hình 2.24. Minh họa Binary Semaphore (a) và Mutex (b)

Cơ chế	Mục đích sử dụng	Ứng dụng trong đề tài

Queue	Trao đổi bản sao dữ liệu giữa Task.	sensorQueue, decisionQueue
Binary Semaphore	Báo hiệu sự kiện, kể cả từ ISR.	Ngắt PIR → PIRTask
Mutex	Bảo vệ vùng dữ liệu dùng chung.	Bảo vệ ControllerSettings khi WebTask ghi và DecisionTask đọc cấu hình ngưỡng.
Task Notification	Báo hiệu trực tiếp cho một Task khi có dữ liệu hoặc sự kiện cần xử lý	Đánh thức DecisionTask khi dữ liệu hoặc cấu hình thay đổi và ControlTask khi có quyết định điều khiển mới.

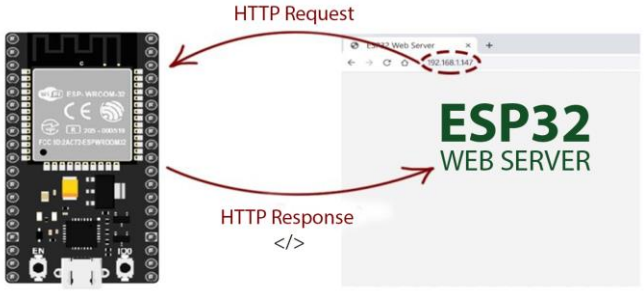
Bảng 2.5. Phân biệt các cơ chế FreeRTOS được sử dụng trong đề tài [6], [16].

2.6. Giao tiếp mạng và Web Dashboard

2.6.1. Kết nối Wi-Fi và các giao thức TCP, IP, HTTP

Wi-Fi cung cấp kết nối không dây giữa ESP32-S3 và thiết bị truy cập Web Dashboard. Trong mạng cục bộ, IP đảm nhiệm địa chỉ và chuyển gói tin, TCP tổ chức kết nối truyền dữ liệu tin cậy, còn HTTP trao đổi yêu cầu và phản hồi ở lớp ứng dụng. Trình duyệt đóng vai trò HTTP client; ESP32-S3 là HTTP server phục vụ trang Web và dữ liệu hệ thống [6].

Trong mô hình Wokwi, ESP32-S3 kết nối tới Wi-Fi "Wokwi-GUEST" ở chế độ station và khởi động Web Server tại cổng TCP 80. Cấu hình chuyển tiếp cổng của Wokwi cho phép truy cập từ trình duyệt tại localhost:8180; đây là địa chỉ phục vụ môi trường mô phỏng, không phải địa chỉ IP cố định trên phần cứng thật [6].

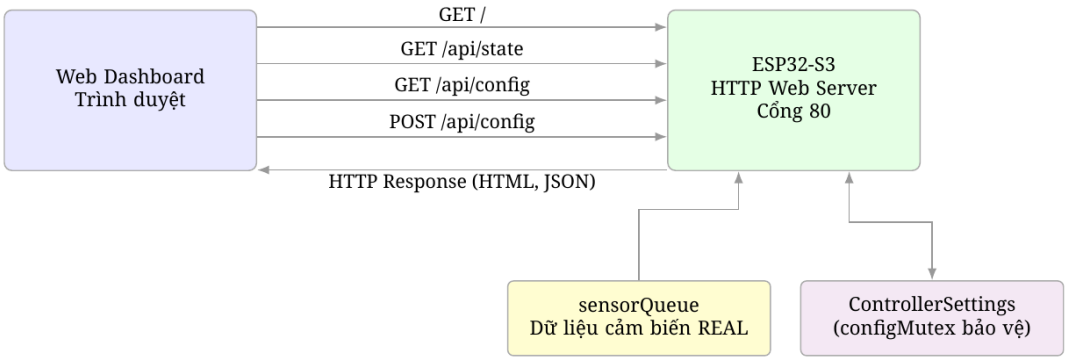


Hình 2.25. Nguyên lý yêu cầu và phản hồi HTTP giữa trình duyệt và ESP32

Theo cơ chế request–response ở Hình 2.25, trình duyệt gửi yêu cầu HTTP để lấy trang giao diện hoặc trao đổi dữ liệu; máy chủ xử lý và trả về phản hồi tương ứng. Trong chương trình, WebTask dùng thư viện WiFi.h và WebServer.h; hàm server.handleClient() được gọi lặp lại để tiếp nhận các yêu cầu, trong khi các Task cảm biến và điều khiển vẫn hoạt động độc lập dưới FreeRTOS [6].

2.6.2. Định dạng dữ liệu và cơ chế trao đổi với Web Dashboard

Web Dashboard hiển thị dữ liệu DHT22, LDR, PIR, trạng thái thiết bị và cho phép cấu hình ngưỡng điều khiển. ESP32-S3 hoạt động như HTTP Web Server tại cổng 80. Dữ liệu được đọc từ các cảm biến trong Wokwi, không được tạo giả từ Web. Nhãn REAL trong chương trình chỉ nguồn dữ liệu cảm biến mô phỏng, không phải kết quả đo trên phần cứng thực tế. ESP32-S3 là HTTP Web Server tại cổng 80 (Hình 2.26) [6].



Hình 2.26. Các yêu cầu HTTP phục vụ giám sát và cấu hình điều khiển trên Web Dashboard

Web Dashboard gọi GET /api/state khoảng mỗi 500 ms để cập nhật dữ liệu, độc lập với chu kỳ đọc cảm biến. GET /api/config trả về cấu hình hiện tại, còn POST /api/config cập nhật ngưỡng COOLING (25–40 °C) và LIGHTING (10–60%). Ngưỡng WARNING cố định 45 °C. Cấu hình được bảo vệ bằng configMutex và trở về mặc định khi khởi động lại. Các đường dẫn được nêu tại Bảng 2.6 [6].

Yêu cầu HTTP	Chức năng và dữ liệu trao đổi
GET /	ESP32-S3 trả về giao diện Web Dashboard gồm HTML, CSS và JavaScript.
GET /api/config	Trả về hai ngưỡng cấu hình, revision và ngưỡng WARNING cố định 45 °C.
POST /api/config	Kiểm tra và cập nhật ngưỡng COOLING (25–40 °C), LIGHTING (10–60%), đồng thời tăng revision.
GET /api/state	Trả về JSON chứa dữ liệu cảm biến, thông số LDR, chế độ điều khiển, trạng thái thiết bị và chỉ số thời gian thực.

Bảng 2.6. Các đường dẫn HTTP và dữ liệu trao đổi với Web Dashboard.

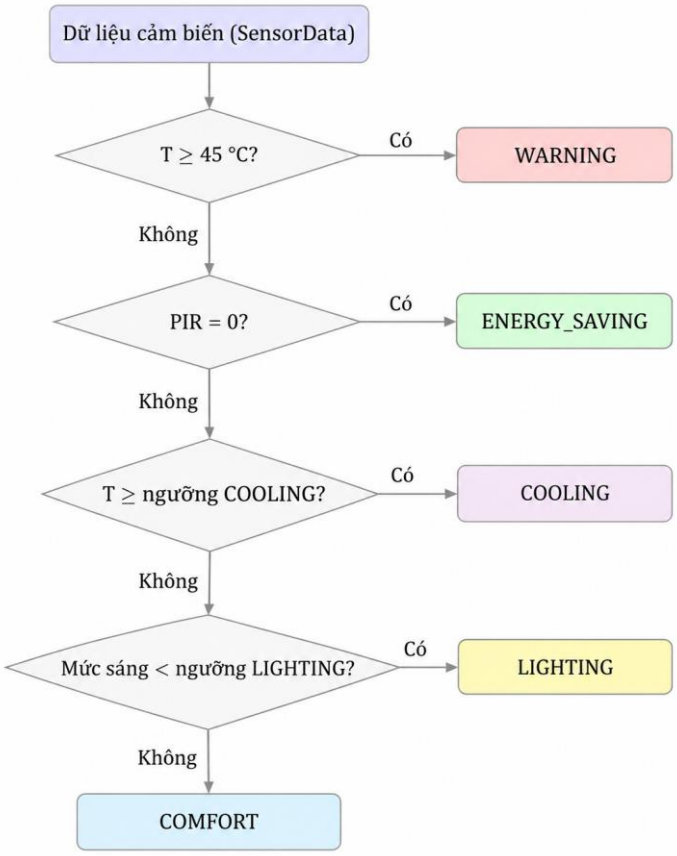
SensorTask thu thập dữ liệu và cập nhật sensorQueue; PIRTask cập nhật trạng thái chuyển động khi có sự kiện ngắt. DecisionTask nhận thông báo, đọc dữ liệu và cấu hình để lựa chọn chế độ, sau đó ghi kết quả vào decisionQueue. ControlTask nhận thông báo để điều khiển relay, quạt PWM và buzzer. Web Dashboard chỉ thực hiện giám sát và cấu hình ngưỡng[6].

2.7. Nguyên lý lựa chọn chế độ theo luật ưu tiên

Hệ thống sử dụng phương pháp điều khiển theo tập luật ưu tiên để lựa chọn một trong năm chế độ hoạt động: COMFORT, COOLING, LIGHTING, ENERGY_SAVING và WARNING. Hàm evaluateMode() đánh giá dữ liệu cảm biến cùng các ngưỡng điều khiển hiện tại, lần lượt kiểm tra điều kiện theo thứ tự ưu tiên và trả về chế độ đầu tiên thỏa mãn. Phương pháp này không lưu trạng thái trước đó và không xây dựng bảng chuyển trạng thái như một máy trạng thái hữu hạn có nhớ[6].

Các luật được xét theo thứ tự WARNING, ENERGY_SAVING, COOLING, LIGHTING và cuối cùng là COMFORT. Hình 2.27 thể hiện WARNING khi nhiệt độ đạt từ 45 °C (cố định), ENERGY_SAVING khi PIR = 0, COOLING khi nhiệt độ từ ngưỡng cấu hình trở lên, và LIGHTING khi mức sáng thấp hơn ngưỡng cấu hình. Mặc định, hai

ngưỡng COOLING và LIGHTING lần lượt là 30 °C và 30%; nhánh thỏa điều kiện đầu tiên quyết định chế độ [6].



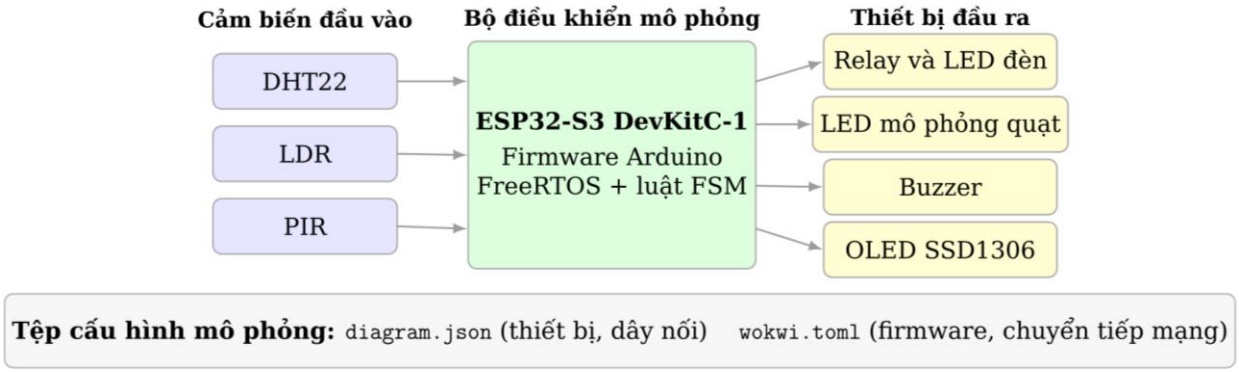
Hình 2.27. Nguyên lý lựa chọn chế độ theo luật ưu tiên với hai ngưỡng cấu hình

Ở chế độ COMFORT và ENERGY_SAVING, các thiết bị đầu ra đều tắt. LIGHTING bật đèn; COOLING điều khiển quạt ở mức PWM 180 hoặc 255 tùy nhiệt độ dưới hay từ 35 °C, đồng thời có thể bật đèn khi trời tối. WARNING kích hoạt quạt PWM 255 và buzzer 1000 Hz khi nhiệt độ đạt 45 °C. Các giá trị PWM dùng để điều khiển LED mô phỏng quạt.

Ưu tiên của luật điều khiển khác với ưu tiên lập lịch Task trong FreeRTOS. DecisionTask lựa chọn chế độ theo thứ tự WARNING, ENERGY_SAVING, COOLING, LIGHTING và COMFORT. Khi dữ liệu hoặc cấu hình thay đổi, kết quả được cập nhật vào decisionQueue và ControlTask được thông báo để thực hiện điều khiển[6].

2.8. Nền tảng mô phỏng Wokwi và giới hạn mô phỏng

Wokwi là nền tảng mô phỏng phần cứng và chương trình nhúng, hỗ trợ ESP32-S3 cùng nhiều ngoại vi như GPIO, ADC, LEDC, I2C và Wi-Fi. Trong đề tài, mô hình sử dụng ESP32-S3 DevKitC-1, các cảm biến DHT22, LDR, PIR và thiết bị đầu ra gồm relay điều khiển đèn, LED mô phỏng quạt, buzzer và OLED SSD1306. Nhờ đó, nhóm có thể tạo tình huống đầu vào và quan sát phản hồi của chương trình trước khi kiểm thử trên phần cứng thực tế [6], [13].



Hình 2.28. Các thành phần chính của mô hình Wokwi

Trong cấu hình mô phỏng, tệp diagram.json khai báo linh kiện và kết nối, còn wokwi.toml trỏ đến firmware.bin, firmware.elf và chuyển tiếp cổng localhost:8180 tới cổng 80 của ESP32-S3 ảo. Điều này cho phép quan sát chức năng cảm biến, luồng xử lý FreeRTOS, trạng thái OLED và giao tiếp Web Dashboard trên cùng một mô hình [6], [13].

Tuy nhiên, mô phỏng không thay thế phép thử trên thiết bị thật. Sai số cảm biến, nhiễu ADC, đặc tính tải và điều kiện Wi-Fi thực tế có thể khác mô hình ảo. Wokwi còn có thể giới hạn tốc độ CPU để tăng hiệu năng mô phỏng; vì vậy số liệu thời gian trong Wokwi không đủ để kết luận độ trễ và độ tin cậy của phần cứng thực tế [13].

CHƯƠNG 3

PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG NHÚNG THỜI GIAN THỰC

3.1. Phân tích yêu cầu hệ thống

3.1.1. Yêu cầu chức năng

Các yêu cầu chức năng được xác định từ dữ liệu cảm biến, cơ chế xử lý theo luật ưu tiên và các kênh hiển thị của hệ thống. Bảng 3.1 tổng hợp những chức năng cần hiện thực và thành phần mã nguồn tương ứng để thuận tiện đối chiếu khi thiết kế, mô phỏng và kiểm thử.

Mã	Yêu cầu chức năng	Thành phần minh chứng
F01	Thu thập nhiệt độ và độ ẩm từ cảm biến DHT22.	SensorTask; sensors.cpp.
F02	Đọc mã ADC của LDR; xử lý điện áp, điện trở, độ rọi ước tính và mức sáng quy ước.	analogRead(GPIO4); light_math.h.
F03	Phát hiện thay đổi tín hiệu chuyển động PIR và cập nhật trạng thái tương ứng.	GPIO16; PIRTask; pir.cpp.
F04	Cập nhật và trao đổi mẫu dữ liệu mới nhất giữa các Task phục vụ quyết định điều khiển.	SensorData; sensorQueue; app_state.cpp.
F05	Chọn một trong năm chế độ WARNING, ENERGY_SAVING, COOLING, LIGHTING và COMFORT theo luật ưu tiên.	DecisionTask; evaluateMode().
F06	Điều khiển relay đèn, LED PWM mô phỏng quạt và buzzer theo chế độ hoạt động.	ControlTask; control.cpp.
F07	Hiển thị dữ liệu cảm biến và trạng thái vận hành trên OLED SSD1306.	DisplayTask; display.cpp.
F08	Giám sát dữ liệu cảm biến REAL, trạng thái thiết bị và chỉ số thời gian qua Web Dashboard.	GET /api/state; web_server.cpp; web_page.h.
F09	Cho phép thay đổi ngưỡng COOLING, LIGHTING từ Web; giữ WARNING cố định 45 °C.	POST /api/config; configMutex.
F10	Ghi nhận các mốc thời gian, thời gian xử lý và thống kê tỷ lệ đáp ứng deadline.	pir.cpp; control.cpp; Web Dashboard.

Bảng 3.1. Yêu cầu chức năng và thành phần mã nguồn minh chứng

3.1.2. Yêu cầu phi chức năng

Về tính nhất quán, các Task cần sử dụng mẫu dữ liệu cảm biến mới nhất trong quá trình ra quyết định.

Hai ngưỡng COOLING và LIGHTING phải được cập nhật và đọc đồng bộ khi Web Dashboard thay đổi cấu hình.

Về yêu cầu thời gian thực, hệ thống tổ chức các tác vụ FreeRTOS theo mức ưu tiên và cơ chế kích hoạt phù hợp với từng chức năng. SensorTask thực hiện thu thập dữ liệu theo chu kỳ 2.000 ms, trong khi DisplayTask cập nhật màn hình OLED theo chu kỳ 500 ms. DecisionTask được kích hoạt khi có dữ liệu cảm biến mới, sự kiện PIR hoặc thay đổi cấu hình, đồng thời sử dụng thời gian chờ 50 ms để dự phòng. ControlTask được đánh thức thông qua Task Notification khi có kết quả quyết định mới. Hệ thống sử dụng ngưỡng thời gian 200 ms để thống kê tỷ lệ đáp ứng deadline trong môi trường mô phỏng Wokwi.

Về khả năng quan sát và kiểm chứng, hệ thống cần thể hiện được nhiệt độ, độ ẩm, trạng thái PIR, mã ADC và các đại lượng ước tính từ LDR trên Terminal hoặc Web Dashboard. Các đại lượng như điện áp, điện trở và độ rọi LDR được suy ra theo mô hình mô phỏng, cần phân biệt với mã ADC đọc trực tiếp.

Về tính dễ sử dụng và bảo trì, Web Dashboard cho phép giám sát trạng thái hệ thống và cấu hình hai ngưỡng điều khiển COOLING, LIGHTING mà không can thiệp trực tiếp vào thiết bị đầu ra. Mã nguồn được phân chia thành các module chức năng, thuận lợi cho việc kiểm tra, bảo trì và mở rộng. Việc kiểm chứng chức năng, thời gian đáp ứng và giới hạn mô phỏng sẽ được trình bày trong chương 4.

3.2. Thiết kế kiến trúc tổng thể và phần cứng mô phỏng

3.2.1. Kiến trúc hệ thống và luồng xử lý dữ liệu

Hệ thống sử dụng ESP32-S3 DevKitC-1 và được mô phỏng trên Wokwi, gồm các cảm biến DHT22, LDR, PIR và thiết bị đầu ra. Phần mềm tổ chức thành sáu Task FreeRTOS để thu thập dữ liệu, xử lý, điều khiển và giám sát qua OLED, Web Dashboard. Dashboard hiển thị dữ liệu cảm biến REAL và cho phép cấu hình hai ngưỡng COOLING, LIGHTING; ngưỡng WARNING được cố định ở 45 °C trong firmware.

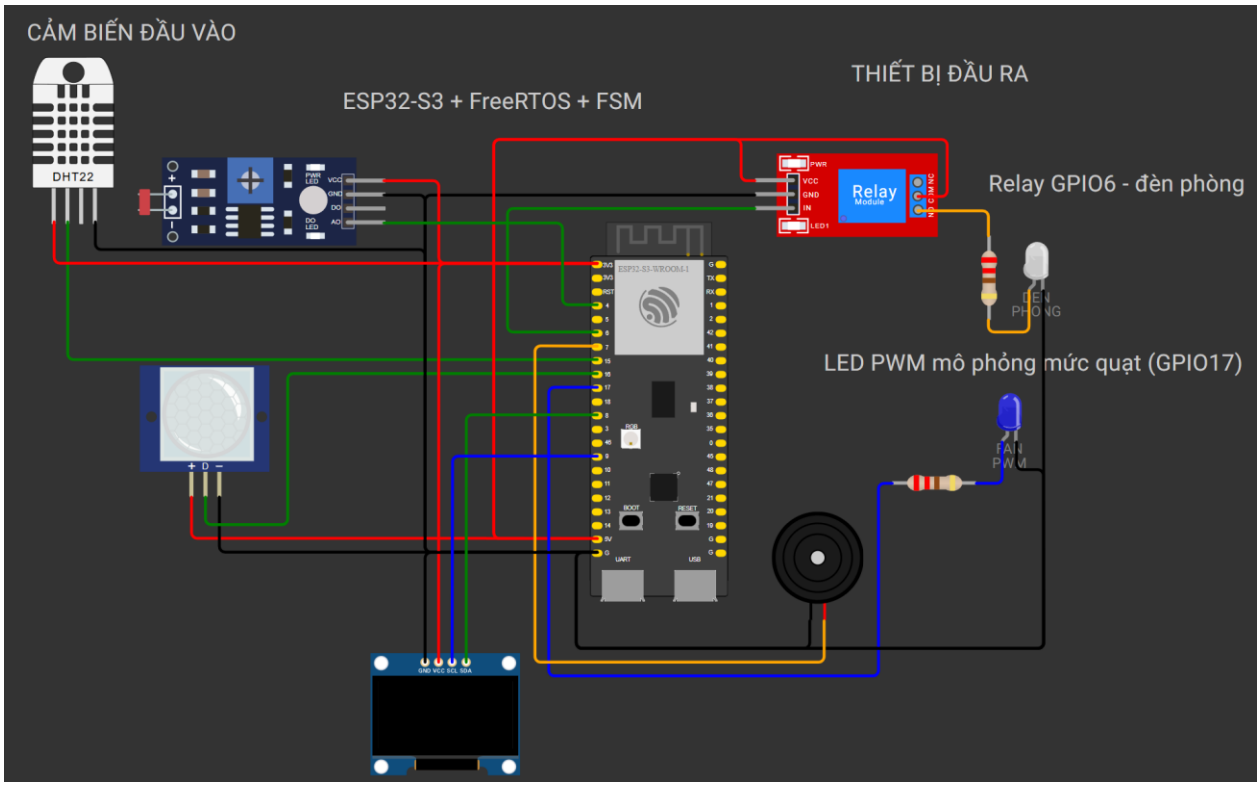
SensorTask thu thập dữ liệu DHT22, LDR và cập nhật vào sensorQueue; PIRTask tiếp nhận thông báo ngắt qua pirSemaphore để cập nhật trạng thái chuyển động. DecisionTask sử dụng dữ liệu cảm biến cùng các ngưỡng được bảo vệ bởi configMutex để lựa chọn chế độ, sau đó chuyển kết quả qua decisionQueue cho ControlTask điều khiển thiết bị. DisplayTask và WebTask đảm nhiệm việc giám sát qua OLED và Web Dashboard. Các nhánh xử lý được tổng hợp trong Bảng 3.2.

Nhánh xử lý	Dữ liệu và tài nguyên trung gian	Kết quả
Thu thập cảm biến	SensorTask; DHT22, analogRead(GPIO4); sensorQueue	Mẫu nhiệt độ, độ ẩm, mã ADC và độ rọi ước tính
Sự kiện chuyển động	Ngắt GPIO16; pirSemaphore; PIRTask; sensorQueue	Trạng thái PIR được cập nhật
Ra quyết định	DecisionTask; sensorQueue; configMutex; decisionQueue	Chế độ hoạt động và bản chụp mẫu dữ liệu
Điều khiển và hiển thị	ControlTask, DisplayTask, WebTask	Đầu ra điện, OLED và JSON trạng thái
Cấu hình qua Web	POST /api/config; configMutex; ControllerSettings	Thay đổi ngưỡng COOLING và LIGHTING trong RAM

Bảng 3.2. Các nhánh dữ liệu chính trong kiến trúc hệ thống

3.2.2. Sơ đồ kết nối và ánh xạ chân GPIO

Hình 3.1 thể hiện sơ đồ kết nối toàn mạch trên Wokwi, gồm ESP32-S3, ba cảm biến đầu vào, relay, LED mô phỏng đèn, LED PWM, buzzer và OLED SSD1306. Các kết nối được đối chiếu với diagram.json và khai báo GPIO trong config.h.



Hình 3.1. Sơ đồ kết nối toàn mạch ESP32-S3 trên Wokwi

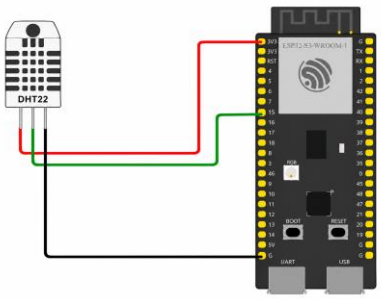
Bảng 3.3 tổng hợp các chân tín hiệu và nguồn cấp của từng linh kiện. Các hình 3.2 và 3.3 minh họa chi tiết đường nối, hỗ trợ đối chiếu khi kiểm tra mạch mô phỏng.

Thiết bị	Chân tín hiệu	Nguồn cấp và GND	Chức năng
DHT22	DATA → GPIO15	VCC → 3V3 GND → GND	Đọc nhiệt độ (°C), độ ẩm (%RH)
LDR module	AO → GPIO4 (ADC1_CH3)	VCC → 3V3 GND → GND	Đọc điện áp analog; DO không dùng
PIR	OUT → GPIO16	VCC → 5V GND → GND	Đọc chuyển động qua GPIO và ngắt
Relay đèn	IN → GPIO6	VCC → 5V GND → GND	COM → 5V; NO → điện trở 220 Ω → LED đèn → GND

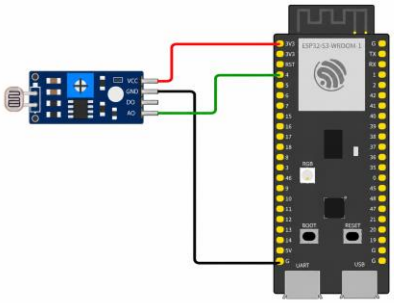
Thiết bị	Chân tín hiệu	Nguồn cấp và GND	Chức năng
LED PWM quạt	GPIO17 → R 220 Ω → Anode	Cathode → GND	Mô phỏng độ sáng tương ứng PWM quạt
Buzzer	GPIO7 → đầu tín hiệu	Đầu còn lại → GND	Phát tín hiệu cảnh báo
OLED SSD1306	SDA → GPIO8; SCL → GPIO9	VCC → 3V3 GND → GND	Hiển thị dữ liệu theo giao thức I2C

Bảng 3.3. Ánh xạ chân giữa ESP32-S3 và linh kiện mô phỏng

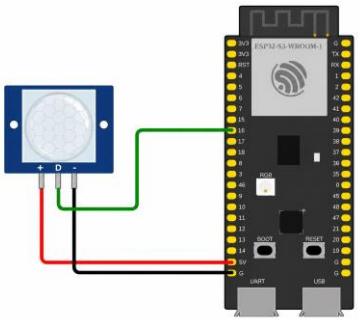
Nhóm cảm biến đầu vào gồm DHT22 (DATA–GPIO15), LDR (AO–GPIO4) và PIR (OUT–GPIO16). Chỉ tín hiệu AO của module LDR được sử dụng; các nguyên lý đọc dữ liệu đã trình bày ở Chương 2.



(a) DHT22 – DATA tại GPIO15



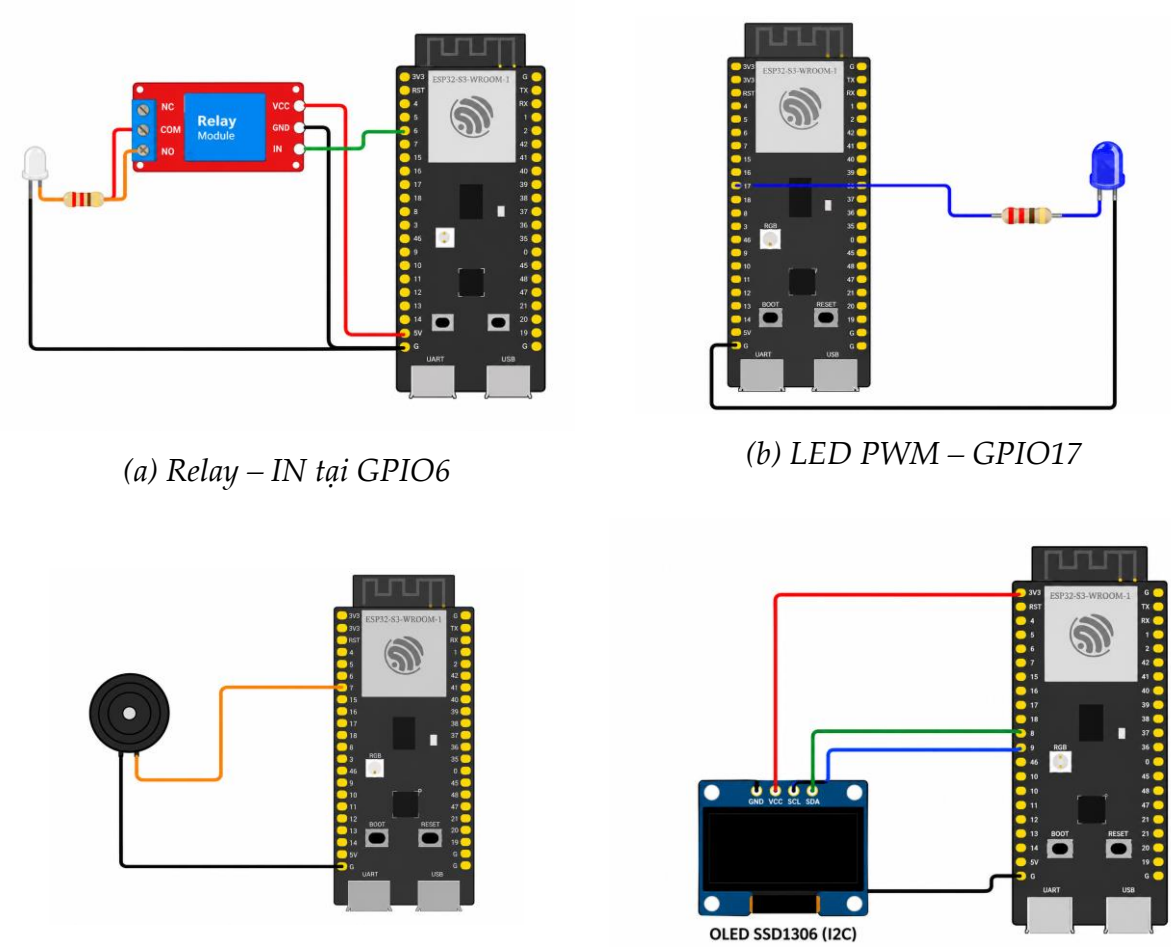
(b) LDR – AO tại GPIO4



(c) PIR – OUT tại GPIO16

Hình 3.2. Kết nối các cảm biến DHT22 (a), LDR (b) và PIR (c) với ESP32-S3

Nhóm đầu ra gồm relay điều khiển đèn tại GPIO6, LED PWM mô phỏng quạt tại GPIO17 và buzzer tại GPIO7. OLED SSD1306 dùng giao tiếp I2C với SDA–GPIO8 và SCL–GPIO9. Mối nối cụ thể được thể hiện ở Hình 3.3 và Bảng 3.3.



(a) Relay – IN tại GPIO6

(b) LED PWM – GPIO17

(c) Buzzer – GPIO7

(d) OLED – SDA GPIO8, SCL GPIO9

Hình 3.3. Kết nối relay (a), LED PWM (b), buzzer (c) và OLED (d) với ESP32-S3.

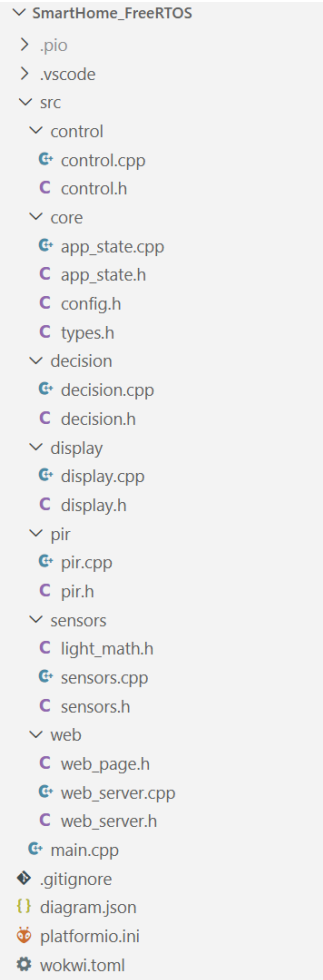
```
#define DHT_PIN      15
#define LDR_PIN      4
#define PIR_PIN      16
#define LIGHT_PIN    6
#define FAN_PIN      17
#define BUZZER_PIN   7
#define OLED_SDA     8
#define OLED_SCL     9
```

Trích mã 3.1. Các chân tín hiệu được định nghĩa trong src/core/config.h.

Trích mã 3.1 thể hiện các chân GPIO được khai báo tập trung trong tệp config.h, tương ứng với sơ đồ kết nối phần cứng trên Wokwi. Việc sử dụng các hằng số định nghĩa giúp thống nhất cấu hình chân giữa các module, hạn chế sai sót khi truy cập ngoại vi và tạo thuận lợi cho quá trình bảo trì, điều chỉnh hoặc mở rộng hệ thống.

3.2.3. Cấu trúc tổ chức mã nguồn

Hình 3.4 thể hiện cấu trúc thư mục mã nguồn của hệ thống. Dự án được tổ chức theo hướng phân chia chức năng thành các module độc lập, bao gồm quản lý cấu hình và trạng thái dùng chung, thu thập dữ liệu cảm biến, xử lý sự kiện chuyển động, ra quyết định điều khiển, điều khiển thiết bị, hiển thị OLED và giao tiếp Web. Bên cạnh đó, thư mục .vscode lưu trữ các thiết lập phục vụ môi trường phát triển, trong khi thư mục .pio chứa các tệp được tạo ra trong quá trình biên dịch và xây dựng chương trình bằng PlatformIO.



Hình 3.4. Cấu trúc thư mục mã nguồn

Module và tệp chính	**Nhiệm vụ triển khai**
core/config.h, types.h, app_state.cpp	Ánh xạ GPIO, ngưỡng, cấu trúc SensorData và DecisionData, Queue và Mutex.
sensors/sensors.cpp, light_math.h	Đọc DHT22 và ADC LDR; tính thông số LDR từ mô hình.
pir/pir.cpp	Xử lý ngắt GPIO16, Binary Semaphore và cập nhật chuyển động.
decision/decision.cpp	Chọn chế độ theo điều kiện ưu tiên và tạo DecisionData.
control/control.cpp	Điều khiển relay, PWM, buzzer và lấy mốc đo thời gian.
display/display.cpp	Giao tiếp I2C và cập nhật OLED.
web/web_server.cpp, web_page.h	HTTP API, giao diện và cấu hình hai ngưỡng.
main.cpp	Khởi tạo ngoại vi, tài nguyên FreeRTOS và sáu Task.

Bảng 3.4. Vai trò các module mã nguồn

Cách phân tách module giúp đối chiếu trực tiếp chức năng hệ thống với các tệp hiện thực. Các đoạn mã vận hành của từng Task sẽ được phân tích tại những mục thiết kế tiếp theo.

3.3. Thiết kế hệ thống đa tác vụ FreeRTOS

Hệ thống được xây dựng trên nền tảng vi điều khiển ESP32-S3, sử dụng FreeRTOS để tổ chức và điều phối các chức năng xử lý. Chương trình được chia thành sáu Task đảm nhiệm các nhiệm vụ thu thập dữ liệu cảm biến, xử lý tín hiệu chuyển động, xác định chế độ hoạt động, điều khiển thiết bị, hiển thị OLED và giao tiếp Web. Các Task trao đổi dữ liệu thông qua Queue, đồng bộ sự kiện bằng Binary Semaphore và bảo vệ cấu hình dùng chung bằng Mutex. Trên cơ sở kiến trúc này, các nội dung tiếp theo trình bày quá trình khởi tạo, phân công mức ưu tiên và tổ chức hoạt động của các Task trong hệ thống.

3.3.1. Khởi tạo và phân công các Task

Hệ thống sử dụng FreeRTOS để tổ chức sáu Task độc lập, tương ứng với các chức năng xử lý cảm biến, phát hiện chuyển động, ra quyết định, điều khiển thiết bị, hiển thị và giao tiếp Web. Các Task được khởi tạo trong hàm setup() của tệp main.cpp thông qua API xTaskCreate(), với mức ưu tiên và kích thước stack được thiết lập theo chức năng.

Trước khi tạo Task, chương trình lần lượt gọi initSensors(), initControl() và initDisplay() để khởi tạo ngoại vi. Tiếp theo, hàm initRTOSObjects() tạo các đối tượng Queue, Binary Semaphore và Mutex; sau khi tạo thành công, initPIRInterrupt() được gọi để cấu hình ngắt PIR. Trình tự này bảo đảm pirSemaphore tồn tại trước khi ISR được kích hoạt. Chương trình kiểm tra kết quả khởi tạo và tạo Task, đồng thời thông báo lỗi qua Serial nếu có thao tác thất bại.

Sau khi hoàn tất khởi tạo các ngoại vi và đối tượng đồng bộ, chương trình sử dụng hàm xTaskCreate() để tạo sáu Task FreeRTOS. Mỗi Task được cấu hình với tên, kích thước stack và mức ưu tiên tương ứng, như thể hiện trong trích mã 3.4:

```
created &= (xTaskCreate(PIRTask, "PIRTask" , 2048 , nullptr , 4 , nullptr ) == pdPASS);
created &= (xTaskCreate(ControlTask, "ControlTask" , 4096 , nullptr , 3 , & controlTaskHandle) == pdPASS);
created &= (xTaskCreate(SensorTask, "SensorTask" , 4096 , nullptr , 2 , nullptr ) == pdPASS);
created &= (xTaskCreate(DecisionTask,"DecisionTask" ,3072 ,nullptr , 2,& decisionTaskHandle) == pdPASS);
created &= (xTaskCreate(DisplayTask, "DisplayTask" , 3072 , nullptr , 1 , nullptr ) == pdPASS);
created &= (xTaskCreate(WebTask, "WebTask" , 6144 , nullptr , 1 , nullptr ) == pdPASS);
```

Trích mã 3.4. Khởi tạo sáu Task FreeRTOS bằng hàm xTaskCreate().

Các Task được phân công mức ưu tiên dựa trên yêu cầu xử lý của từng chức năng. Bên cạnh các tác vụ hoạt động theo chu kỳ, hệ thống sử dụng cơ chế thông báo trực tiếp

giữa các Task để xử lý dữ liệu và kết quả điều khiển ngay khi có sự kiện mới. Bảng 3.2 tổng hợp mức ưu tiên, cơ chế kích hoạt và thời gian chờ của sáu Task FreeRTOS được sử dụng trong hệ thống.

Tác vụ	Ưu tiên	Thời gian chờ và cơ chế đồng bộ
PIRTask	4	Chờ Binary Semaphore từ ISR
ControlTask	3	Chờ Task Notification từ DecisionTask
SensorTask	2	Lặp với thời gian chờ 2.000 ms
DecisionTask	2	Chờ Task Notification, timeout 50 ms
DisplayTask	1	Lặp với thời gian chờ 500 ms
WebTask	1	Xử lý HTTP, nhường CPU 2 ms mỗi vòng

Bảng 3.2. Phân công tác vụ và thông số lập lịch của hệ thống

Mức ưu tiên được phân bổ theo yêu cầu xử lý: PIRTask có ưu tiên cao nhất (4) để tiếp nhận sự kiện chuyển động, tiếp theo là ControlTask (3) để cập nhật thiết bị. SensorTask và DecisionTask cùng ưu tiên 2, đảm nhiệm thu thập dữ liệu và xác định chế độ hoạt động. DisplayTask và WebTask có ưu tiên 1, thực hiện cập nhật OLED và xử lý yêu cầu HTTP.

Các Task được điều phối bởi bộ lập lịch FreeRTOS thông qua cơ chế ưu tiên và đồng bộ sự kiện. PIRTask sử dụng xSemaphoreTake() để chờ tín hiệu ngắt, trong khi DecisionTask và ControlTask sử dụng ulTaskNotifyTake() để tiếp nhận thông báo từ các tác vụ liên quan. SensorTask và DisplayTask sử dụng vTaskDelay() để thực hiện chức năng theo chu kỳ. WebTask xử lý các yêu cầu HTTP và nhường quyền sử dụng CPU sau mỗi vòng lặp. Hàm loop() chỉ thực hiện lệnh trì hoãn 1.000 ms, không đảm nhiệm thuật toán xử lý chính của hệ thống.

3.3.2. Thu thập và xử lý dữ liệu trong SensorTask

SensorTask được triển khai trong tệp src/sensors/sensors.cpp, có nhiệm vụ thu thập nhiệt độ, độ ẩm từ DHT22, tín hiệu ánh sáng từ LDR và trạng thái chuyển động PIR. Trong hàm initSensors(), DHT22 được khởi tạo bằng dht.setup(), các chân cảm biến được cấu hình đầu vào và ADC được thiết lập độ phân giải 12 bit thông qua analogReadResolution(12).

Trong mỗi vòng xử lý, Task sử dụng dht.getTempAndHumidity() để đọc nhiệt độ, độ ẩm và isfinite() để kiểm tra tính hợp lệ. Đồng thời, analogRead(LDR_PIN) thu thập mã ADC từ GPIO4, sau đó hàm ldrFromAdc() chuyển đổi thành mức sáng tương đối từ 0–

100%, đồng thời ước tính điện áp, điện trở và độ rọi theo mô hình Wokwi. Trạng thái PIR được lấy từ biến currentOccupancy.

Các dữ liệu được đóng gói vào cấu trúc SensorData, với các trường được trình bày trong Bảng 3.5.

Trường dữ liệu	**Ý nghĩa**
temperature	Nhiệt độ DHT22 (°C)
sampleTick	Thời điểm ghi nhận mẫu theo tick FreeRTOS
humidity	humidity
Độ ẩm DHT22 (%RH)	Độ ẩm DHT22 (%RH)
light	Mức sáng quy ước (%)
ldrAdcRaw	Mã ADC 12 bit đọc từ GPIO4
occupancy	Trạng thái chuyển động PIR
ldrVoltageV	Điện áp ước tính theo mô hình (V)
ldrResistanceKOhm	Điện trở LDR ước tính (kΩ)
ldrLux	Độ rọi ước tính theo mô hình Wokwi (lux)

Bảng 3.5. Các trường dữ liệu trong cấu trúc SensorData

Sau khi thu thập, chương trình sử dụng xTaskGetTickCount() để ghi nhận thời điểm lấy mẫu. Khi nhiệt độ và độ ẩm đã có giá trị hợp lệ, lệnh xQueueOverwrite(sensorQueue, &data) cập nhật dữ liệu mới nhất vào Queue một phần tử. Quá trình xử lý chính được thể hiện trong Trích mã 3.5.

```
TempAndHumidity th = dht.getTempAndHumidity();
if (isfinite(th.temperature)) data.temperature = th.temperature;
 if (isfinite(th.humidity)) data.humidity = th.humidity;
            const int raw = analogRead(LDR_PIN);
            const LdrDiagnostics ldr = ldrFromAdc(raw);
            data.ldrAdcRaw = ldr.adcRaw;
            data.ldrVoltageV = ldr.voltageV;
            data.ldrResistanceKOhm = ldr.resistanceKOhm;
            data.ldrLux = ldr.lux;
            data.light = ldr.brightnessPct;
```

```
            data.occupancy = currentOccupancy;
            data.sampleTick = xTaskGetTickCount();
if (!isfinite(data.temperature) ||!isfinite(data.humidity))
          {
          Serial.println("[SENSOR] DHT22 waiting for first valid measurement");
          vTaskDelay(pdMS_TO_TICKS(SENSOR_REAL_PERIOD_MS));
           continue;
          }
xQueueOverwrite(sensorQueue, &data);
```

Trích mã 3.5. Thu thập và cập nhật dữ liệu trong SensorTask (src/sensors/sensors.cpp).

Sau mỗi vòng xử lý, SensorTask gọi vTaskDelay(pdMS_TO_TICKS(SENSOR_REAL_PERIOD_MS)) với thời gian chờ 2000 ms. Chương trình cũng xuất thông tin cảm biến ra Serial Monitor theo khoảng thời gian 5000 ms để theo dõi kết quả thu thập. Hình 3.5 minh họa dữ liệu được ghi nhận trong quá trình mô phỏng.

Các đại lượng điện áp, điện trở và độ rọi của LDR được tính toán theo mô hình mô phỏng Wokwi dựa trên mã ADC đọc được. SensorTask sử dụng vTaskDelay() với thời gian chờ 2.000 ms sau mỗi lượt thu thập dữ liệu. Do quá trình đọc cảm biến và thực thi các lệnh xử lý cũng tiêu tốn thời gian, khoảng cách thực tế giữa hai lần lấy mẫu có thể lớn hơn khoảng thời gian chờ đã cấu hình.

3.3.3. Thiết kế Queue, Binary Semaphore, Mutex và xử lý ngắt PIR

Để trao đổi dữ liệu và đồng bộ giữa các Task, hệ thống sử dụng hai Queue, một Binary Semaphore và một Mutex. Các đối tượng này được khởi tạo trong hàm initRTOSObjects() tại tệp src/core/app_state.cpp. Mỗi cơ chế đảm nhiệm một chức năng riêng, được tổng hợp trong Bảng 3.6.

Đối tượng	**Cấu hình**	**Task sử dụng**	**Vai trò**
sensorQueue	Queue 1 phần tử, SensorData	SensorTask, PIRTask, DecisionTask, DisplayTask, WebTask	Lưu dữ liệu cảm biến mới nhất
decisionQueue	Queue 1 phần tử, DecisionData	DecisionTask,ControlTask, DisplayTask, WebTask	Lưu kết quả quyết định
pirSemaphore	Binary Semaphore	ISR, PIRTask	Báo hiệu sự kiện thay đổi tín hiệu PIR

configMutex	Mutex	WebTask, DecisionTask	Bảo vệ cấu hình ngưỡng điều khiển

Bảng 3.6. Các đối tượng trao đổi dữ liệu và đồng bộ trong hệ thống.

Hai Queue sensorQueue và decisionQueue đều có sức chứa một phần tử, lần lượt lưu cấu trúc SensorData và DecisionData. Chương trình sử dụng xQueueOverwrite() để cập nhật dữ liệu mới nhất và xQueuePeek() để đọc mà không xóa phần tử. Trong khi đó, pirSemaphore dùng để báo hiệu sự kiện ngắt PIR, còn configMutex bảo vệ cấu trúc ControllerSettings khi WebTask cập nhật và DecisionTask đọc các ngưỡng điều khiển. Các lệnh khởi tạo được thể hiện trong Trích mã 3.6.

```
bool initRTOSObjects ()
{
sensorQueue = xQueueCreate(1,sizeof(SensorData));
 decisionQueue = xQueueCreate(1,sizeof(DecisionData));
 pirSemaphore = xSemaphoreCreateBinary();
 configMutex = xSemaphoreCreateMutex();
 return sensorQueue && decisionQueue && pirSemaphore && configMutex;
}
```

Trích mã 3.6. Khởi tạo Queue, Binary Semaphore và Mutex (src/core/app_state.cpp).

Ngoài Queue, Binary Semaphore và Mutex, hệ thống sử dụng Task Notification để thông báo dữ liệu mới giữa các tác vụ. SensorTask và PIRTask gửi thông báo đến DecisionTask khi dữ liệu cảm biến được cập nhật. Sau khi xác định chế độ hoạt động và ghi kết quả vào decisionQueue, DecisionTask gửi thông báo để ControlTask thực hiện điều khiển. Khi ngưỡng điều khiển được thay đổi thông qua Web Dashboard, DecisionTask cũng được thông báo để đánh giá lại chế độ hoạt động. Cơ chế này giúp giảm thời gian chờ xử lý so với việc chỉ kiểm tra dữ liệu theo chu kỳ cố định.

Đối với cảm biến PIR tại GPIO16, chương trình cấu hình ngắt theo chế độ CHANGE thông qua attachInterrupt(). Khi tín hiệu thay đổi, hàm pirISR() ghi nhận thời điểm ngắt bằng xTaskGetTickCountFromISR() và gọi xSemaphoreGiveFromISR() để đánh thức PIRTask. ISR chỉ thực hiện các thao tác cần thiết, còn việc đọc trạng thái cảm biến và cập nhật dữ liệu được chuyển sang PIRTask.

Sau khi nhận tín hiệu bằng xSemaphoreTake(), PIRTask đọc GPIO16 thông qua digitalRead(), cập nhật biến currentOccupancy và tính độ trễ từ ISR đến Task. Nếu sensorQueue đã có dữ liệu, Task đọc bản sao bằng xQueuePeek(), cập nhật trường

occupancy và ghi lại bằng xQueueOverwrite(). Quá trình xử lý chính được thể hiện trong Trích mã 3.7.

```
void  IRAM_ATTR pirISR () {
 BaseType_t taskWoken = pdFALSE;
 pirIsrTick = xTaskGetTickCountFromISR();
 xSemaphoreGiveFromISR(pirSemaphore, & taskWoken);
 if (taskWoken == pdTRUE)
              portYIELD_FROM_ISR();
            } xSemaphoreTake(pirSemaphore,  portMAX_DELAY);
            currentOccupancy = digitalRead(PIR_PIN);
            TickType_t  now = xTaskGetTickCount();
            pirLatencyMs = (now - pirIsrTick) * portTICK_PERIOD_MS;
            SensorData  sensor = {};
 if (xQueuePeek(sensorQueue, & sensor, 0 ) == pdTRUE) {
              sensor.occupancy = currentOccupancy;
              sensor.sampleTick = now;
              xQueueOverwrite(sensorQueue, & sensor);
 if (decisionTaskHandle)
               xTaskNotifyGive(decisionTaskHandle);
}
```

Trích mã 3.7. Xử lý ngắt và cập nhật trạng thái PIR (src/pir/pir.cpp).

Cách tổ chức trên giúp tách xử lý ngắt khỏi phần xử lý dữ liệu và cho phép các Task trao đổi trạng thái thông qua các đối tượng FreeRTOS. Đồng thời, Mutex bảo đảm hai ngưỡng điều khiển được đọc và cập nhật nhất quán, hạn chế xung đột khi nhiều Task truy cập cấu hình dùng chung.

3.4. Thiết kế thuật toán quyết định và điều khiển

Sau khi thu thập và đồng bộ dữ liệu cảm biến, hệ thống thực hiện đánh giá các điều kiện môi trường để lựa chọn chế độ hoạt động và điều khiển thiết bị tương ứng. Thuật toán được xây dựng theo tập luật có thứ tự ưu tiên, gồm năm chế độ: COMFORT, COOLING, LIGHTING, ENERGY_SAVING và WARNING. Quá trình xử lý được phân chia giữa DecisionTask và ControlTask, sử dụng decisionQueue để chuyển kết quả quyết định đến khối điều khiển. Các luật lựa chọn chế độ và cơ chế thực thi của hai Task được trình bày trong các mục tiếp theo.

3.4.1. Xây dựng luật xác định chế độ hoạt động

Hệ thống sử dụng thuật toán điều khiển dựa trên luật (Rule-based Control) để lựa chọn một trong năm chế độ: COMFORT, COOLING, LIGHTING, ENERGY_SAVING và WARNING. Việc xác định chế độ dựa trên nhiệt độ, mức sáng tương đối, trạng thái chuyển động PIR và các ngưỡng cấu hình. Các điều kiện được xét theo thứ tự ưu tiên từ cao xuống thấp nhằm xác định duy nhất một chế độ hoạt động cho mỗi lần đánh giá.

Thứ tự ưu tiên được thiết lập là WARNING → ENERGY_SAVING → COOLING → LIGHTING → COMFORT. Trong đó, WARNING được kích hoạt khi nhiệt độ đạt từ 45 °C trở lên, không phụ thuộc trạng thái PIR. Khi không có tín hiệu chuyển động, hệ thống chọn ENERGY_SAVING nếu chưa thỏa điều kiện WARNING. Trường hợp có tín hiệu PIR, hệ thống lần lượt kiểm tra ngưỡng nhiệt độ COOLING và mức sáng LIGHTING trước khi lựa chọn COMFORT. Các điều kiện được tổng hợp trong Bảng 3.7.

Ưu tiên	**Chế độ**	**Điều kiện**
1	WARNING	T ≥ 45^oC
2	ENERGY_SAVING	T < 45^oC và PIR = 0
3	COOLING	T < $45^oCL, PIR = 1, T \geq T_{cool}$
4	LIGHTING	T < $45^oCL, PIR = 1$, T < T_{cool} , $L < L_{dark}$
5	COMFORT	Không thỏa các điều kiện trên

Bảng 3.7. Các luật xác định chế độ hoạt động theo thứ tự ưu tiên.

Hai ngưỡng COOLING và LIGHTING có giá trị mặc định lần lượt là 30 °C và 30%, được phép điều chỉnh thông qua Web Dashboard trong khoảng 25–40 °C và 10–60%. Ngưỡng WARNING được cố định ở 45 °C trong firmware. Thuật toán lựa chọn chế độ được triển khai bằng hàm evaluateMode() trong tệp decision.cpp, như thể hiện trong Trích mã 3.8.

```
SystemMode evaluateMode (
 const SensorData & data,
 const ControllerSettings & settings
)
{ if (data.temperature >= TEMP_WARNING)
        return WARNING;
 if (! data.occupancy)
        return ENERGY_SAVING;
```

```
 if (data.temperature >= settings.coolingTempC)
            return COOLING;
 if (data.light < settings.darkLightPct)
            return LIGHTING;
            return COMFORT;
}
```

Trích mã 3.8. Hàm evaluateMode() xác định chế độ hoạt động theo luật ưu tiên (src/decision/decision.cpp).

Hàm evaluateMode() kiểm tra tuần tự các điều kiện và trả về ngay khi tìm được chế độ phù hợp. Cách triển khai này bảo đảm mỗi lần đánh giá chỉ trả về một chế độ, đồng thời ưu tiên điều kiện cảnh báo nhiệt độ. Thuật toán không lưu trạng thái trước đó mà xác định lại chế độ dựa trên dữ liệu cảm biến và cấu hình tại thời điểm đánh giá.

3.4.2. Thiết kế DecisionTask và ControlTask

Quá trình quyết định và điều khiển được phân chia thành hai Task độc lập, trao đổi dữ liệu thông qua decisionQueue. Trong đó, DecisionTask thực hiện đánh giá các luật lựa chọn chế độ, còn ControlTask chuyển kết quả quyết định thành tín hiệu điều khiển thiết bị.

DecisionTask sử dụng ulTaskNotifyTake() để chờ thông báo khi có mẫu cảm biến mới, sự kiện PIR hoặc thay đổi cấu hình điều khiển. Thời gian chờ tối đa được thiết lập là 50 ms. Khi nhận thông báo hoặc hết thời gian chờ, Task đọc mẫu SensorData mới nhất từ sensorQueue bằng xQueuePeek() và lấy bản sao cấu hình ngưỡng thông qua readControllerSettings(). Hàm evaluateMode() được gọi để xác định chế độ hoạt động theo các luật ưu tiên. Kết quả được đóng gói vào cấu trúc DecisionData, bao gồm chế độ, dữ liệu cảm biến và cấu hình tương ứng. Chương trình chỉ cập nhật decisionQueue khi xuất hiện mẫu dữ liệu mới, thay đổi phiên bản cấu hình hoặc thay đổi chế độ hoạt động so với lần gửi trước. Sau khi cập nhật Queue, DecisionTask gửi Task Notification để đánh thức ControlTask thực hiện điều khiển. Quá trình này được thể hiện trong Trích mã 3.9.

```
void DecisionTask ( void * parameter) {
 SensorData sensor = {};
 ControllerSettings settings = {};
 DecisionData decision = {};
 bool haveDispatched = false ;
 TickType_t lastDispatchedTick = 0 ;
 uint32_t lastDispatchedRevision = 0 ;
```

```
SystemMode lastDispatchedMode = COMFORT;
while ( true ) {
    ulTaskNotifyTake(pdTRUE, pdMS_TO_TICKS(DECISION_PERIOD_MS)  );
   if (xQueuePeek(sensorQueue, & sensor, 0) != pdTRUE || !readControllerSettings(settings)) continue ;
          decision.sensor = sensor;
          decision.settings = settings;
          decision.mode = evaluateMode(sensor, settings);
          const bool changed = ! haveDispatched || sensor.sampleTick != lastDispatchedTick ||
          settings.revision != lastDispatchedRevision || decision.mode != lastDispatchedMode;
   if ( ! changed) continue ;
          xQueueOverwrite(decisionQueue, & decision);
          haveDispatched = true ; lastDispatchedTick = sensor.sampleTick;
          lastDispatchedRevision = settings.revision; lastDispatchedMode = decision.mode;
   if (controlTaskHandle)
          xTaskNotifyGive(controlTaskHandle);
   }
  }
```

Trích mã 3.9. Xử lý thông báo và cập nhật kết quả quyết định trong DecisionTask.

ControlTask đọc kết quả từ decisionQueue để xác định trạng thái relay đèn, mức PWM mô phỏng quạt và buzzer. Các tín hiệu được cập nhật thông qua digitalWrite(), ledcWrite(), tone() và noTone(). Giá trị PWM sử dụng các mức 0, 180 và 255 trên thang 8 bit; trong đó mức 180 tương ứng khoảng 70,6% giá trị điều khiển cực đại. Trạng thái thiết bị được xác định theo từng chế độ, như tổng hợp trong Bảng 3.8.

Chế độ	**Relay đèn**	**PWM quạt**	**Buzzer**
COMFORT	Tắt	0	Tắt
ENERGY_SAVING	Tắt	0	Tắt
LIGHTING	Bật	0	Tắt
COOLING	Bật nếu PIR = 1 và ánh sáng dưới ngưỡng, ngược lại tắt	180 nếu $T < 35$ °C; 255 nếu $T \geq 35$ °C	Tắt
WARNING	Tắt	255	Bật, 1000 Hz

Bảng 3.8. Trạng thái điều khiển thiết bị theo chế độ hoạt động.

Quá trình lựa chọn trạng thái thiết bị được thực hiện bằng cấu trúc switch (decision.mode) trong ControlTask, như thể hiện trong Trích mã 3.10. Task sử dụng ulTaskNotifyTake() để chờ thông báo từ DecisionTask thay vì thực hiện điều khiển theo chu kỳ cố định. Khi có kết quả quyết định mới, ControlTask cập nhật tín hiệu điều khiển đến relay đèn, LED PWM mô phỏng quạt và buzzer. Sau khi điều khiển, các trạng thái fanPwm, lightOn và alarmOn được cập nhật để phục vụ giám sát. Đồng thời, chương trình ghi nhận các chỉ số thời gian thực để đánh giá khả năng đáp ứng của hệ thống.

```
switch  (decision.mode)  {
 case COMFORT :
 case ENERGY_SAVING :
 break ;
 case LIGHTING :
   nextLight = true ;
 break ;
 case COOLING :
   nextFan = sensor.temperature >= TEMP_FAN_MAX ? 255 : 180 ;
 if (sensor.occupancy && sensor.light < decision.settings.darkLightPct) { nextLight = true ; }
            break ;
            case WARNING :
              nextFan = 255 ;
              nextAlarm = true ;
            break ;
}
digitalWrite(LIGHT_PIN, (nextLight == RELAY_ACTIVE_HIGH) ? HIGH : LOW);
  ledcWrite(PWM_CHANNEL,  nextFan);
 if  (nextAlarm != buzzerWasOn) {
            if (nextAlarm) tone(BUZZER_PIN, 1000 );
                    else noTone(BUZZER_PIN);
                    buzzerWasOn = nextAlarm;
}
```

Trích mã 3.10. Xác định trạng thái thiết bị trong ControlTask (src/control/control.cpp).

Việc truyền cấu trúc DecisionData giúp ControlTask sử dụng cùng mẫu cảm biến và cấu hình ngưỡng đã được DecisionTask đánh giá, hạn chế sai lệch do đọc dữ liệu tại những thời điểm khác nhau. Cơ chế đo thời gian thực của Task được trình bày riêng ở mục 3.6.

3.5. Thiết kế giao diện giám sát và hiển thị

Hệ thống được thiết kế với hai phương thức hiển thị gồm Web Dashboard và màn hình OLED SSD1306 nhằm hỗ trợ theo dõi dữ liệu cảm biến, chế độ hoạt động và trạng thái thiết bị. Web Dashboard sử dụng giao thức HTTP để trao đổi dữ liệu với ESP32-S3 thông qua kết nối Wi-Fi, đồng thời cho phép người dùng điều chỉnh các ngưỡng điều khiển. Trong khi đó, màn hình OLED sử dụng giao thức I2C để hiển thị trực tiếp các thông số cơ bản tại thiết bị. Hai phương thức này được triển khai thông qua WebTask và DisplayTask, giúp người dùng giám sát hệ thống tại chỗ hoặc qua trình duyệt.

3.5.1. Thiết kế HTTP Web Server và Web Dashboard

Hệ thống sử dụng thư viện WiFi.h và WebServer.h để xây dựng HTTP Server trên ESP32-S3, hoạt động tại cổng 80. Trong WebTask, chương trình kết nối mạng Wokwi-GUEST thông qua WiFi.begin(), sau đó đăng ký các đường dẫn API bằng server.on() và khởi động máy chủ bằng server.begin(). Trong môi trường mô phỏng, tệp wokwi.toml ánh xạ cổng localhost:8180 của máy tính đến cổng 80 của ESP32-S3, cho phép truy cập Dashboard bằng trình duyệt.

Web Dashboard được xây dựng bằng HTML, CSS và JavaScript, lưu trong tệp web_page.h dưới dạng hằng WEB_PAGE[] PROGMEM. Giao diện cung cấp các khu vực giám sát nhiệt độ, độ ẩm, ánh sáng, chuyển động PIR, chế độ hoạt động và trạng thái thiết bị; đồng thời hiển thị các chỉ số thời gian thực. Dữ liệu được truy xuất thông qua HTTP theo cơ chế yêu cầu và phản hồi, với các API chính tổng hợp trong Bảng 3.9.

Phương thức	Đường dẫn	Chức năng
GET	/	Trả về giao diện Web Dashboard
GET	/api/state	Cung cấp JSON dữ liệu cảm biến, trạng thái thiết bị và chỉ số thời gian
GET	/api/config	Đọc các ngưỡng điều khiển hiện tại
POST	/api/config	Kiểm tra và cập nhật ngưỡng COOLING, LIGHTING

Bảng 3.9. Các API HTTP phục vụ giám sát và cấu hình hệ thống.

Trong hàm handleState(), dữ liệu cảm biến và trạng thái hệ thống được đọc từ các Queue và biến dùng chung, sau đó đóng gói thành chuỗi JSON để trả về trình duyệt. Web

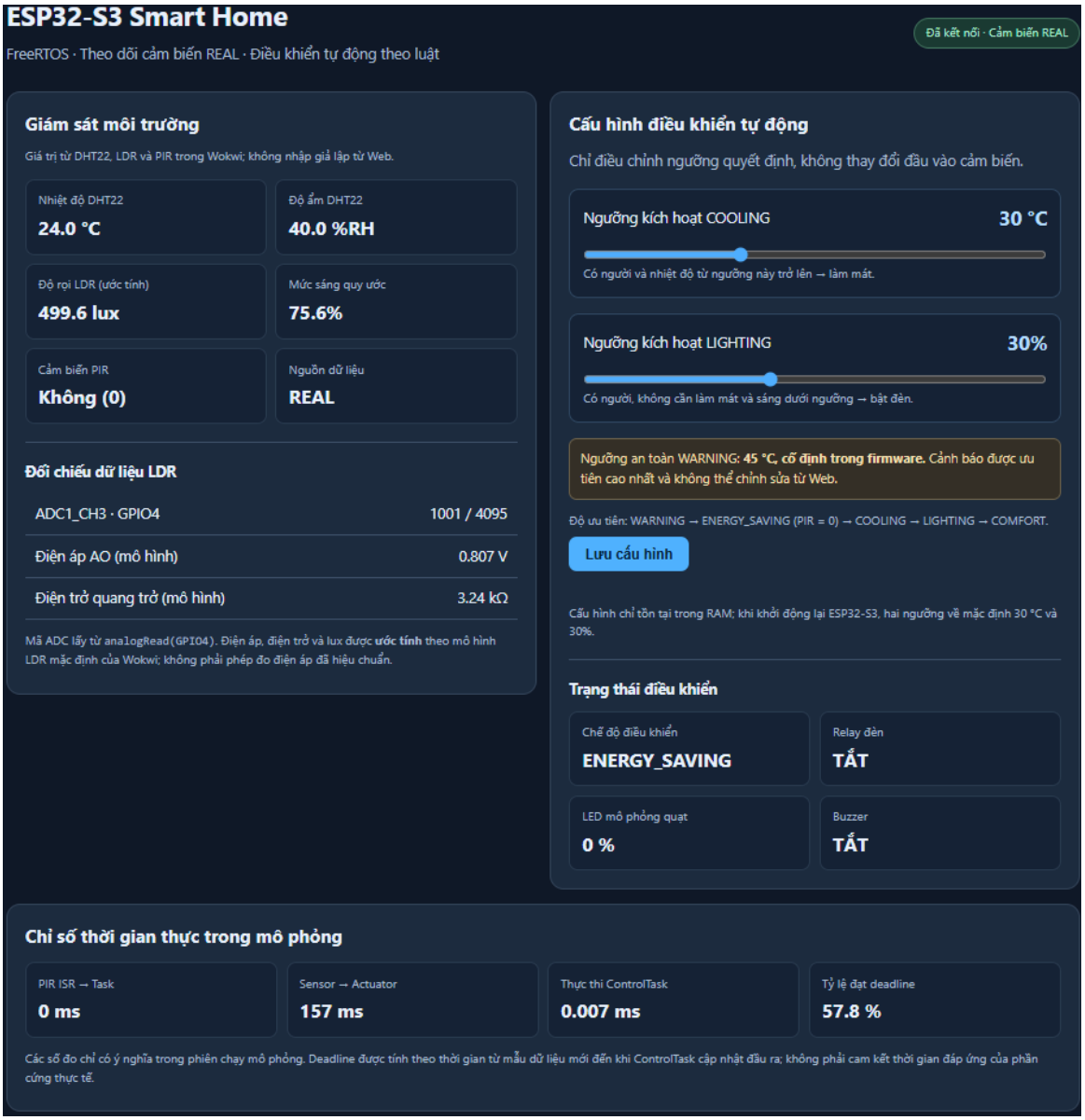
Dashboard sử dụng hàm JavaScript refresh() gọi GET /api/state theo khoảng thời gian 500 ms để cập nhật các thông số hiển thị. Việc đăng ký các đường dẫn và xử lý yêu cầu HTTP được thể hiện trong Trích mã 3.11.

```
void WebTask ( void * parameter) {
 WiFi.begin( "Wokwi-GUEST" , "" , 6 );
 while (WiFi.status() != WL_CONNECTED) {
              vTaskDelay(pdMS_TO_TICKS( 250 ));
            }
 server.on( "/" , HTTP_GET, handleRoot);
 server.on( "/api/state" , HTTP_GET, handleState);
 server.on( "/api/config" , HTTP_GET, handleGetConfig);
 server.on( "/api/config" , HTTP_POST, handleSetConfig);
 server.begin();
 while ( true ) {
              server.handleClient();
              vTaskDelay(pdMS_TO_TICKS( 2 ));
              }
}
```

Trích mã 3.11. Đăng ký các API và xử lý yêu cầu HTTP trong WebTask (src/web/web_server.cpp).

Ngoài chức năng giám sát, Dashboard cho phép điều chỉnh ngưỡng COOLING từ 25–40 °C và LIGHTING từ 10–60%. Khi người dùng lưu cấu hình, trình duyệt gửi yêu cầu POST /api/config; máy chủ kiểm tra giá trị đầu vào và gọi updateControllerSettings() để cập nhật hai ngưỡng dưới sự bảo vệ của configMutex. Ngưỡng WARNING được cố định ở 45 °C và không cho phép thay đổi. Các giá trị cấu hình được lưu trong RAM và trở về mặc định khi khởi động lại hệ thống.

Giao diện Web Dashboard được minh họa trong Hình 3.7, thể hiện quá trình giám sát các cảm biến và điều chỉnh ngưỡng điều khiển trên trình duyệt.



Hình 3.7. Giao diện giám sát môi trường và cấu hình ngưỡng điều khiển trên Web Dashboard.

3.5.2. Thiết kế DisplayTask và hiển thị OLED

Hệ thống sử dụng màn hình OLED SSD1306 có độ phân giải 128 × 64 pixel, giao tiếp với ESP32-S3 thông qua I2C tại chân SDA GPIO8 và SCL GPIO9, địa chỉ thiết bị 0x3C. Trong hàm initDisplay() của tệp display.cpp, chương trình khởi tạo giao tiếp bằng Wire.begin(), kiểm tra màn hình thông qua display.begin() và hiển thị thông báo khởi động.

DisplayTask thực hiện đọc dữ liệu cảm biến từ sensorQueue và kết quả quyết định từ decisionQueue bằng xQueuePeek(). Khi cả hai Queue có dữ liệu, Task sử dụng các hàm

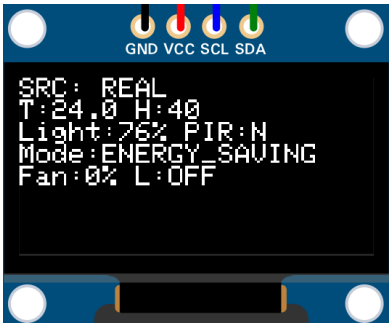
clearDisplay(), setCursor(), print() và println() để xây dựng nội dung hiển thị, sau đó gọi display.display() để cập nhật lên OLED. Quá trình xử lý chính được thể hiện trong Trích mã 3.12.

```
void DisplayTask ( void * parameter) {
SensorData sensor;
 DecisionData decision;
 while ( true ) {
        if (xQueuePeek(sensorQueue, & sensor, 0) == pdTRUE && xQueuePeek(decisionQueue, & decision, 0 ) == pdTRUE) {
            int fanPercent = ( int )((fanPwm / 255.0f ) * 100.0f );
            display.clearDisplay();
            display.setTextSize( 1 );
            display.setTextColor(SSD1306_WHITE);
            display.setCursor( 0 , 0 );
            display.print( "SRC: " );
            display.println( "REAL" );
            display.print( "T:" );
            display.print(sensor.temperature, 1 );
            display.print( " H:" );
            display.println(sensor.humidity, 0 );
            display.print( "Đèn:" );
            display.print(sensor.light, 0 );
            display.print( "% PIR:" );
            display.println(sensor.occupancy ? "Có" : "Không" );
            display.print( "Chế độ:" );
            display.println(modeName(decision.mode));
            display.print( "Quạt:" );
            display.print(fanPercent);
            display.print( "% L:" );
            display.println(lightOn ? "BẬT" : "TẮT" );
            display.display();
           }
```

```
        vTaskDelay(pdMS_TO_TICKS(DISPLAY_PERIOD_MS));
    }
}
```

Trích mã 3.12. Đọc dữ liệu từ Queue và cập nhật màn hình OLED trong DisplayTask (src/display/display.cpp).

Thông tin hiển thị gồm nhiệt độ, độ ẩm, mức sáng tương đối, trạng thái PIR, chế độ hoạt động, mức PWM quạt và trạng thái đèn. Giá trị PWM được quy đổi sang tỷ lệ phần trăm để thuận tiện theo dõi. Sau mỗi vòng xử lý, Task gọi vTaskDelay(pdMS_TO_TICKS(DISPLAY_PERIOD_MS)) với thời gian chờ 500 ms, tránh cập nhật màn hình liên tục. Kết quả hiển thị trên OLED trong mô phỏng Wokwi được minh họa ở Hình 3.8.



Hình 3.8. Kết quả hiển thị dữ liệu cảm biến và trạng thái hệ thống trên OLED SSD1306.

Việc sử dụng DisplayTask riêng giúp phân tách chức năng hiển thị khỏi quá trình thu thập cảm biến và điều khiển thiết bị. Dữ liệu được đọc từ Queue mà không xóa phần tử, cho phép các Task khác tiếp tục sử dụng thông tin hiện tại.

3.6. Thiết kế cơ chế đo và đánh giá thời gian thực

Để đánh giá khả năng đáp ứng thời gian thực của hệ thống, chương trình tích hợp các cơ chế ghi nhận độ trễ xử lý sự kiện PIR, thời gian phản hồi điều khiển, thời gian thực thi ControlTask và tỷ lệ đáp ứng deadline. Các chỉ số được tính từ bộ đếm tick FreeRTOS và hàm micros() của Arduino, sau đó lưu vào các biến dùng chung để theo dõi qua Web Dashboard. Bảng 3.10 tổng hợp các chỉ số và phương pháp đo được triển khai.

Chỉ số	Biến trong code	Phương pháp xác định
Độ trễ xử lý PIR	pirLatencyMs	Thời điểm PIRTask xử lý trừ thời điểm ISR

Thời gian phản hồi điều khiển	responseTimeMs	Hiệu tick từ sampleTick đến thời điểm ControlTask xử lý sau khi cập nhật đầu ra
Thời gian thực thi ControlTask	controlExecUs	Hiệu hai giá trị micros() tại thời điểm kết thúc và bắt đầu lượt xử lý
Số mẫu điều khiển	controlCycles	Tăng khi gặp sampleTick mới
Số lần vượt deadline	controlDeadlineMiss	Tăng khi responseTimeMs > 200 ms
Tỷ lệ đạt deadline	deadlineRate	$(N - M)/N \times 100\%$

Bảng 3.10. Các chỉ số đánh giá thời gian thực và phương pháp đo trong hệ thống.

Trong Bảng 3.10, N là tổng số mẫu điều khiển được thống kê thông qua biến controlCycles, còn M là số mẫu có thời gian phản hồi vượt quá ngưỡng deadline 200 ms, được ghi nhận bởi biến controlDeadlineMiss. Tỷ lệ đạt deadline được tính khi N lớn hơn 0; trường hợp chưa có mẫu, giá trị deadlineRate được khởi tạo bằng 0 trong quá trình tạo dữ liệu JSON của Web Dashboard.

Đối với cảm biến PIR, hàm pirISR() ghi nhận thời điểm xảy ra ngắt bằng xTaskGetTickCountFromISR(). Khi PIRTask nhận Binary Semaphore, Task sử dụng xTaskGetTickCount() để tính khoảng thời gian từ ngắt đến lúc xử lý, lưu trong biến pirLatencyMs. Phép đo phản ánh độ trễ đánh thức và lập lịch PIRTask theo độ phân giải tick của FreeRTOS.

Trong ControlTask, chương trình ghi nhận thời điểm bắt đầu vòng xử lý bằng micros() và tính thời gian thực thi bằng hiệu hai lần đọc bộ đếm, lưu tại controlExecUs. Đồng thời, thời gian từ mốc sampleTick của dữ liệu cảm biến đến khi ControlTask thực hiện cập nhật đầu ra được tính và lưu vào responseTimeMs. Cơ chế đo và kiểm tra deadline được thể hiện trong Trích mã 3.13.

```
// Đầu tiên, lấy mẫu xung nhịp ControlTask
const uint32_t startUs = micros();
 // Sau khi lấy mẫu đầu tiên, trong nhanh // xQueuePeek(decisionQueue, &decision, 0) thanh công:
if (sensor.sampleTick != lastAppliedTick) {
            lastAppliedTick = sensor.sampleTick;
              const TickType_t now = xTaskGetTickCount();
              responseTimeMs = (now - sensor.sampleTick) * portTICK_PERIOD_MS;
            if (responseTimeMs > maxResponseTimeMs)
                    maxResponseTimeMs = responseTimeMs;
                    controlCycles ++ ;
            if (responseTimeMs > CONTROL_DEADLINE_MS) { controlDeadlineMiss ++ ;
           }
} // Cuoi luot xu ly cua ControlTaskcontrolExecUs = micros() - startUs;
controlExecMs = controlExecUs / 1000 ;
```

Trích mã 3.13. Đo thời gian phản hồi và kiểm tra deadline trong ControlTask (src/control/control.cpp).

Hệ thống thiết lập CONTROL_DEADLINE_MS = 200 ms. Mỗi khi ControlTask xử lý một mốc sampleTick mới, biến controlCycles được tăng lên; nếu responseTimeMs vượt quá deadline, biến controlDeadlineMiss cũng tăng. Tỷ lệ đạt deadline được tính từ số mẫu đáp ứng thời hạn trên tổng số mẫu đã đánh giá. Các chỉ số được đóng gói thành JSON trong handleState() và cập nhật lên Web Dashboard để phục vụ theo dõi.

Các chỉ số thời gian được ghi nhận trong quá trình hoạt động của hệ thống và cung cấp cho Web Dashboard thông qua HTTP API. Dữ liệu này được sử dụng để đối chiếu thời gian xử lý giữa các tác vụ, thống kê số mẫu vượt deadline và đánh giá khả năng đáp ứng thời gian thực trong các kịch bản mô phỏng được trình bày ở Chương 4.

CHƯƠNG 4

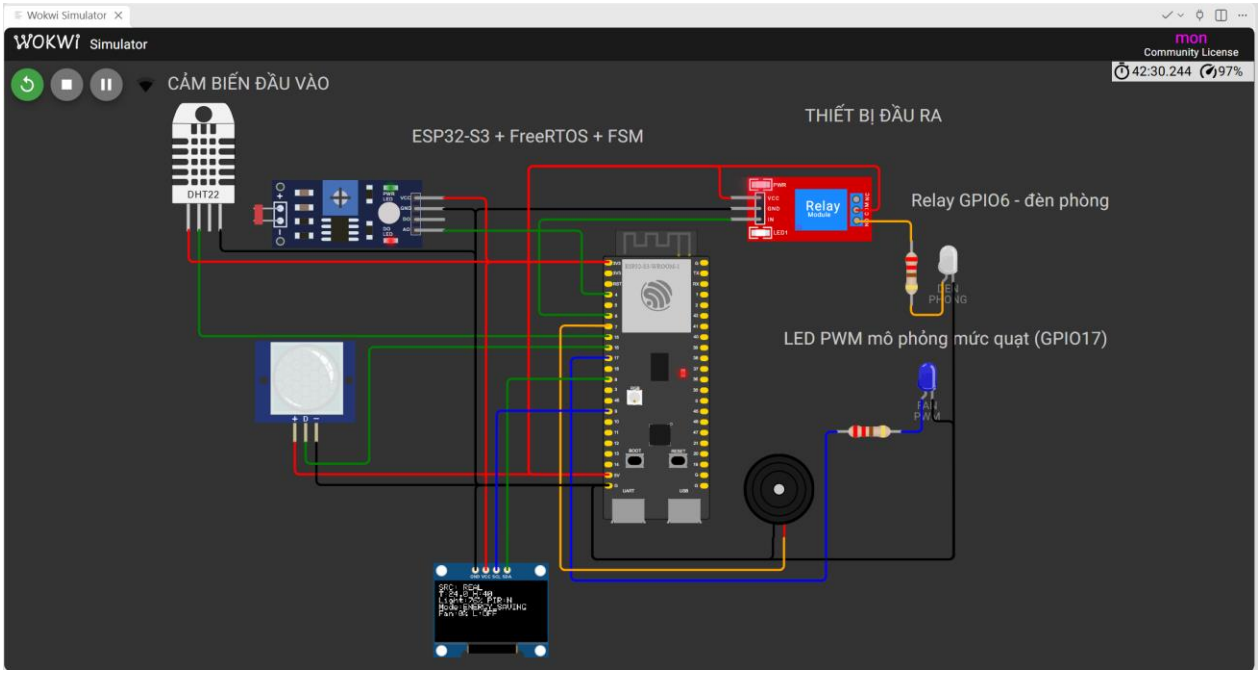
THỰC NGHIỆM, KIỂM CHỨNG VÀ ĐÁNH GIÁ

4.1. Kết quả thiết kế phần cứng mô phỏng

Hệ thống được triển khai trên ESP32-S3 DevKitC-1 và mô phỏng bằng Wokwi, gồm các cảm biến DHT22, LDR, PIR cùng relay đèn, LED PWM mô phỏng quạt, buzzer và OLED SSD1306. Chương trình biên dịch bằng PlatformIO sử dụng framework Arduino. Hình 4.1 thể hiện sơ đồ lắp nối của mô hình; Bảng 4.1 ghi các thông số cấu hình được sử dụng trong kiểm thử.

Nhóm	Thiết lập	Ý nghĩa
Phần cứng	ESP32-S3 DevKitC-1	Vi điều khiển trung tâm của mô hình
Môi trường	PlatformIO, Arduino-ESP32, Wokwi	Biên dịch và mô phỏng
Dữ liệu đầu vào	DHT22, LDR và PIR trên Wokwi	Dashboard chỉ theo dõi cảm biến từ mô phỏng
Ngưỡng điều khiển	COOLING mặc định 30 °C; LIGHTING mặc định 30%	Có thể cấu hình trên Web
Ngưỡng cảnh báo	WARNING = 45 °C	Cố định trong firmware
Thời gian	SensorTask 2000 ms; DisplayTask 500 ms	Chu kỳ cập nhật theo config.h
Deadline	200 ms	Mốc đánh giá từ thời điểm ghi mẫu đến lúc ControlTask xử lý

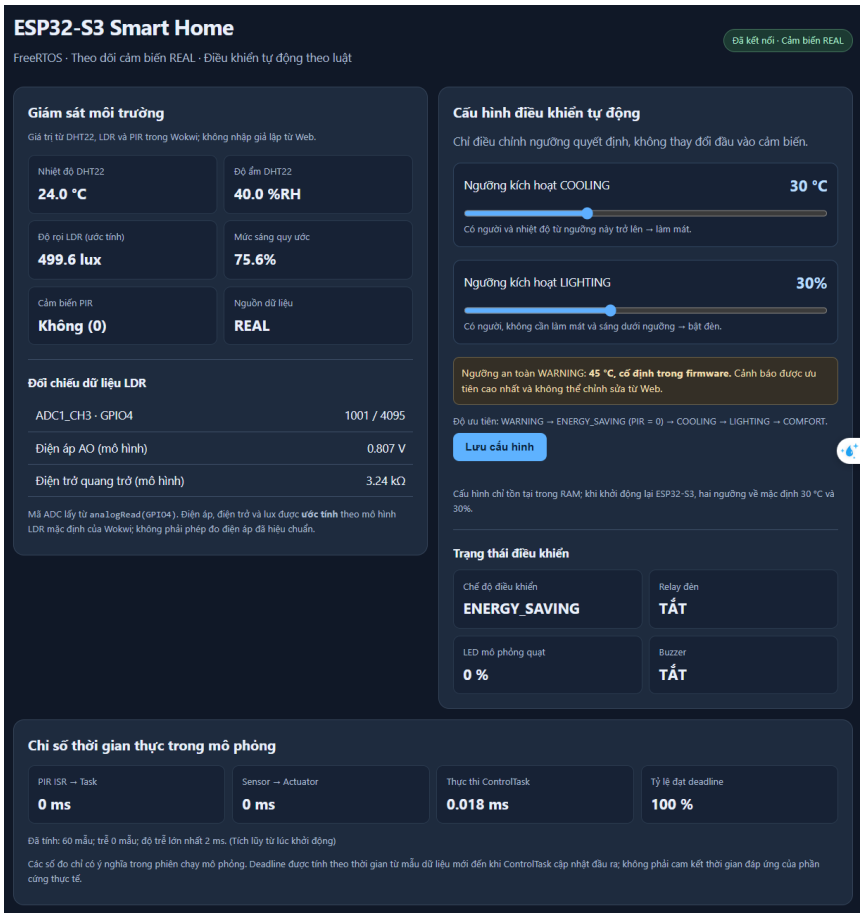
Bảng 4.1. Cấu hình dùng làm cơ sở kiểm thử



Hình 4.1. Giao diện Wokwi và phần cứng mô phỏng của hệ thống

4.2. Kết quả thiết kế phần mềm

Phần mềm được chia theo chức năng: SensorTask và PIRTask cập nhật dữ liệu, DecisionTask lựa chọn chế độ, ControlTask tác động lên relay, PWM và buzzer; DisplayTask và WebTask thực hiện hiển thị và phục vụ HTTP. Các Task truyền mẫu mới qua Queue và thông báo trực tiếp để giảm thời gian chờ. Phần triển khai chi tiết đã trình bày ở Chương 3; mục này chỉ ghi nhận kết quả tích hợp.



Hình 4.2. Web Dashboard giám sát cảm biến, cấu hình ngưỡng và hiển thị trạng thái điều khiển.

Trên ESP32-S3, các tác vụ FreeRTOS phối hợp đọc cảm biến, xử lý sự kiện PIR, lựa chọn chế độ FSM, điều khiển thiết bị và cập nhật hai kênh hiển thị OLED, Web Dashboard. Dữ liệu được thu thập từ các cảm biến mô phỏng trên Wokwi; Dashboard dùng để giám sát và cập nhật hai ngưỡng COOLING, LIGHTING, không điều khiển trực tiếp các thiết bị đầu ra.

Web Dashboard hiển thị nhiệt độ, độ ẩm, trạng thái chuyển động, độ rọi ước tính, mã ADC và mức sáng quy ước. Phần trạng thái điều khiển cho biết chế độ FSM hiện tại, trạng thái relay, tốc độ quạt PWM và buzzer; các chỉ số thời gian được ghi nhận trong cùng phiên mô phỏng.

Màn hình OLED SSD1306 sử dụng I2C địa chỉ 0x3C, hiển thị nhiệt độ, độ ẩm, mức sáng, trạng thái PIR, chế độ hoạt động và trạng thái thiết bị. Thông tin hiển thị bao gồm nhiệt độ, độ ẩm, mức sáng, trạng thái PIR, chế độ điều khiển và trạng thái các thiết bị đầu ra.

Trình duyệt nhận giao diện từ GET /, đọc trạng thái qua GET /api/state và đọc cấu hình qua GET /api/config. Hai giá trị coolingTempC và darkLightPct được gửi bằng POST

/api/config khi người dùng nhấn "Lưu cấu hình". Phía firmware kiểm tra giới hạn ngưỡng và cập nhật đồng bộ dưới configMutex, sau đó gửi thông báo tới DecisionTask; Dashboard không gửi lệnh bật, tắt trực tiếp tới thiết bị.

Yêu cầu	Chức năng	Định dạng
GET /	Trả giao diện Web Dashboard	HTML, CSS và JavaScript
GET /api/state	Trả trạng thái cảm biến, FSM, đầu ra và các chỉ số thời gian	JSON
GET /api/config	Đọc hai ngưỡng hiện tại, phiên bản cấu hình và WARNING cố định	JSON
POST /api/config	Nhận coolingTempC và darkLightPct; kiểm tra giới hạn và ghi cấu hình	Form fields; phản hồi JSON

Bảng 4.2. Các HTTP API phục vụ giám sát và cấu hình

Hai ngưỡng COOLING (25–40 °C) và LIGHTING (10–60%) có thể thay đổi trên Dashboard, chỉ lưu trong RAM và trở về giá trị mặc định sau khi khởi động lại. Ngưỡng WARNING = 45 °C cố định trong firmware. Ảnh giao diện xác nhận có chức năng cấu hình; chưa có bộ ảnh phản hồi HTTP riêng để kiểm tra độc lập các mã trạng thái trả về.

4.3. Mô phỏng một số kịch bản

Các kịch bản được thực hiện bằng cách thay đổi điều kiện cảm biến trên Wokwi hoặc cấu hình ngưỡng qua Dashboard, sau đó đối chiếu số đo, chế độ FSM và thiết bị đầu ra. Nhóm sử dụng các giá trị thực tế được ghi nhận vì thanh trượt không cho phép chọn chính xác mọi điểm ngưỡng.

4.3.1. Kiểm thử cảm biến DHT22 và PIR

SensorTask đọc nhiệt độ và độ ẩm bằng thư viện DHTesp; PIRTask nhận semaphore do ISR phát ra khi GPIO16 thay đổi trạng thái. Kết quả nhiệt độ và độ ẩm thể hiện dưới đơn vị °C, %RH; PIR được biểu diễn bởi hai trạng thái logic 0 và 1. Trong source, khi có thay đổi chuyển động, PIRTask cập nhật bản ghi trong sensorQueue và thông báo DecisionTask để xử lý lại luật điều khiển.

Đầu vào	Giá trị đặt trong Wokwi	Dữ liệu quan sát
DHT22	46,3 °C; 40,0% RH	Dashboard: 46,3 °C; 40,0% RH
PIR – không chuyển động	Tín hiệu PIR = 0	Dashboard: Không (0)

Đầu vào	Giá trị đặt trong Wokwi	Dữ liệu quan sát
PIR – có chuyển động	Kích hoạt Simulate motion	Dashboard: Có tín hiệu (1)

Bảng 4.3. Đối chiếu dữ liệu đầu vào từ DHT22 và PIR



Hình 4.3. Bộ điều khiển DHT22 và kích hoạt cảm biến PIR trên Wokwi.

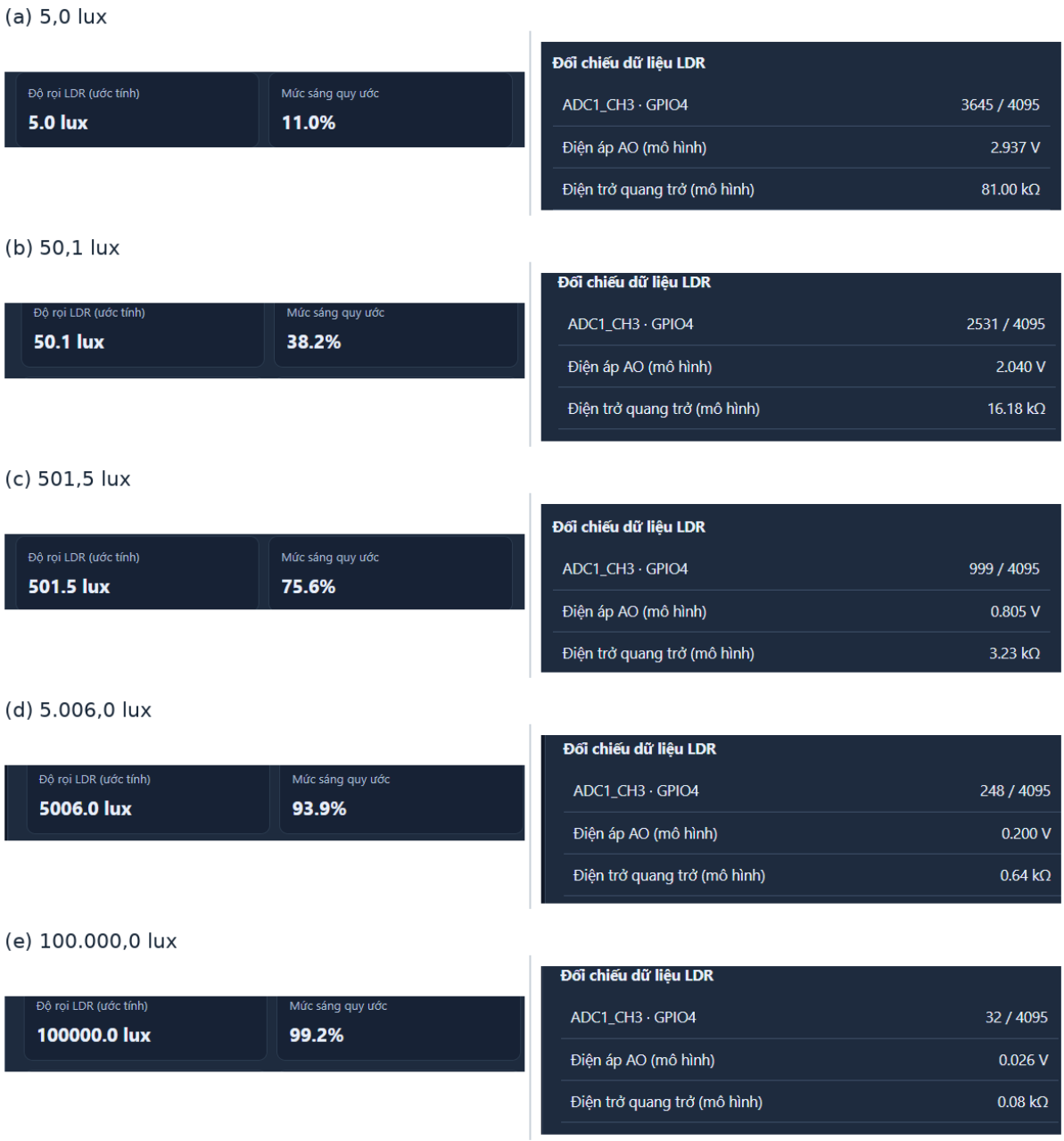
Hình 4.3 minh họa thao tác thay đổi DHT22 và kích hoạt sự kiện PIR. Bảng 4.3 đối chiếu giá trị đầu vào với kết quả giám sát; ảnh này không thay thế phép thử ngưỡng cảnh báo hay kiểm thử chi tiết thuật toán FSM ở mục 4.3.3.

4.3.2. Kiểm thử mô hình LDR và chuyển đổi ADC

Trong Wokwi, độ rọi E (lux) là giá trị đầu vào mô phỏng. Theo mô hình LDR, tín hiệu biến đổi theo chuỗi E →R_LDR →V_AO của mạch phân áp 3,3 V (R_F = 10 kΩ) →mã ADC 12 bit ở GPIO4 (ADC1_CH3) →Light (%) phục vụ FSM. Chương trình không đọc trực tiếp lux từ Wokwi, mà nhận mã ADC tại GPIO4 rồi ước tính lại điện áp, điện trở và độ rọi theo mô hình.

STT	E (lux)	R (kΩ)	AO (V)	ADC	Light (%)
1	5,0	81,00	2,937	3645	11,0
2	50,1	16,18	2,040	2531	38,2
3	501,5	3,23	0,805	999	75,6
4	5.006,0	0,64	0,200	248	93,9
5	100.000,0	0,08	0,026	32	99,2

Bảng 4.4. Đối chiếu các mức độ rọi đầu vào và kết quả chuyển đổi LDR



Hình 4.4. Năm mức đo LDR trên Web Dashboard: (a) 5,0; (b) 50,1; (c) 501,5; (d) 5.006,0 và (e) 100.000,0 lux.

Năm ảnh Dashboard trong Hình 4.4 minh chứng trực tiếp cho Bảng 4.4; log Terminal được lưu kèm để kiểm tra chuỗi chuyển đổi ADC, điện áp AO, điện trở và lux. Đây là các giá trị mô hình, không phải kết quả hiệu chuẩn trên phần cứng thật.

Xét mức 2 có độ rọi E ≈ 50,1 lux. Với tham số của mô hình LDR (R_{10} = 50 kΩ tại 10 lux, γ = 0,7), các đại lượng được tính theo chuỗi độ rọi →điện trở →điện áp →mã ADC lý tưởng →phần trăm ánh sáng:

$$R_{\mathrm{LDR}} = 50\left(\frac{10}{50{,}1}\right)^{0{,}7} \approx 16{,}18\,\mathrm{k}\Omega$$

$$V_{AO} = 3{,}3 \times \frac{16{,}18}{10 + 16{,}18} \approx 2{,}040\,\mathrm{V}$$

$$N_{ADC} \approx \frac{2{,}040}{3{,}3} \times 4095 \approx 2531$$

$$\mathrm{Light}(\%) = 100\left(1 - \frac{2531}{4095}\right) \approx 38{,}2\%$$

Dữ liệu ở năm mức cho thấy khi độ rọi tăng từ 5,0 lên 100.000,0 lux, mã ADC giảm từ 3645 xuống 32 và phần trăm ánh sáng tăng từ 11,0% lên 99,2%. Điều này phù hợp với chiều biến thiên của mạch phân áp trong mô hình mô phỏng. Điện áp, điện trở và lux ở đây là các giá trị ước tính từ mô hình Wokwi, không phải phép đo điện áp và độ rọi đã hiệu chuẩn trên phần cứng vật lý.

4.3.3. Kiểm thử các chế độ điều khiển

Bộ lựa chọn chế độ trong source hiện hành thực hiện luật theo thứ tự WARNING → ENERGY_SAVING → COOLING → LIGHTING → COMFORT. Điều kiện ưu tiên cao được kiểm tra trước điều kiện thấp hơn, vì vậy ở mức nhiệt cao hơn hoặc bằng 45 °C, hệ thống chuyển WARNING ngay cả khi PIR = 0. Nếu không cảnh báo và PIR = 0, chế độ ENERGY_SAVING được lựa chọn bất kể điều kiện làm mát hoặc chiếu sáng.

Chế độ	**Điều kiện điển hình**	**Kết quả và phạm vi xác nhận**
WARNING	45,6 °C; PIR = 1 hoặc 46,3 °C; PIR = 0	Đã quan sát WARNING; buzzer bật, quạt 100%
ENERGY_SAVING	35,3 °C; PIR = 0	Đã quan sát đầu ra tắt
COOLING	31,6 °C; PIR = 1; sáng 75,6%; ngưỡng 31 °C	Đã quan sát COOLING; quạt 70%
LIGHTING	26,4 °C; PIR = 1; E = 21,9 lux; Light = 25,7%; ngưỡng LIGHTING hiển thị 16%.	Ảnh ghi nhận chế độ LIGHTING: relay bật, quạt 0%, buzzer tắt. Mức sáng 25,7% cao hơn ngưỡng 16% đang hiển thị nên ảnh chưa chứng minh sự phù hợp của điều kiện kích hoạt theo ngưỡng đó.
COMFORT	26,4 °C; PIR = 1; sáng 75,6%	Đã quan sát COMFORT và các đầu ra tắt

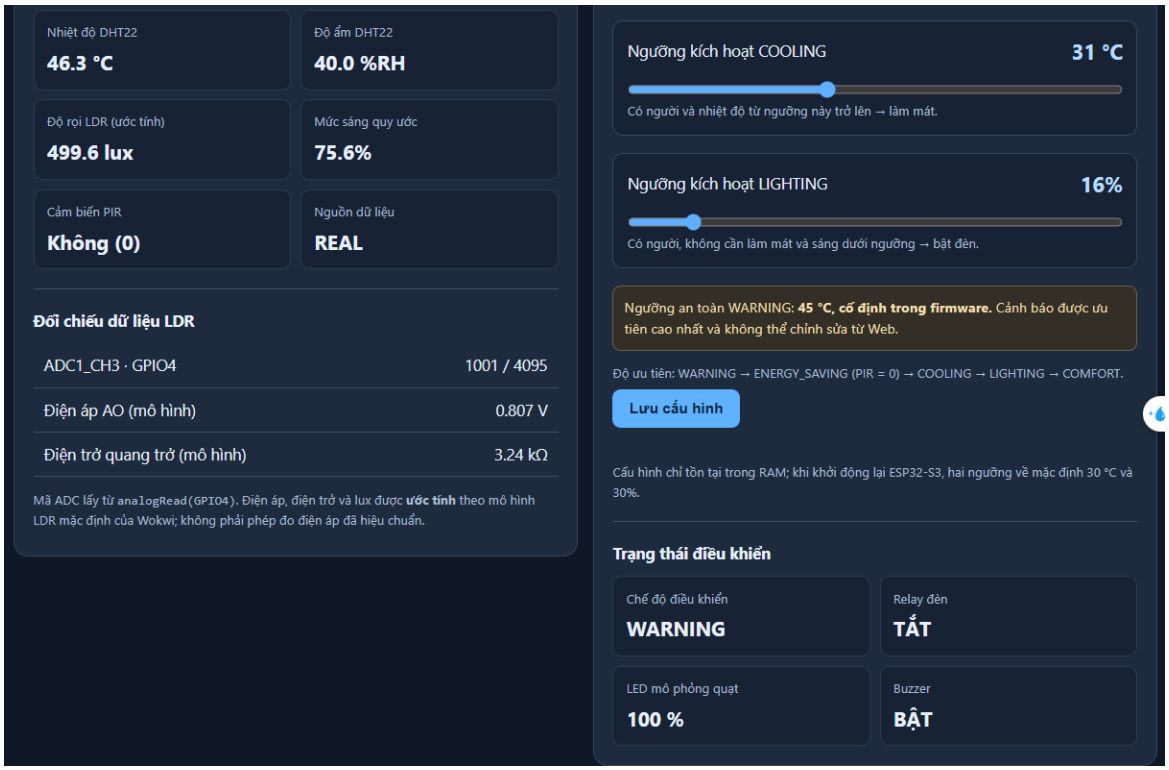
Bảng 4.5. Đối chiếu điều kiện thử nghiệm và các chế độ FSM

ControlTask nhận bản sao DecisionData từ decisionQueue và ghi trạng thái đến GPIO6 (relay đèn), GPIO17 (PWM quạt) và GPIO7 (buzzer). Việc thực thi chuyển chế độ được thể hiện bằng thay đổi các tín hiệu đầu ra; các trường hợp trong Bảng 4.6 phản ánh đúng luật của hàm ControlTask hiện hành.

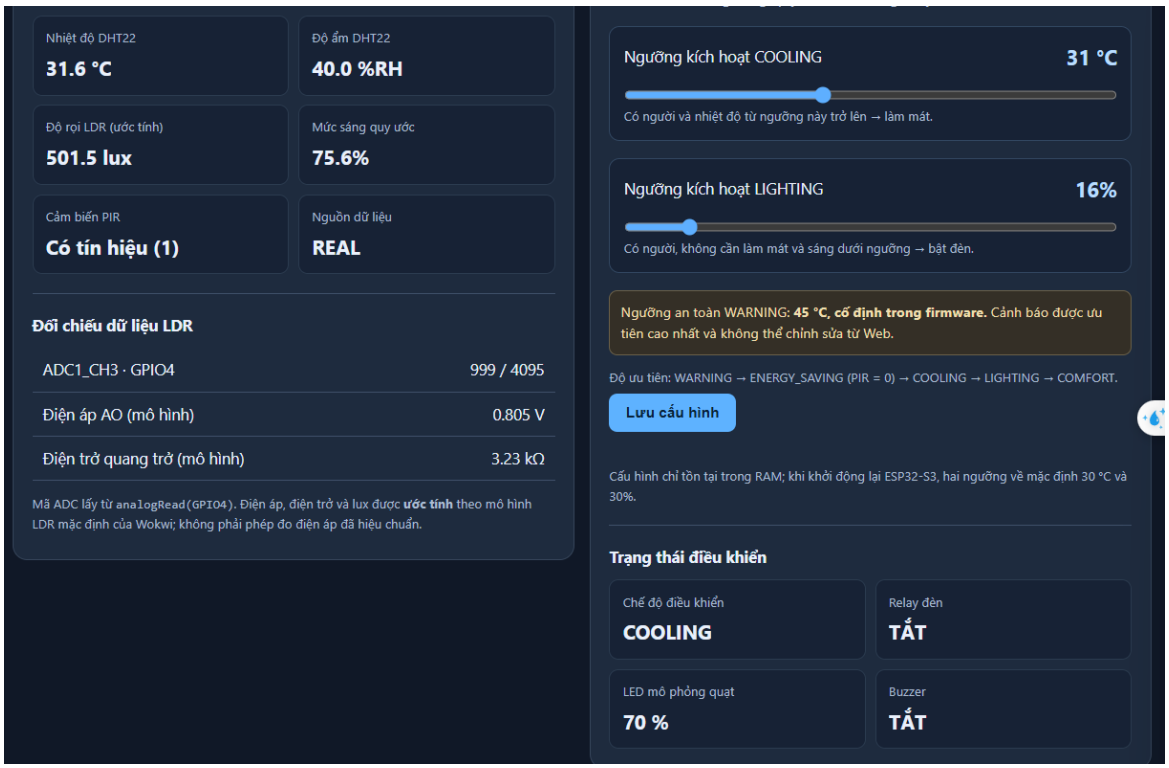
Chế độ	**Relay đèn**	**LED PWM quạt**	**Buzzer**
COMFORT	Tắt	0%	Tắt
ENERGY_SAVING	Tắt	0%	Tắt
LIGHTING	Bật	0%	Tắt
COOLING	Phụ thuộc mức sáng	70% hoặc 100% theo nhiệt độ	Tắt
WARNING	Tắt	100%	Bật

Bảng 4.6. Trạng thái đầu ra tương ứng các chế độ

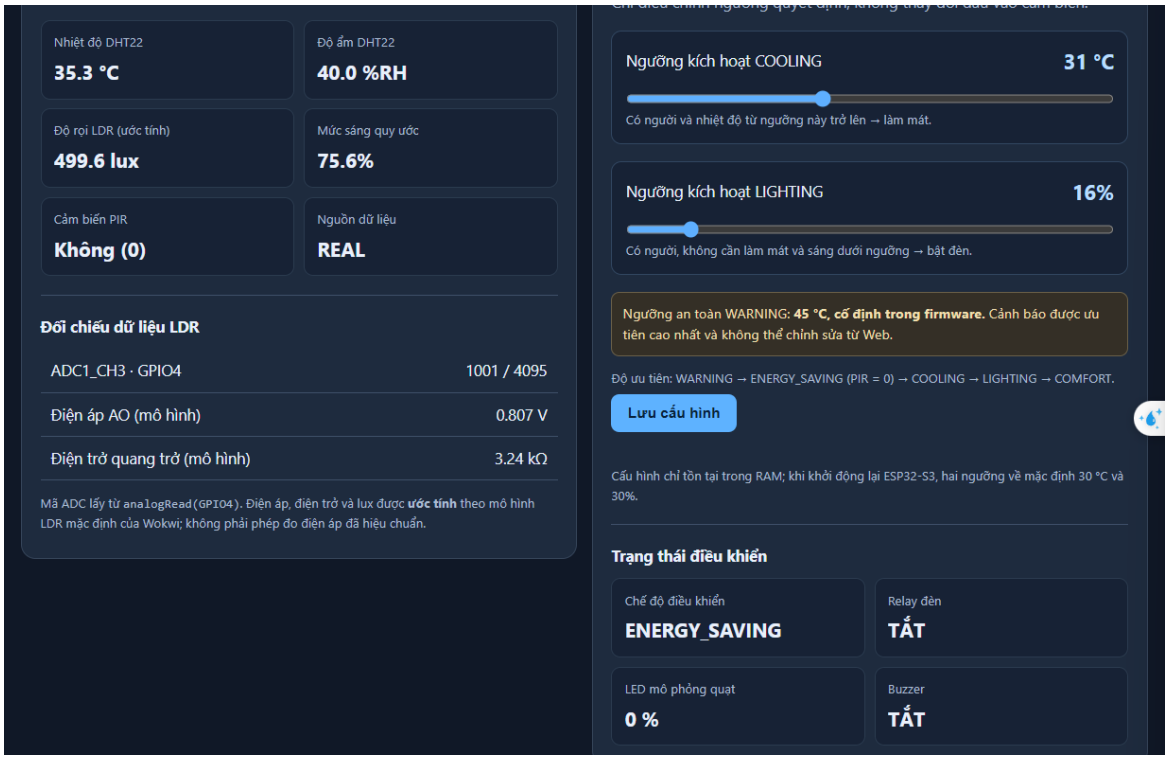
Ở COOLING, quạt có thể chạy 70% hoặc 100% tùy mức nhiệt; relay đèn còn phụ thuộc độ sáng. Bảng 4.6 mô tả hành vi điều khiển theo thiết kế, còn các ảnh thể hiện những tổ hợp đầu vào cụ thể đã quan sát.



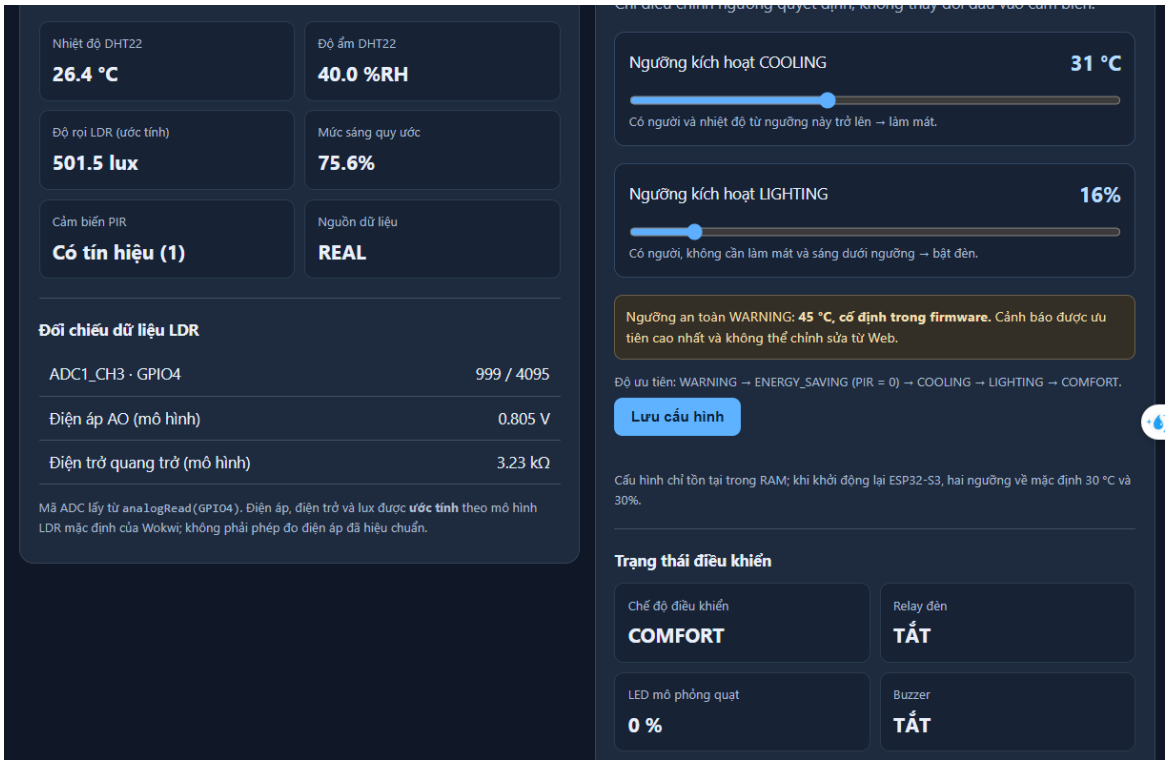
Hình 4.5. Kết quả WARNING: 46,3 °C, PIR = 0, buzzer bật và quạt 100%.



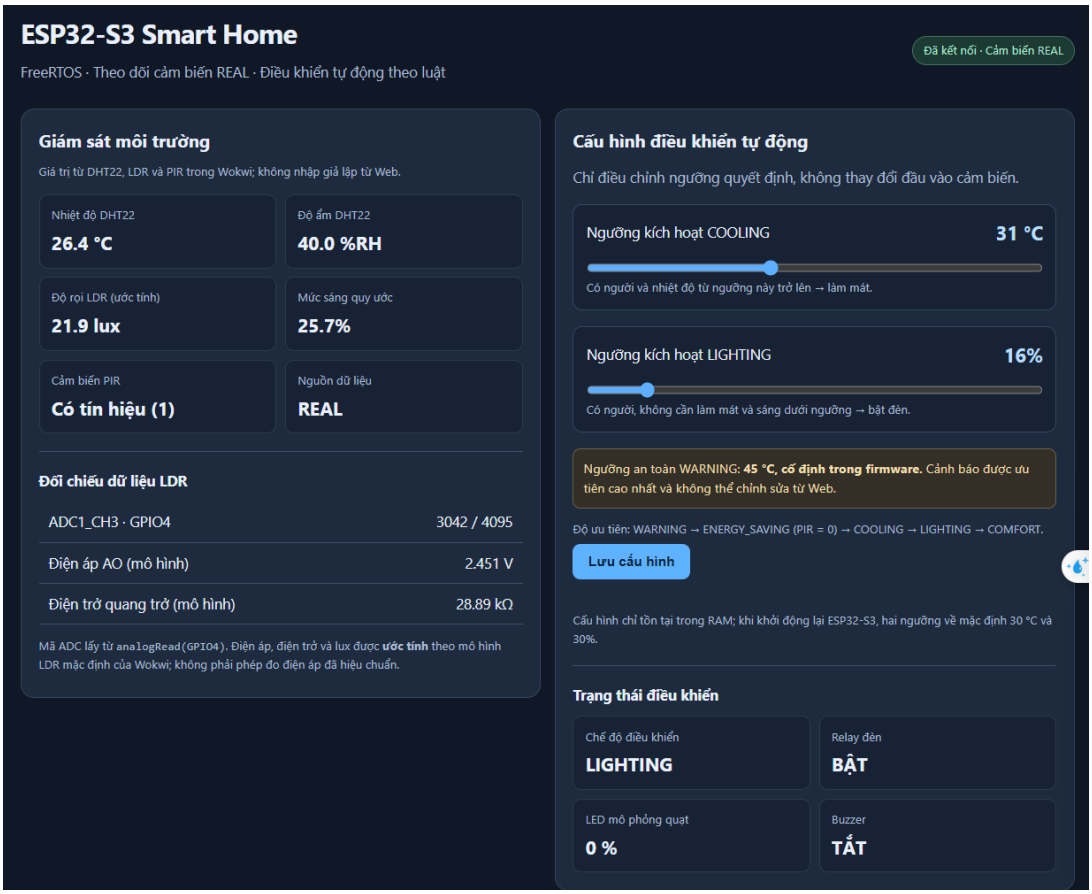
Hình 4.6. Kết quả COOLING: 31,6 °C, PIR = 1 và LED mô phỏng quạt 70%.



Hình 4.7. Trạng thái ENERGY_SAVING khi PIR = 0: các thiết bị đầu ra tắt.



Hình 4.8. Trạng thái COMFORT với PIR = 1, nhiệt độ 26,4 °C và ánh sáng 75,6%.



Hình 4.9. Kết quả quan sát LIGHTING: 26,4 °C; 21,9 lux; PIR = 1; relay bật và quạt 0%.

Hình 4.5–4.9 minh họa kết quả quan sát của năm chế độ FSM. Riêng ảnh LIGHTING ghi nhận 26,4 °C, PIR = 1, độ rọi 21,9 lux, mức sáng 25,7% và relay bật. Ngưỡng 16% đang hiển thị thấp hơn mức sáng đo được, nên ảnh xác nhận trạng thái quan sát nhưng chưa chứng minh điều kiện so sánh ngưỡng 16% đã được thực thi chính xác.

4.3.4. Kiểm chứng khả năng đáp ứng thời gian thực

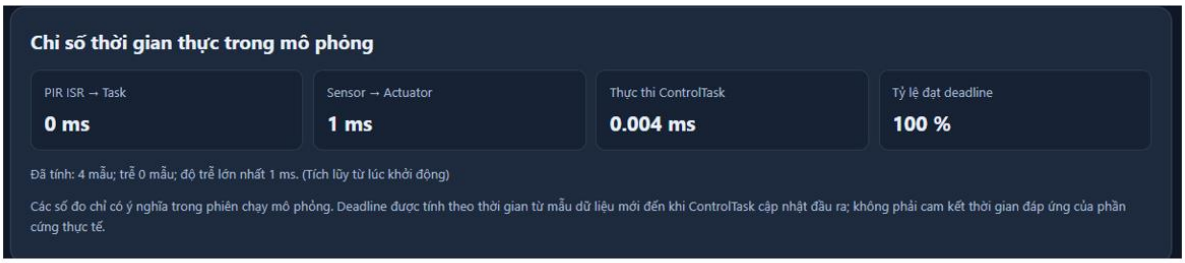
Trong source của báo cáo, SensorTask thông báo cho DecisionTask sau khi cập nhật sensorQueue; DecisionTask thông báo trực tiếp cho ControlTask sau khi ghi decisionQueue. Cơ chế này nhằm giảm thời gian chờ do kiểm tra Queue theo chu kỳ. Chỉ số Sensor →Actuator được tính theo hiệu tick FreeRTOS từ lúc mẫu SensorData được ghi nhận đến khi ControlTask xử lý bản quyết định có cùng dấu thời gian mẫu.

Bảng 4.7 và Hình 4.10 thể hiện năm mốc tích lũy của cùng quá trình mô phỏng: 4, 9, 60, 152 và 199 mẫu. Các mốc này không tương đương năm lần chạy độc lập; deadline cấu hình là 200 ms kể từ dấu thời gian ghi mẫu cảm biến tới khi ControlTask cập nhật đầu ra.

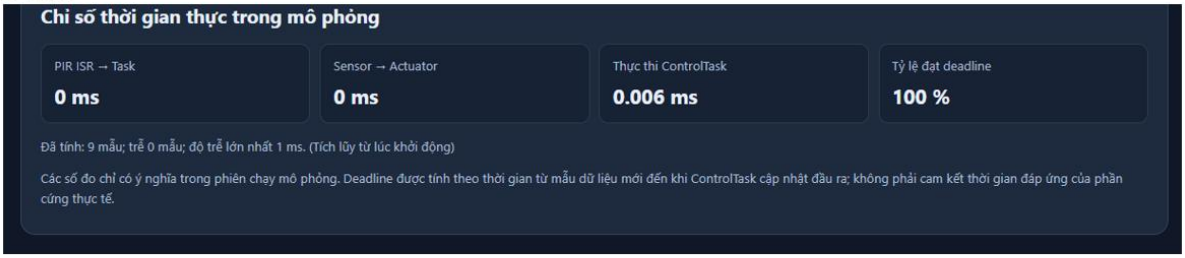
Mốc ảnh	Mẫu tích lũy	Mẫu trễ	Trễ lớn nhất	Đạt deadline
1	4	0	1 ms	100%
2	9	0	1 ms	100%
3	60	0	2 ms	100%
4	152	0	2 ms	100%
5	199	0	2 ms	100%

Bảng 4.7. Các mốc thống kê deadline trên source báo cáo

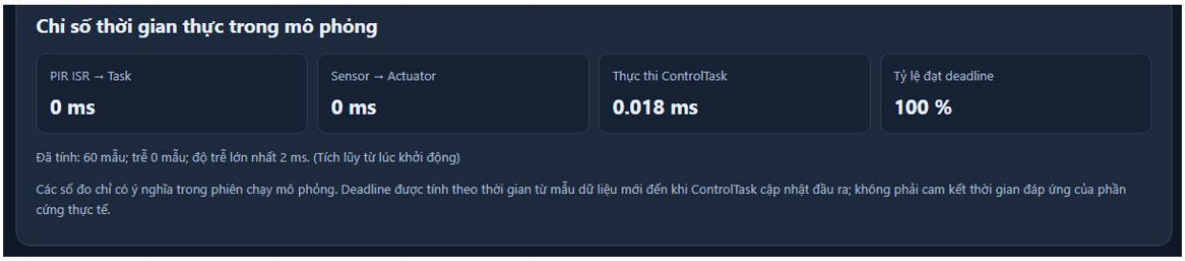
4 mẫu



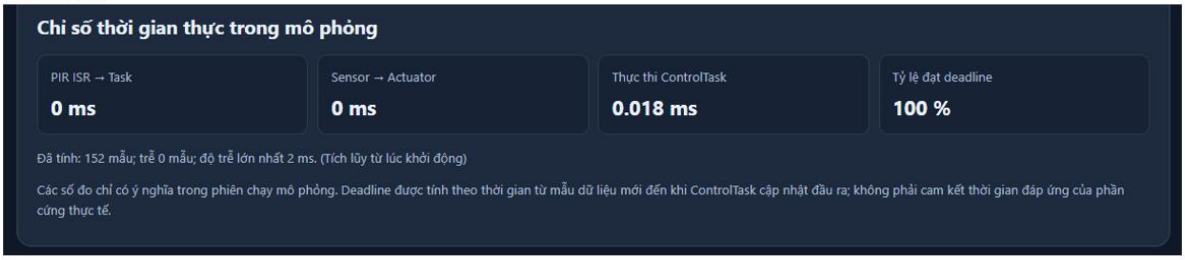
9 mẫu



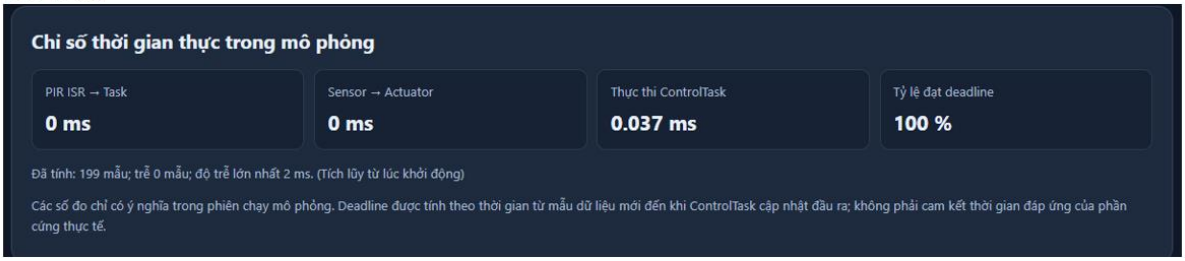
60 mẫu



152 mẫu



199 mẫu



Hình 4.10. Năm mốc thống kê deadline từ Dashboard của source báo cáo.

Tại thời điểm ghi nhận mốc 199 mẫu, thời gian thực thi ControlTask hiển thị 0,037 ms và Sensor →Actuator hiển thị 0 ms. Các chỉ số tức thời này chỉ ứng với lần xử lý gần

nhất, không phải giá trị trung bình của 199 mẫu. Giá trị 0 ms không chứng minh thời gian vật lý bằng không, do bộ đếm dùng tick FreeRTOS có độ phân giải hữu hạn.

Tỷ lệ đạt được tính theo số mẫu được ControlTask tính trong phiên mô phỏng, với mẫu trễ là mẫu có độ trễ lớn hơn 200 ms. Ở mốc 199 mẫu đã có 0 mẫu trễ, do đó tỷ lệ đạt deadline của các mẫu được thống kê tại mốc đó bằng 100%.

$$\text{Tỷ lệ đạt deadline} = (N - N_\text{trễ}) / N \times 100\%$$

$$= (199 - 0) / 199 \times 100\% = 100\%$$

Các mẫu đã thống kê tại mốc 199 đều đạt deadline 200 ms, phản ánh độ trễ thấp trong điều kiện kiểm thử tương ứng. Tuy nhiên, phần lớn bằng chứng ghi nhận chế độ ENERGY_SAVING ổn định với PIR = 0; vì vậy chưa thể khẳng định tỷ lệ tương tự khi đổi chế độ liên tục hoặc khi thiết bị chịu tải lớn.

Ngoài ra, SensorTask đọc DHT22 theo chu kỳ 2000 ms. Deadline 200 ms trong chương trình chỉ tính từ thời điểm tạo mẫu cảm biến mới đến thời điểm cập nhật đầu ra, không tính thời gian chờ vật lý trước lần lấy mẫu. Bởi vậy, không được diễn giải đây là cam kết phản hồi mọi biến thiên nhiệt độ trong 200 ms.

4.4. Nhận xét và đánh giá kết quả

Các ảnh kiểm thử xác nhận mô hình thu thập được dữ liệu DHT22, PIR và ADC LDR; kết quả của năm mức sáng nhất quán với chiều biến thiên của mạch phân áp. Cả năm trạng thái FSM đã được ghi nhận bằng hình ảnh, trong đó LIGHTING có đầy đủ thông số độ rọi, mức sáng và trạng thái đầu ra; ảnh này chưa xác nhận được tính nhất quán của điều kiện kích hoạt vì mức sáng 25,7% lớn hơn ngưỡng 16% hiển thị. Dashboard giám sát dữ liệu cảm biến và hỗ trợ cấu hình hai ngưỡng; bộ ảnh OLED cận cảnh và phản hồi HTTP lưu cấu hình chưa đủ để đánh giá chi tiết.

Trong phiên mô phỏng dùng, mốc 199 mẫu ghi nhận 0 mẫu trễ, độ trễ lớn nhất 2 ms và tỷ lệ đạt deadline 100%. Chỉ số này không bao gồm toàn bộ độ trễ vật lý từ khi môi trường biến thiên đến lần lấy mẫu DHT22 tiếp theo, và chưa được xác thực trên bo mạch thật. Các giá trị lux và điện áp là ước tính từ mô hình, không phải số đo đã hiệu chuẩn.

CHƯƠNG 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Đề tài đã xây dựng được mô hình hệ thống nhà thông minh trên ESP32-S3 DevKitC-1 với môi trường mô phỏng Wokwi và hệ điều hành thời gian thực FreeRTOS. Hệ thống thu thập nhiệt độ, độ ẩm từ DHT22, tín hiệu chuyển động từ PIR và tín hiệu analog của LDR qua ADC1_CH3. Các dữ liệu được xử lý và cung cấp cho bộ quyết định theo luật, đồng thời hiển thị trên OLED SSD1306 và Web Dashboard.

Phần mềm được tổ chức thành các Task chuyên trách và sử dụng Queue, semaphore, mutex cùng cơ chế thông báo Task để phối hợp xử lý. Bộ luật FSM lựa chọn năm chế độ WARNING, ENERGY_SAVING, COOLING, LIGHTING và COMFORT theo thứ tự ưu tiên; các đầu ra gồm relay đèn, LED PWM mô phỏng quạt và buzzer. Trên giao diện Web, người dùng có thể theo dõi dữ liệu cảm biến và cấu hình hai ngưỡng COOLING, LIGHTING; WARNING được cố định ở 45 °C trong firmware.

Kết quả kiểm thử cho thấy các giá trị DHT22 và PIR được phản ánh trên Dashboard; năm mức độ rọi mô phỏng xác nhận quan hệ nhất quán giữa độ rọi, điện trở LDR, điện áp AO, mã ADC và phần trăm ánh sáng quy ước. Các tình huống thử nghiệm đã ghi nhận hoạt động tương ứng của các chế độ điều khiển. Với source được sử dụng trong báo cáo, tại mốc 199 mẫu tích lũy có 0 mẫu vượt deadline 200 ms, độ trễ lớn nhất trong tập mẫu là 2 ms và tỷ lệ đạt deadline là 100%. Đây là kết quả trong phạm vi phép thử đã thực hiện, chủ yếu khi ENERGY_SAVING ổn định; không đại diện cho mọi điều kiện vận hành.

Nhìn chung, mô hình đáp ứng mục tiêu tích hợp cảm biến, xử lý đa tác vụ, điều khiển tự động theo ngưỡng và giám sát trên giao diện Web. Do mới triển khai trên môi trường mô phỏng, đề tài chưa chứng minh được độ chính xác tuyệt đối của cảm biến, hiệu năng khi tải cao hoặc chất lượng đáp ứng trên phần cứng vật lý. Một số minh chứng chi tiết về OLED, phản hồi cập nhật ngưỡng và trường hợp LIGHTING còn cần bổ sung hoặc kiểm chứng thêm.

5.2. Hướng phát triển

Trong giai đoạn tiếp theo, nhóm có thể triển khai hệ thống trên bo mạch ESP32-S3 và cảm biến thật, sau đó hiệu chuẩn tín hiệu ADC, đối chiếu lux với thiết bị đo chuẩn và đánh giá sai số của nhiệt độ, độ ẩm. Việc thay LED mô phỏng quạt bằng bộ điều khiển quạt thực tế cũng giúp kiểm tra đáp ứng PWM, công suất tải và độ an toàn của ngõ ra.

Đối với phần mềm thời gian thực, cần mở rộng bộ kiểm thử theo nhiều phiên chạy độc lập, các đợt chuyển trạng thái liên tục và các điều kiện tải Web khác nhau. Nên bổ

sung thống kê độ trễ trung bình, cực đại, phân vị và số lần trễ hạn; đồng thời đo thời gian từ khi môi trường thay đổi đến lúc hệ thống thực sự cập nhật đầu ra. Cách này cho phép đánh giá đầy đủ hơn thay vì chỉ dựa vào độ trễ tính từ dấu thời gian của mẫu SensorData.

Ở lớp ứng dụng, có thể lưu cấu hình ngưỡng vào NVS để không mất khi khởi động lại, bổ sung cơ chế xác thực khi thay đổi ngưỡng và lưu lịch sử cảm biến để phân tích xu hướng. Nhóm cũng có thể mở rộng luật FSM với điều kiện độ ẩm, bổ sung hysteresis để hạn chế chuyển chế độ liên tục gần ngưỡng, đồng thời cải thiện giao diện cảnh báo và khả năng truy cập trên thiết bị di động.

Các hướng trên cần được kiểm thử và đối chiếu với nguồn lực thực tế trước khi triển khai; chúng là đề xuất mở rộng, chưa phải chức năng đã được hiện thực và xác nhận trong phiên bản mô phỏng của đề tài.

TÀI LIỆU THAM KHẢO

[1] Rhee, J. H., Ma, J. H., Seo, J., và Cha, S. H. (2022). "Review of applications and user perceptions of smart home technology for health and environmental monitoring". Journal of Computational Design and Engineering, 9(3), 857–889. https://doi.org/10.1093/jcde/qwac030.

[2]Bakare, M. S., và Abubaker, K. (2026). "IoT-based indoor environmental monitoring system using multi-parameter sensing and ESP32-WROOM integration". Discover Electronics, 3, bài số 6. https://doi.org/10.1007/s44291-026-00157-3.

[3]Gao, R. (2025). "A Comparative Study of an STM32F103C8T6-Based FreeRTOS Smart Home System for Environmental Monitoring and Control". Informatica, 49(24), 401–416. https://doi.org/10.31449/inf.v49i24.11427.

[4] Espressif Systems. (2026). ESP32-S3 Series Datasheet, phiên bản 2.2. https://documentation.espressif.com/esp32-s3_datasheet_en.pdf.

[5] Espressif Systems. (2026). ESP32-S3 Technical Reference Manual, phiên bản 1.8. https://documentation.espressif.com/esp32-s3_technical_reference_manual_en.pdf.

[6] Espressif Systems. Arduino ESP32 Documentation (Arduino API và các ngoại vi ESP32). https://docs.espressif.com/projects/arduino-esp32/en/latest/.

[7] Espressif Systems. ESP32-S3-DevKitC-1 User Guide (thông tin sơ đồ chân bo mạch). https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32s3/esp32-s3-devkitc-1/index.html.

[8] Espressif Systems. Arduino ESP32: GPIO API (digitalRead, digitalWrite, attachInterrupt). https://docs.espressif.com/projects/arduino-esp32/en/latest/api/gpio.html.

[9] Arduino. tone() — Arduino Language Reference. https://docs.arduino.cc/language-reference/en/functions/advanced-io/tone/.

[10] Espressif Systems. Arduino ESP32: ADC API (tài liệu về analogRead, độ phân giải và suy hao ADC). https://docs.espressif.com/projects/arduino-esp32/en/latest/api/adc.html.

[11] Wokwi Docs. wokwi-photoresistor-sensor Reference (đặc tuyến RL10, gamma và mạch phân áp LDR). https://docs.wokwi.com/parts/wokwi-photoresistor-sensor.

[12] Wokwi Docs. wokwi-dht22 Reference (tài liệu tham chiếu cảm biến DHT22 trong mô phỏng). https://docs.wokwi.com/parts/wokwi-dht22.

[13] Wokwi Docs. ESP32 Simulation (cấu hình và khả năng mô phỏng ESP32-S3). https://docs.wokwi.com/guides/esp32.

[14]Adafruit. Adafruit SSD1306 Arduino Library (thư viện OLED SSD1306). https://github.com/adafruit/Adafruit_SSD1306.

[15]Adafruit. Adafruit GFX Library (thư viện đồ họa dùng với OLED). https://github.com/adafruit/Adafruit-GFX-Library.

[16] FreeRTOS. FreeRTOS Kernel Documentation: Queues, Mutexes and Semaphores. https://www.freertos.org/Documentation/02-Kernel/02-Kernel-features/02-Queues-mutexes-and-semaphores/01-Queues.

[17] Espressif Systems. ESP-IDF Programming Guide: FreeRTOS trên ESP32. https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/freertos.html.